LayerTree: A Shared Bitcoin UTXO with an $O(1)$ Unilateral Exit

Olaoluwa Osuntokun

Lightning Labs
laolu@lightning.engineering

2026-10-10
Draft (commit 56c2a1b)
Full version

Abstract. In LayerTree any number of users share one Bitcoin output, and any one of them leaves alone in *one* transaction: the unilateral exit is $O(1)$ in transactions, whatever the number of members, and $O(D)$ in bytes for a tree of depth D (about 32 virtual bytes per level, 1,020 at depth 16). In transaction-tree constructions a user who leaves publishes $O(\log N)$ transactions, the interior must be pre-signed or template-committed, and one unresponsive user forces the whole tree on-chain at expiry. Here the output commits to an off-chain Merkle-sum tree of balances and tapscript covenants enforce every state transition: the operator evicts an expired lease in one transaction too, adding members needs only its signature on the covenant (self-funders still sign their inputs), a cohort is added or swept in one transaction, and the shared output never expires.
We state what the shared output does as an ideal functionality, give an abstract protocol whose covenant predicates realise it, and compile the predicates to four tiers of opcodes already proposed for Bitcoin, some assumed in a tapleaf version their proposals do not define; the smallest tier needs `OP_CAT` and one point-tweak opcode, recovering introspection from a generator-key signature. We prove that the abstract protocol UC-realises the functionality in a global-ledger hybrid model, under a stated assumption on how many competing spends of the shared output can confirm ahead of an honest one, with the tree-level lemmas mechanised in Lean 4. Lightning channels compose with the functionality as leaves whose exit key is shared, so a hub sized by exit throughput occupies one output. A simulator, a byte-exact Go implementation and 2,349 executable checks accompany the design.

Keywords: Bitcoin · covenants · Merkle-sum trees · universal composability · payment channels · formal verification

1 Introduction

LayerTree lets N users share one Bitcoin unspent transaction output (UTXO) and lets any one of them leave alone in a single transaction: the unilateral exit is $O(1)$ in transactions for any N, and $O(D)$ in bytes for a tree of depth D. Payment

channels [37] share an output between two parties; most approaches for *groups* pay for the sharing when it is dissolved. In channel factories [7], Ark [22,38], timeout trees [23,51] and covenant trees built on `OP_CHECKTEMPLATEVERIFY` (CTV) [39] or `OP_CHECKCONTRACTVERIFY` [19,18], the shared output is the root of a tree of *transactions*. A member who leaves unilaterally publishes the path from the root to its leaf, $O(\log N)$ transactions each paying a fee; the interior must exist in advance as pre-signed transactions (every member signs, online) or covenant templates; and when the tree expires, reclaiming the funds of one unresponsive member means publishing everything. Payment-pool opcode proposals (`OP_TAPLEAF_UPDATE_VERIFY` [45], CoinPool's `OP_MERKLESUB` [27], `OP_EVICT` [50]) target one-transaction withdrawal instead, with purpose-built opcodes.

LayerTree takes a different route. The tree stays off-chain and is a tree of *data*: a binary Merkle-sum tree of balances whose root one output commits to. A member exits by presenting its leaf and sibling path to the output itself; the script recomputes the root from the proof, checks it against the commitment, recomputes it with the leaf removed, and requires the transaction to recreate the output under the new root with the leaf's amount paid out. The member needs nobody's signature, and a deeper tree adds bytes to the exit, not transactions. The same covenant lets the operator evict members whose lease has expired, graft a pre-built subtree of new members in one step, and sweep an expired cohort cheaply.

We present the design from the interface inwards: an ideal functionality in the universal composability (UC) framework [8] (§3); an abstract protocol of covenant *predicates* and honest-party behaviour, independent of opcodes (§4); and four realisations by opcode tier (§5), from restored string opcodes (BIPs 440 and 441 [42,43]), introspection (`OP_TX` [41]), stack signature checks (`OP_CHECKSIGFROMSTACK` [5]) and elliptic-curve arithmetic [32,40] down to `OP_CAT` and one point-tweak opcode, plus one built on `OP_CHECKCONTRACTVERIFY` [19]. We propose no opcode for this design, but the tiers are not deployable as proposals stand: FULL, MID and both CCV variants assume opcodes in a tapleaf version their proposals do not define, and two drafts claim the same opcode numbers (§5). The elliptic-curve draft [32] is by the author of this paper. The separation lets the security argument be made once, for the predicates.

Contributions. (1) $\mathcal{F}_{\mathsf{LT}}$, with interfaces for exit, eviction, grafting, update, vouchers and sweeps, and liveness under an explicit contention assumption. (2) An abstract protocol built from one *step* predicate over a Merkle-sum tree. (3) Every predicate compiled in three tiers, and the exit in a fourth (MIN) that rebuilds the BIP-341 message in script and checks it with Poelstra's generator-key signature [35]; an upper bound on the opcodes needed. (4) A UC proof in a global-ledger hybrid model with the tree-level lemmas mechanised in Lean 4, which isolated that sums are conserved unconditionally but exits are guaranteed only for members who verified their path. (5) Lightning channels as leaves, and the conditions under which they compose. (6) A simulator, a byte-exact Go implementation, and measured costs: a 1,020 vB exit at depth 16, 665 vB for a second

exit in the same transaction, 158 to 203 vB per swept leaf. The specification, code and proofs are public [33]; this text describes revision 56c2a1b.

This version. A short version for developers walks through the operations, vouchers and trust model and only states what is proved. This full version is built around $\mathcal{F}_{\mathsf{LT}}$: it carries the UC proof (§7) and the channel-leaf scripts (§9), the Lean statements inline, sections on costs against the alternatives (§11), profiles and radix (§12), batch shapes (§13), grafting (§14) and receiving over Lightning (§15), and the Lean development and script listings (Appendices A and D).

2 Background

Bitcoin. A pay-to-taproot output [47] commits to an internal key P and the Merkle root m of a *taptree* of scripts through $Q = P + H_{\mathsf{TapTweak}}(P \,\|\, m) \cdot G$, and is spent either by a Schnorr signature [46] under Q or by revealing one script with its Merkle path (the *control block*) and satisfying it. Tapscript [48] has no loops, so every "loop" below is unrolled. BIP-441 [43] defines tapleaf version `0xC2`, restoring the string opcodes disabled in 2010 (`OP_CAT`, `OP_SUBSTR`, `OP_LEFT`, `OP_RIGHT`, bit operations) with unsigned numbers of any length, metered by BIP-440 [42] ("varops"); BIP-347 [17] proposes `OP_CAT` alone in the current version, `0xC0`. `OP_TX` [41] pushes fields of the spending transaction, optionally collated into one hash preimage; `OP_CHECKSIGFROMSTACK` (CSFS) [5] verifies a signature over a stack element; `OP_TWEAKADD` [40] computes $x(P + tG)$, and the EC-ops draft [32] adds point arithmetic. All are proposals. Sizes are in virtual bytes (vB), a quarter of a transaction's weight in weight units (WU), where a witness byte weighs one unit and any other byte four. A Merkle-sum tree [25,31] hashes each node's children with their sums, so an inclusion proof proves a leaf's contribution to the root sum.

Composable security. We use the UC framework [8] with global subroutines [10]. A protocol Π *UC-realises* a functionality $\mathcal{F}$ if for every adversary $\mathcal{A}$ there is a simulator $\mathcal{S}$ such that no environment $\mathcal{Z}$ distinguishes Π with $\mathcal{A}$ from $\mathcal{F}$ with $\mathcal{S}$. Following Badertscher et al. [3] the chain is a global ledger $\mathcal{G}_{\mathsf{ledger}}$ that accepts transactions satisfying a validity predicate, orders them at the adversary's discretion within a liveness bound Δ_L, and never reverts them; time is a global clock $\mathcal{G}_{\mathsf{clock}}$ [21] whose rounds are blocks; signatures are the ideal functionality $\mathcal{F}_{\mathsf{sig}}$ [9]; channels follow [2,16].

3 The LayerTree Functionality

A *tree* has one *operator* and any number of *members*. Each member owns a *leaf*: a balance and three keys (an owner key that authorises the member's actions, an exit key paid on exit, and an evictor key, normally the operator's, that may remove the leaf after a lease height). The operator holds no key over the funds, only an *operator key* that authorises additions and co-signs cooperative changes.

Functionality $\mathcal{F}_{\mathsf{LT}}[D, \mathsf{profile}, pk_{\mathrm{op}}, \mathsf{exact}, \Delta]$

State. A *partial* tree T over 2^D slots. A node is *known* (empty, a leaf $\ell = (a, pk_{\mathrm{exit}}, pk_{\mathrm{own}}, pk_{\mathrm{ev}}, h_{\mathrm{ev}}, nonce, seq)$, or an inner node with its two children) or $opaque(h, s)$, a hash and a sum never opened; known nodes may sit below an opaque one where a path has been shown. A is the root's sum; V is the set of outstanding vouchers. Every call is public (caller, arguments, effect). $\mathcal{A}$ decides when, within $\Delta(q)$ rounds of $\mathcal{G}_{\mathsf{clock}}$, an accepted honest call takes effect, q being the honest calls queued ahead of it; effects are atomic and ordered.

On $\mathsf{CREATE}(T_0)$ **from** the operator, once: after $\sum T_0$ coins are deposited on $\mathcal{G}_{\mathsf{ledger}}$, set $T := T_0$, or the opaque root of T_0 if $\mathcal{A}$ withholds it.
On $\mathsf{JOIN}(p, h, u)$ **from** the operator: require node p at height h empty and $\mathsf{profile}$ to admit p; require $\sum u$ coins deposited; set $T[p] := u$, a tree of height h, or its opaque root if $\mathcal{A}$ withholds it.
On $\mathsf{REVEAL}(p, n_L, n_R)$ **from** $\mathcal{A}$: require $T[p]$ opaque and equal to the parent of n_L and n_R; replace it by an inner node with children n_L, n_R (opaque, or leaves at height 1).
On $\mathsf{EXIT}(p, v, f)$ **from** $\mathcal{P}$: require $T[p] = \ell$ known, $\mathcal{P}$ holding pk_{own}, f a coin of $\mathcal{P}$, and $v = a$ if exact else $v \le a$; set $T[p] :=$ empty; post the transaction of Fig. 2, funded by f, paying v to pk_{exit}.
On $\mathsf{EVICT}(p, f)$ **from** $\mathcal{P}$: as EXIT with pk_{ev} in place of pk_{own}, $\mathcal{G}_{\mathsf{clock}} \ge h_{\mathrm{ev}}$, and $v = a$.
On $\mathsf{UPDATE}(p, \ell')$ **from** the owner of $T[p] = \ell$ and the operator jointly (and the holder of $\ell.pk_{\mathrm{exit}}$ in variant UPDATE^x): set $T[p] := \ell'$; require $\ell'.a - \ell.a$ coins deposited if positive, else pay the difference to the callers.
On $\mathsf{VOUCH}(p, \ell_c)$ **from** the owner of $T[p] = \ell$: require $\ell_c.a \le \ell.a$, $\ell_c.pk_{\mathrm{ev}} = \ell.pk_{\mathrm{ev}}$ and $\ell_c.h_{\mathrm{ev}} \le \ell.h_{\mathrm{ev}}$; add (p, ℓ, ℓ_c) to V and return it to the caller.
On $\mathsf{CLAIM}(p, \ell_c, q)$ **from** any party holding $(p, \ell, \ell_c) \in V$ with $T[p] = \ell$: set $T[p] :=$ empty if $\ell_c.a = a$, else ℓ with amount $a - \ell_c.a$ and $seq + 1$; then require $T[q]$ empty and set $T[q] := \ell_c$. (SPLIT is VOUCH and CLAIM by the owner at once.)
On $\mathsf{SWEEP}(p, h)$ **from** $\mathcal{P}$: require every leaf below node p (height h) known and naming $\mathcal{P}$ as evictor with $\mathcal{G}_{\mathsf{clock}} \ge h_{\mathrm{ev}}$; set the subtree empty; pay each leaf's a to its pk_{exit}.
On READ **from** any party: return T and A.

Fig. 1. The ideal functionality. Batched exits (EXIT_K) are K EXIT calls taking effect together. $\mathcal{A}$ may refuse to serve JOIN (there is no liveness for joining), schedules accepted calls within $\Delta(q)$, and may graft opaque subtrees and later reveal them; value under an opaque node that is never revealed is frozen.

Figure 1 gives $\mathcal{F}_{\mathsf{LT}}$; its parameters are the depth D, the addressing *profile* (dense: the operator assigns slots; sparse: a slot is a hash of the owner key and a nonce), the operator key, the payout rule exact (whether an owner may leave part of its balance as fee), and the liveness bound Δ below. $\mathcal{F}_{\mathsf{LT}}$ is protocol-aware, as in Generalized Channels [2]: it builds an honest party's transaction from a fee coin the caller names and posts it.

What the functionality promises. (G1) *Conservation*: A is the sum of T and every payout is funded by a removed leaf. (G2) *Unilateral exit*: the owner of a known leaf is paid its balance with no other party's consent; no party can prevent the exit except by outbidding it, and under [C] below it takes effect within $\Delta(q)$. (G3) *Eviction*: after the lease height the evictor can do the same, and since a carve keeps the evictor and a lease no later, this holds per balance. (G4) *Operator-only membership*: JOIN and SWEEP need the operator and no member. (G5) *No expiry of the shared output*, though each leaf has a lease. Not promised: liveness for joining, and anything for value under an opaque node never revealed (frozen, not stolen); honest members verify their path before funding (§4.4), so their leaves are known.

The liveness bound. Every call spends and recreates one ledger output, and an honest call can be overtaken by any number of valid conflicting spends (a corrupt

member alone can carve its balance back and forth), so the delay does not follow from the ledger model; we state it under an assumption whose parameters we do not derive. *[C] (contention):* at most $\kappa \geq 1$ spends of a tree output confirm per block, and while an honest transition is pending at the prevailing feerate at most β conflicting adversarial spends confirm ahead of it. κ is mempool policy: 25 under Bitcoin Core's chain limits before version 31, at most 64 under its cluster limit since [4,12], and as low as 1 once anyone hangs descendants off a payout or change output. β is economic, the adversary's fee budget against the honest bid; unbounded, it makes exit a fee auction. Under [C] an honest call with q honest calls queued ahead takes effect within $\Delta(q) = \Delta_L(\lceil(\beta + q)/\kappa\rceil + 1)$ rounds, Δ_L being the ledger's liveness parameter. Cooperatively ($\beta = 0$, $\kappa = 25$, $\Delta_L = 1$ block) a tree drains 25 exits per block, so 10,000 channel leaves closing at once take 400 blocks: trees are sized by exit throughput, not depth.

4 The Abstract Protocol

The protocol Π has a data structure whose root one output commits to, one *predicate* per interface, enforced by the ledger, and the behaviour of honest parties.

4.1 The Tree

We write $\mathsf{le}_k(x)$ for the k-byte little-endian encoding of x. A leaf is a fixed 116-byte record

$$\mathsf{le}_{64}(\mathsf{amount}) \,\|\, \mathsf{exit_key} \,\|\, \mathsf{owner_key} \,\|\, \mathsf{evictor_key} \,\|\, \mathsf{le}_{32}(\mathsf{evict_after}) \,\|\, \mathsf{le}_{32}(\mathsf{nonce}) \,\|\, \mathsf{le}_{32}(\mathsf{seq}),$$

with keys as 32-byte x-only points, $\mathsf{evict_after}$ a block height, nonce a per-leaf identifier and seq a counter advanced by vouchers (Appendix C.2). A *node* is a pair (h, s) of a 32-byte hash and an unsigned sum. A position holding leaf ℓ has node $(H_{\mathsf{leaf}}(\ell), \ell.\mathsf{amount})$; an empty position has node $E_0 = (0^{32}, 0)$. The parent of (h_L, s_L) and (h_R, s_R) is

$$\big(H_{\mathsf{node}}(h_L \,\|\, \mathsf{enc}_{64}(s_L) \,\|\, h_R \,\|\, \mathsf{enc}_{64}(s_R)),\ s_L + s_R\big), \qquad \mathsf{enc}_{64}(s) = \mathsf{le}_{64}(s) \,\|\, \mathtt{0x01}.$$

Every hash is a BIP-340 tagged hash with its own tag; $\mathsf{enc}_{64}(s)$ is the minimal BIP-441 encoding of $s + 2^{64}$, produced from a computed sum with one addition (Lemma 1). E_h is the node of the empty tree of height h.

The *sibling path* π of a position is the list of D pairs $(\mathsf{dir}_k, (h_k, s_k))$, one per level, giving the side the position lies on and the sibling node not taken. $\mathsf{fold}(n, \pi)$ combines a node with a path from the bottom up, hashing with each sibling in the order the direction gives and adding the sums. The honest path folds a leaf's node to the root, and folding a *different* node up the *same* path yields the root with that position replaced (Theorem 1).

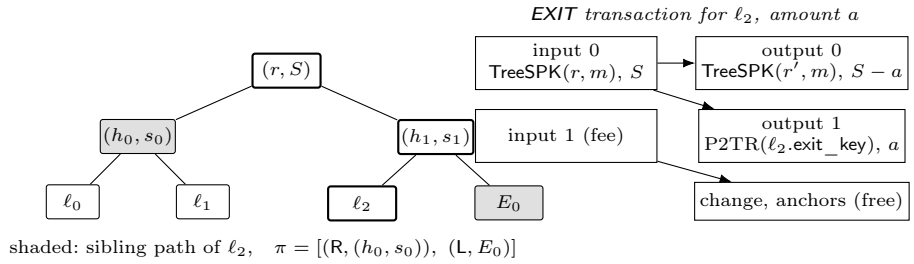


Fig. 2. A depth-2 tree and the exit of ℓ_2. The predicate checks $\mathsf{fold}((H_{\mathsf{leaf}}(\ell_2), a), \pi) = (r, S)$, computes $r' = \mathsf{fold}(E_0, \pi)$, verifies a signature by $\ell_2.\mathsf{owner_key}$, and constrains outputs 0 and 1. Everything else in the transaction is free.

Profiles. In the *dense* profile positions are assigned by the operator and a path carries explicit direction bits; each level costs 88 bytes of script and 41 bytes of witness. In the *sparse* profile a leaf's position is the low D bits of $H_{\mathsf{pos}}(\mathsf{owner_key} \,\|\, \mathsf{nonce})$, direction bits are derived in script from the position hash, and empty siblings are elided from the witness (the script substitutes E_k), so witness data grows with $\log_2 N$ rather than with D. It is the Merkle-sum sparse Merkle tree of Taproot Assets [31] truncated to $D = 32$ or 64, at 1.7 to 2.8 times the dense per-exit cost with 2^{16} members (Table 8). Dense is the default, and the rest of the paper measures it unless it says otherwise.

4.2 The Tree Output

Let r be the root hash and S the root sum. The tree output is a taproot output with internal key $P = \mathsf{lift_x}\big(x(\mathsf{NUMS} + H_{\mathsf{state}}(r) \cdot G)\big)$, where NUMS is the BIP-341 nothing-up-my-sleeve point and lift_x the BIP-340 even-y normalisation, so the key path is unspendable and every spend goes through a script. Its taptree has one leaf per predicate; its Merkle root m identifies the rule set and is fixed for the tree's life. A *tree output* for state (r, S) has scriptPubKey $\mathsf{TreeSPK}(r, m)$ and amount S. Every predicate recomputes $\mathsf{TreeSPK}(r, m)$ from its witness and checks it against the spent input, binding the root the proofs are checked against and the script set the successor carries. FULL, MID and MIN predicates read input 0, so two such trees cannot be spent in one transaction; a CCV tree's input may sit at any index.

4.3 The Step and the Transition Predicates

Every transition spends the tree output and recreates it as output 0 with the new root and total; payouts occupy outputs $1, \ldots, K$; other inputs and outputs are free, for fees, change and anchors. The core is the *step*: $\mathsf{fold}(n_{\mathrm{old}}, \pi) = (r, S)$ with S the input's amount (CHECK_OLD), and $(r', S') = \mathsf{fold}(n_{\mathrm{new}}, \pi)$. A predicate Pred_X is a sequence of steps, each proving its old node against the previous step's root (CHECK_PREV), an authorisation, and a check that output 0 is

Table 1. Transition predicates. Each row is one script leaf under m; σ_k is a transaction signature under key k covering all outputs; steps are written old node → new node.

Interface (predicate)	Steps	Authorisation and outputs
EXIT (EXIT)	$(H_{\mathsf{leaf}}(\ell), a) \to E_0$	σ_{own}; out_1 pays exit_key, $v \le a$ ($v = a$ if exact)
EVICT (EXIT, flag)	same	σ_{ev}, lock time $\ge$ evict_after; out_1 pays exactly a
$K\times$EXIT (EXIT$_K$)	K leaf steps, chained	each step its own σ; $\mathrm{out}_{1..K}$
JOIN (JOIN$_h$)	$E_h \to (h_{\mathrm{new}}, s_{\mathrm{new}})$	σ_{op}; $\mathrm{out}_0 = S + s_{\mathrm{new}}$, funded by other inputs
$K\times$JOIN (JOINK$_K$)	K steps $E_0 \to$ leaf	one σ_{op}; the sparse profile cannot graft
UPDATE (UPDATE)	$(H_{\mathsf{leaf}}(\ell), a) \to (H_{\mathsf{leaf}}(\ell'), a')$	$\sigma_{\mathsf{own}}, \sigma_{\mathsf{op}}$, plus σ_{exit} in UPDATE^x; $\mathrm{out}_0 = S - a + a'$
CLAIM (CLAIM)	source → source′; $E_0 \to \ell_c$	voucher signature checked from the stack; ℓ_c keeps the evictor, lease no later; $\mathrm{out}_0 = S$
CLAIM (CLAIM^h)	same	as above, over a message naming a payment hash, with the preimage in the witness
CLAIM (SPLIT)	same	σ_{own} of the source leaf; no stack signature, so no `OP_CHECKSIGFROMSTACK`
SWEEP (EVICT_SUBTREE$_h$)	subtree, post-order → E_h	σ_{ev}, every lease expired; one payout per leaf

$(S', \mathsf{TreeSPK}(r', m))$ unless the tree is now empty (FINAL). Table 1 gives each interface's predicate; Appendix B gives the step in pseudocode and tapscript.

Signatures must cover all outputs, or a relayer could redirect the fee slack of a non-exact exit. JOIN$_h$ grafts a whole subtree (with $h = 8$, 256 members) for a constant script cost, but cannot look inside the grafted node: this is where opaque nodes come from. EVICT_SUBTREE$_h$ is a shape-specialised multiproof, one path to the subtree and its 2^h leaves hashed once. VOUCH is an off-chain signature by the source leaf's owner over the source leaf, the claim leaf and the tree's lineage identifier tree_id (fixed at creation and inherited by detached cohorts, so a voucher verifies in no other tree), and every carve gives the claim leaf the source's evictor key and a lease no later than the source's, so moving a balance cannot escape eviction; both are spec 0.10.0 rules in every tier (Appendix C.2). FULL also builds CLAIM^x (the exit key consents to a redemption), REWRITE$_h$ (a subtree's published leaves replaced at once, each emptied leaf paid exactly to its exit key) and DETACH$_h$ (a subtree moved to an output of its own).

4.4 Honest Parties

A member stores its leaf and path and follows every spend of the tree output; each witness contains the path it used, so the member updates its own path from the chain alone. Before funding a JOIN or a creation, it verifies its path against the grafted node (or r_0) and the deposit; one who skips the check may end up under an opaque node. A self-funded graft therefore needs every funder to sign one transaction after the cohort is fixed, an abort-and-retry round like an Ark round or a coinjoin; in an operator-funded graft the operator fronts the value and members pay it off-chain. To exit, a member builds the transaction of Figure 2 with a fee input. Its signature commits to the tree outpoint, which every transition changes, so an exit cannot be pre-signed or delegated: the owner

Table 2. Deployment assumptions (spec 4.1, `spec/layertree.md`). A proposal *defines* its opcode for one tapleaf version; a tier may *assume* it in another.

	assumption	tiers
[A-1]	`OP_TX` [41] (a draft on a personal fork, defined for `0xC2` with BIP-440 costs) pushes amounts as 8-byte little-endian strings, which FULL compares with BIP-441 numerics	FULL, MID, CCV
[A-2]	CSFS (BIP-348, defined for `0xC0`) available in `0xC2` at the cost of one signature check	FULL, CCV
[A-3]	the EC-ops draft [32] available in `0xC2` at its stated costs	FULL
[A-4]	opcode numbers: `OP_TX` and `OP_EC_POINT_NEGATE` both claim `0xbd`, BIP-443 and `OP_EC_POINT_ADD` both `0xbb`; the simulator parks `OP_TX`, `OP_TWEAKADD` and CCV at `0xbf`, `0xc0`, `0xc1`, so the golden scripts are byte-exact under that numbering only (sizes are unaffected)	all
[A-5]	`OP_TX` in `0xC0` with `CScriptNum` outputs, which no proposal defines	MID, CCV
[A-6]	BIP-443 (defined for `0xC0`) also in `0xC2`; its sigop cost, open upstream, charged as one signature check; amount rules as of `bitcoin/bips` commit `24e96e87`. Conservation rests on the tier's `OP_TX` equalities, so a change to BIP-443 can only make honest spends fail	CCV
[MIN]	`OP_CHECKSIG` under key G with $s = e + 1$ accepts only if the script-built message is the real one, barring a collision in the TapSighash or challenge hash	MIN
[C]	contention: κ and β of §3 (policy and fees, not consensus)	all

key must be online to rebuild it after a competing spend and to bump its fee. Transitions are final at the member's confirmation depth. The operator keeps the full tree, assigns slots, signs joins and updates, and evicts or sweeps expired leaves.

5 Realisations by Opcode Tier

Each predicate of Table 1 is compiled to tapscript against four opcode sets (Table 3), so that the design's dependence on each candidate opcode is measured rather than asserted; the tree format and the predicates are the same in every tier. FULL, MID and CCV compile every predicate of Table 1, MIN the dense single-leaf exit and SPLIT; CLAIM^x, $\mathsf{REWRITE}_h$ and DETACH_h are FULL's alone. Table 2 lists what each tier assumes of the proposals it uses, mirroring the assumptions [A-1] to [A-6] of the specification. MIN is the tier closest to the proposals as written: it needs only `OP_CAT` (BIP-347) and `OP_TWEAKADD` (BIP-449), both defined for the current tapleaf version. MID and CCV in `0xC0` need `OP_TX` in a version its draft does not define; CCV in `0xC2` needs BIP-443 there; FULL needs CSFS and EC ops in `0xC2` and two drafts whose opcode numbers collide.

FULL and MID. With `0xC2` the leaf is one 116-byte stack element read with `OP_SUBSTR`, sums use arbitrary-length arithmetic, one LEVEL of the fold is 88 bytes, and TreeSPK is recomputed with the EC opcodes; replacing the splice and EC opcodes by `OP_CAT` and `OP_TWEAKADD` costs 2,378 bytes of script at $D = 16$ against 2,101.

Under `0xC0` (MID) fields arrive as separate witness elements, checked with `OP_SIZE` and joined with `OP_CAT`, and a computed sum is widened to 8 bytes by

Table 3. The four opcode tiers. "Shape" says whether inputs and outputs the predicate does not constrain are free or fixed by the script. Sizes are bytes of script / witness data for EXIT_1 in the dense profile at $D = 16$; the last row is the transaction size on the model of §10. MID's sweep fits at most 54 occupied positions at $h = 6$ (stack limit).

	FULL	**MID**	**MIN**	**CCV**
Tapleaf version	`0xC2`	`0xC0`	`0xC0`	`0xC2` or `0xC0`
Beyond BIP-342	`OP_CAT`, splice, bit ops, `OP_TX`, CSFS, EC ops	`OP_CAT`, `OP_TX`, CSFS, `OP_TWEAKADD`	`OP_CAT`, `OP_TWEAKADD`	`OP_CCV` (BIP-443), `OP_TX`
Numerics	any length	4-byte, $< 2^{31}$	4-byte, $< 2^{31}$	as FULL or MID
Owner payout	$\leq$ amount	exact	exact	exact
Predicates	all	all	EXIT_1, SPLIT	all
Vouchers / channels / shape	yes / yes / free	yes / yes / free	no / no / fixed	option / yes / free
EXIT_1 script / witness (B)	2,101 / 902	3,367 / 978	3,640 / 1,175	1,716 / 838
EXIT_1 transaction (vB)	1,020	1,355	1,473	908

a witness-supplied zero pad (Appendix B); a cap of 2^{31} satoshis (21.47 BTC) per tree and exact payouts are the price of 4-byte arithmetic.

MIN: introspection from a signature. Without `OP_TX` the only opcode that sees the transaction is `OP_CHECKSIG`. The script *constructs* the BIP-341 signature message from values it has derived (the recomputed $\mathsf{TreeSPK}(r, m)$, the input amount, the new tree output, the payout) and raw witness pieces for what it does not constrain, and demands a Schnorr signature under the generator G with nonce G, Poelstra's construction [35]: the challenge is $e = H_{\mathsf{challenge}}(G_x \,\|\, G_x \,\|\, \mathsf{msg})$, and $sG = (1 + e)G$ has the unique solution $s = e + 1 \bmod n$. The script computes e from its message and calls `OP_CHECKSIG` with signature $G_x \,\|\, s$ and key G_x; since `OP_CHECKSIG` hashes the real transaction, it passes only if the two messages are equal, which is assumption [MIN]. Without long arithmetic, $e + 1$ is formed by grinding the fee input's sequence number until e ends in `00` (256 trials on average, repeated on every fee bump; Appendix B). What is new is the use: the field-by-field enforcement and the signature as a tier's only introspection. EXIT_1 costs 3,640 bytes of script at $D = 16$ (73% more than FULL) and 1,473 vB (44% more); vouchers and channel leaves need a signature over stack data and are out of reach, SPLIT is not (6,651 bytes). Whether a 64-leaf sweep (16 kB in FULL) fits MIN is open.

CCV: binding by opcode. `OP_CHECKCONTRACTVERIFY` [19] replaces every taproot-key recomputation of FULL: one call binds the old root to the input under the input's own taptree, one commits output 0 to the new root with the residual assigned, one per payout asserts a P2TR of the leaf's exit key and deducts its amount. The state key becomes the BIP-443 data tweak of NUMS by the root; the fold and the authorisation are FULL's, in either numeric encoding, and r and m leave every witness. One `OP_TX` read per constrained amount survives, hence exact payouts.

6 Security

The tree-level facts are theorems about the abstract predicates, mechanised in Lean 4 [26] in about 3,400 lines without Mathlib (Appendix A) and cited by name; the composable claim is a simulation argument in the hybrid model of §2, sketched here and given in §7. We work in the idealised hash model: H_{leaf} and H_{node} are injective on the domains the protocol uses, and digests under different tags never coincide or equal the zero string. Predicates and bytecode correspond by execution (§10), not proof.

6.1 Tree-Level Lemmas

Lemma 1 (Encodings). *For $s < 2^{64}$ the minimal BIP-441 encoding of $s + 2^{64}$ is $\mathsf{le}_{64}(s) \,\|\, \mathtt{01}$; leaf and node serialisations are injective; 2^{32} has a five-byte minimal encoding and 2^{31} a five-byte `CScriptNum`, while $-(2^{31}-1)$ is* `ff ff ff ff` *with successor* `fe ff ff ff`. *(enc64_eq_minimal, serializeLeaf_inj, serializeNode_inj, minimalLE_two_pow_32_length, scriptNum_two_pow_31_length, scriptNum_neg_max31_succ.)*

Theorem 1 (Binding). *Roots bind trees: two trees of the same depth with equal roots are equal. Any node n and path π with $\mathsf{fold}(n, \pi)$ equal to the root of a tree T are the root of one of T's subtrees and that subtree's honest path; in particular the sum in n is the true sum of that subtree, and a leaf node that verifies with a full-length path is the leaf at that position. (root_inj, fold_sound.)*

```
theorem root_inj {d} (t₁ t₂ : Tree d) : (t₁.root : Node H) = t₂.root → t₁ = t₂
theorem fold_sound {d} (t : Tree d) (m : Node H) (π : List (Bool × Node H)) :
    fold m π = t.root → ∃ (h : Nat) (p : Pos d h), m = (t.getSub p).root ∧ π =
      t.pathSub p
theorem fold_setSub {d h} (t : Tree d) (p : Pos d h) (s : Tree h) :
    fold s.root (t.pathSub p) = ((t.setSub p s).root : Node H)
```

Proof (sketch). Induction on the path. A non-empty path claims an internal node, so the tree is not a tip by tag separation. Injectivity of H_{node} on the parent's preimage equates both children's hashes *and* sums with the claimed ones, and the induction hypothesis applies to the child the direction bit selects. This is the only place the hash axioms are used.

Lemma 2 (A step is a subtree replacement). *If the current root is the root of a tree T and a well-formed step is valid from it to r', its old node is the root of T's subtree at some position p of the step's height, and r' is the root of T with that subtree replaced by any tree whose root is the step's new node. (Step.sound.)*

Theorem 2 (Conservation). *For any sequence of valid steps from (r, S) to (r', S'), $S' + \sum s_{\text{old}} = S + \sum s_{\text{new}}$; for EXIT_K, $S' + \sum_k \text{payout}_k \le S$, with equality under exact payouts. No hash assumption is needed. (Steps.sum, ExitK.no_inflation.)*

```
theorem Steps.sum {r r' : Node H} {ss : List (Step H)} (h : Steps r ss r') :
    r'.sum + oldSum ss = r.sum + newSum ss
theorem ExitK.no_inflation {exact} {r r'} {blocks} {out0} (h : ExitK exact r blocks r'
     out0) :
    out0 + payoutSum blocks ≤ r.sum      -- ExitK.exact_conservation: equality if exact
```

Theorem 3 (Payability). *For any tree T and position p holding leaf ℓ, the honest witness (the leaf, its path, E_0) is a valid step from T's root to the root of T with p emptied, the sum falls by ℓ.*amount*, and every other position is unchanged. (payable.)*

```
theorem payable {d} (t : Tree d) (p : Pos d 0) (l : Leaf) (hl : t.get p = some l) :
    (leafStep (t.path p) (some l) none).valid t.root (t.set p none).root ∧
    (t.set p none).root.sum + l.amount = t.root.sum ∧ ∀ q, q ≠ p → (t.set p none).get q
     = t.get q
```

Theorem 4 (Real-tree invariant). *If the current root is the root of a tree and every step is well formed and* grounded *(its new node is the root of some tree of the step's height), the final root is the root of a tree of the same depth. Leaf-level steps are grounded by construction; a JOIN graft is grounded exactly when the operator's node has a preimage tree. (Steps.sound, join_sound.)*

```
theorem Steps.sound {d} (t : Tree d) {r'} {ss : List (Step H)} (h : Steps t.root ss r')
    (hwf : ∀ s ∈ ss, s.WF d) (hg : ∀ s ∈ ss, s.Grounded) : ∃ t' : Tree d, t'.root = r'
theorem join_sound {auth : Prop} {d h} (t : Tree d) (path) (new r' : Node H) (out0 : Nat)
    (hwf : path.length + h = d) (hj : Join auth h t.root path new r' out0) :
    ∃ p : Pos d h, t.getSub p = Tree.empty h ∧ out0 = t.root.sum + new.sum
      ∧ ∀ u : Tree h, u.root = new → r' = (t.setSub p u).root
```

Lemma 3 (Path maintenance, not mechanised). *If leaf ℓ at p with path π folds to the root r, and a valid step at a position p' that is neither p nor above it takes r to r', then ℓ folds to r' with the path π in which the one sibling on the step's path is replaced by the node the step computes at that height.*

Lemma 3 needs no real tree, so it covers a path through an opaque node; §7 proves it on paper. Other transitions have theorems of their own (Appendix A). Authorisation and lock times are an opaque proposition auth in the theorems, discharged by $\mathcal{F}_{\mathsf{sig}}$ and $\mathcal{G}_{\mathsf{clock}}$; which key a payout pays (the leaf's exit key, not the evictor) is established by execution, not by these theorems. The predicates bind the witness root and taptree to the spent output through $\mathsf{TreeSPK}(r', m') = \mathsf{TreeSPK}(r, m)$ (witness_root_binds), which reduces to the discrete logarithm of NUMS and hash collisions when the two internal keys have opposite signs, and to taproot binding in the random-oracle model, as in BIP-341's own argument, when they share one (Appendix A).

6.2 The Composable Statement

Let Π_τ be the protocol with the predicates compiled by tier τ, and $\mathcal{G}_{\mathsf{ledger}}[\mathsf{Pred}_\tau]$ the global ledger whose validity predicate, for a spend of a tree output, is that tier's predicate. Let $\mathcal{F}_{\mathsf{LT}}|_\tau$ be $\mathcal{F}_{\mathsf{LT}}$ restricted to what τ compiles into m: the

interfaces of Table 3, the heights h and batch sizes K in the taptree, exact and amounts below 2^{31} in MID and MIN (exact in CCV), no graft in the sparse profile, at most 54 occupied positions per MID sweep, and for MIN only the dense one-leaf EXIT and SPLIT, without the terminal exit. Trees whose taptree carries $\mathsf{REWRITE}_h$ or DETACH_h are excluded: neither is an interface of $\mathcal{F}_{\mathsf{LT}}$, and their Lean theorems are all we claim for them.

Theorem 5 (UC realisation). *In the idealised hash model, with taproot binding in the ROM and under [C], for each tier τ, Π_τ UC-realises $\mathcal{F}_{\mathsf{LT}}|_\tau$ with $\Delta(q) = \Delta_L(\lceil(\beta+q)/\kappa\rceil+1)$, in the hybrid model with $\mathcal{G}_{\mathsf{ledger}}[\mathsf{Pred}_\tau]$, $\mathcal{G}_{\mathsf{clock}}$ and $\mathcal{F}_{\mathsf{sig}}$ (in MIN, under [MIN]), against a static adversary corrupting the operator and any members, for trees without $\mathsf{REWRITE}_h$ and DETACH_h.*

Claim 1 (Simulation invariant). *After each ledger event, $\mathcal{F}_{\mathsf{LT}}$'s partial tree has the root the tree output commits to, and agrees with every honest party's leaf and path.*

The proof is the next section.

What the theorem leaves out. The bytecode is matched to Pred_τ by 2,349 executable checks, not a verified compiler. Fee pinning is outside the ledger model: a low-feerate transition early in the chain caps the feerate behind it, a descendant hung off any payout or change output consumes the chain's policy limit, and TRUC transactions [49] cannot help, since they cap an unconfirmed chain at two; [C] assumes this away, and cooperative batching, anchors and sizing by exit throughput mitigate it. The operator can evict any leaf after its lease, a cohort at about 160 vB per leaf, so griefing is cheap; a longer guarantee needs a later lease at entry, and no carve can extend it.

7 The UC Proof

This section gives the simulator and the hybrid argument behind Theorem 5. We follow the conventions of the composable ledger treatment of Badertscher et al. [3] and of Generalized Channels [2]. $\mathcal{F}_{\mathsf{LT}}$ is protocol-aware: for an honest EXIT or EVICT it constructs the transaction of Figure 2 itself, from the honest party's leaf and path and the fee coin f named in the call, obtains the signature through $\mathcal{F}_{\mathsf{sig}}$, and submits it to $\mathcal{G}_{\mathsf{ledger}}$. The simulator never builds an honest transaction; it only schedules. The two worlds therefore agree on the ledger contents and differ only in how the tree state is kept.

7.1 The Hybrid World

The real protocol Π_τ runs in the presence of:

$\mathcal{G}_{\mathsf{ledger}}[\mathsf{Pred}_\tau]$ the global ledger functionality of Badertscher et al., parameterised by a validity predicate that, for a transaction spending a tree output, evaluates

the tier's covenant predicate Pred_τ on the input state, the witness and the outputs (§4). It guarantees persistence, and liveness with parameter Δ_L: a valid transaction with sufficient fee that an honest party submits, and that stays valid, is in the ledger state of every honest party within Δ_L rounds. The adversary orders transactions within the rules, and may get a conflicting spend in first, which invalidates the honest one; [C] bounds how often.

$\mathcal{G}_{\mathsf{clock}}$ the global clock of Katz et al. [21]; rounds are blocks.

$\mathcal{F}_{\mathsf{sig}}$ the ideal signature functionality [9], which we use in place of an EUF-CMA reduction; every `CHECKSIG` and `CHECKSIGFROMSTACK` in the scripts is a call to its verification interface, with one exception: MIN's `OP_CHECKSIG` under the generator G, whose key has no honest signer, is real BIP-340 verification, and its binding is assumption [MIN] (Table 2).

Hashes are treated in the idealised model of §6 (injectivity plus tag separation), and the taproot commitment in the random-oracle model where the two internal keys share a sign (Appendix A). Every abort event below is one in which the simulator holds two distinct preimages of one digest, so the argument transfers to a collision-resistance reduction.

7.2 The Simulator and the Invariant

$\mathcal{S}$ runs an internal copy of $\mathcal{A}$ and of every honest party's protocol state, and keeps a partial tree $\hat{T}$ equal to $\mathcal{F}_{\mathsf{LT}}$'s. Claim 1 is the invariant: after each ledger event, $\hat{T}$ has the root the tree output commits to, and every honest party's leaf is a known node of $\hat{T}$ whose path folds to that root.

Ledger events. Whenever a transaction tx spending a tree output appears in the ledger, $\mathcal{S}$ parses its witness:

1. Identify the script leaf from the control block; this fixes the transition type and its parameters (K, h, profile, variant).
2. For each step, fold the supplied node up the supplied path. Where the path passes through an opaque node of $\hat{T}$, call REVEAL for each level with the nodes the witness supplies (they hash to the opaque node because the fold reaches the committed root); where it passes through a known node and the fold disagrees with it, $\mathcal{S}$ holds two preimages of one digest (abort event Coll). Otherwise Theorem 1 gives the unique position p the step proves.
3. Map the steps to the interface: a leaf replaced by E_0 is EXIT (owner flag) or EVICT (evictor flag); E_h replaced by a node is JOIN, with the subtree $\mathcal{S}$ obtains from the operator's join messages, or its opaque root; a leaf replaced by a leaf is UPDATE; the two-step carve is VOUCH followed by CLAIM (for SPLIT, or a voucher a corrupt owner issued) or CLAIM of a recorded voucher; the post-order pattern is SWEEP.
4. Check the authorising signatures with $\mathcal{F}_{\mathsf{sig}}$. A valid signature under an honest key that the honest party did not produce is a forgery (abort event Forge).
5. If tx is an honest call that $\mathcal{F}_{\mathsf{LT}}$ posted, let it take effect; otherwise invoke the interface on behalf of the corrupt party that authorised it.

Honest calls. When an honest party calls an interface, $\mathcal{F}_{\mathsf{LT}}$ builds and posts the transaction and $\mathcal{S}$ chooses, within $\Delta(q)$, when it takes effect, matching the round in which $\mathcal{G}_{\mathsf{ledger}}$ includes it. When a corrupt operator grafts or creates a node whose preimage $\mathcal{S}$ cannot obtain, $\hat{T}$ holds it as opaque; an honest member never funds such a node without verifying its own path, and $\mathcal{S}$ reveals that path from the member's state, so the member's leaf is known.

The invariant, by case. Assume Claim 1 before an accepted spend. *Exit, evict, update, carve.* Each step is valid from the committed root, so by Lemma 2 the new root is the root of $\hat{T}$ with the step's position replaced, which is $\mathcal{F}_{\mathsf{LT}}$'s tree after the call; Theorem 2 gives that the amounts agree, and the bytecode that the destinations do. A drained carve source becomes E_0 in both ($\mathsf{claimedSource}$). *Join and create.* The new node is placed at an empty position ($\mathsf{join_sound}$), known or opaque. *Sweep.* $\mathsf{evictSubtree_sound}$ gives the emptied subtree and one payout per leaf. *Reveal.* It changes no root. In every case an honest party's leaf outside the changed position keeps a valid path by Lemma 3, and one inside it was removed by its owner's or evictor's own call.

Proof of Lemma 3. Let the step at p' have old node n, path π_s and new node n', and let k be the height at which the paths of p and p' meet. Both $\mathsf{fold}(\ell, \pi)$ and $\mathsf{fold}(n, \pi_s)$ equal r; reading the two folds from the root down, injectivity of H_{node} equates, level by level, the two parents' preimages, so the siblings above height k coincide, ℓ's sibling at height $k-1$ is the node the step's fold reaches at that height, and vice versa. Folding n' up π_s changes only that node and the nodes above it; replacing ℓ's sibling at that level by the new node and folding with the unchanged siblings above therefore yields r'. Below height $k-1$ nothing on ℓ's path changed. No real tree is needed, only the two folds.

7.3 Hybrids

$\mathcal{H}_0$ the real execution of Π_τ with $\mathcal{A}$ in the $(\mathcal{G}_{\mathsf{ledger}}, \mathcal{G}_{\mathsf{clock}}, \mathcal{F}_{\mathsf{sig}})$-hybrid world.

$\mathcal{H}_1$ as $\mathcal{H}_0$, but the execution aborts on Forge. Indistinguishable by the definition of $\mathcal{F}_{\mathsf{sig}}$.

$\mathcal{H}_2$ as $\mathcal{H}_1$, but the execution aborts on Coll, on a taproot-commitment collision, and in MIN on a violation of [MIN]. In the idealised hash model and the ROM these have negligible probability; in each, the simulator outputs the colliding preimages.

$\mathcal{H}_3$ as $\mathcal{H}_2$, but the honest parties' tree state is replaced by $\hat{T}$ and their transactions are built and posted by $\mathcal{F}_{\mathsf{LT}}$. The invariant above makes the two executions identical transaction by transaction; payability (Theorem 3, and Lemma 3 for a path through an opaque node) gives that every honest exit $\mathcal{F}_{\mathsf{LT}}$ builds is valid.

$\mathcal{H}_4$ the ideal execution of $\mathcal{F}_{\mathsf{LT}}$ with $\mathcal{S}$. Identical to $\mathcal{H}_3$ by construction.

Liveness. $\mathcal{F}_{\mathsf{LT}}$ lets $\mathcal{S}$ delay an honest call by up to $\Delta(q)$. In the real world the honest transaction confirms unless a conflicting spend confirms first; under [C] at most κ spends of the output confirm per block, at most β of the conflicting ones are adversarial, and q honest calls are queued ahead, so after at most $\lceil(\beta+q)/\kappa\rceil$ blocks of Δ_L rounds each the honest party's rebuilt transaction, valid by Lemma 3, is included within one further Δ_L. Nothing beyond [C] is claimed: the model does not bound β, and pinning (§6) is one way it grows.

7.4 Per-Tier Remarks

The argument is the same for every tier because the predicates are the same; what differs is $\mathcal{F}_{\mathsf{LT}}|_\tau$ (§6) and one detail each of MIN and CCV. In MIN, CHECK_OLD and FINAL are enforced through the constructive signature check of §5, so $\mathcal{S}$ recovers the enforced fields from the witness's raw pieces rather than from `OP_TX` selectors, and their binding is [MIN]. In CCV the witness carries neither r nor m: the input's commitment to the old root and output 0's to the new one are checked by `OP_CHECKCONTRACTVERIFY` under the input's own taptree, so $\mathcal{S}$ reads r and m from the spent output, and the binding of witness_root_binds rests on the BIP-443 data tweak in place of TreeSPK (CCVTweak.binding, ccv_input_binds). In every tier the voucher message names the lineage's tree_id (spec 0.10.0), so a voucher for one instance of $\mathcal{F}_{\mathsf{LT}}$ does not verify in another, and $\mathcal{F}_{\mathsf{LT}}$'s VOUCH records it per instance. $\mathsf{REWRITE}_h$ and DETACH_h are outside the theorem (§6).

7.5 Composition for Channel Leaves

Conjecture 1 would follow the pattern of Generalized Channels [2], in the hybrid model with $\mathcal{F}_{\mathsf{LT}}$, $\mathcal{G}_{\mathsf{ledger}}$, $\mathcal{G}_{\mathsf{clock}}$ and $\mathcal{F}_{\mathsf{sig}}$. The channel simulator would reuse the channel-level simulator of that work for the update and settlement transactions, which are ordinary ledger transactions once the funding output exists, and use $\mathcal{F}_{\mathsf{LT}}$'s EXIT and EVICT to materialise it. The three tree-layer preconditions of §8 are the conditions under which $\mathcal{F}_{\mathsf{LT}}$ guarantees that the materialised output carries the full capacity to the channel key: exact payouts (ExitK.exact_conservation), non-claimability (the SEQ_FINAL facts), and a lease at or below the tip so that EVICT is available to the counterparty. The missing pieces are the HTLC deadline argument under [C] and the rebinding of pre-signed states across tree transitions; they are exercised block by block on the simulator's chain model (§9), not proved.

8 Channel Leaves as Composition

A leaf whose exit key is a 2-of-2 between its owner and a counterparty (normally the operator) can host a payment channel that exists only off-chain and whose funding output materialises at the payout of the EXIT or EVICT that removes the leaf, so a tree of such leaves is a Lightning hub behind one UTXO. The channel protocol runs in the $\mathcal{F}_{\mathsf{LT}}$-hybrid model and treats the payout as its funding transaction; §9 gives scripts, state machine and costs. The funding outpoint

changes with every tree transition, so states are signed without committing to it (`OP_TX` collates the signed fields behind a per-channel context, and CSFS checks the signature over the hash, emulating `SIGHASH_ANYPREVOUT` [14]), and the channel uses the eltoo ratchet [13,44] with zero-fee version-3 transactions and an ephemeral anchor [49]. This needs `OP_TX` and CSFS: FULL, MID, or CCV with the channel scripts in their own outputs.

Deadlines and hub size. Settlement outputs include one per in-flight hashed time-locked contract (HTLC). A forwarding node keeps $\mathsf{cltv_expiry_delta}$ blocks between an incoming and an outgoing HTLC's expiry, and a channel leaf needs $\mathsf{cltv_expiry_delta} \geq G_{\text{exit}}+3+\mathsf{csv_delay}+G_{\text{sweep}}$, with $\mathsf{csv_delay}$ the settlement's relative delay, G_{sweep} the time to confirm an HTLC sweep and G_{exit} the time to confirm the exit, $\Delta(q)$ under [C]. With $\mathsf{csv_delay} = 144$, $G_{\text{sweep}} = 2$ and $G_{\text{exit}} = 72$ this is about 220 blocks, against 40 to 80 on the public network. At $\Delta_L = 1$, $G_{\text{exit}} = 72$ needs $\beta + q \leq 71\kappa$; with $\kappa = 25$ and $\beta = 0$, about 1,775 channel leaves may close in one window, and larger hubs shard. The counterparty is normally the operator, which can issue competing spends, so the budget holds only while the member can outbid them, which is what β assumes. MID allows seven HTLCs per state (the 520-byte element limit) and 2^{31} satoshis per tree.

Composition conditions. Let $\mathcal{F}_{\mathsf{chan}}$ be a channel functionality in the style of Generalized Channels [2]. Three of $\mathcal{F}_{\mathsf{LT}}$'s interfaces are too lenient for a leaf whose balance is not the owner's alone, and each becomes a precondition: (i) exact is set, so the owner cannot underfund the channel; (ii) seq is the value whose successor does not fit in four bytes (`0xFFFFFFFF` under BIP-441 numerics, `0x7FFFFFFF` under `CScriptNum`, Lemma 1), so the owner cannot carve the leaf; (iii) the lease height is zero, giving the counterparty EVICT (between two members, UPDATE^x instead).

Conjecture 1. Under [C], in the $(\mathcal{F}_{\mathsf{LT}}, \mathcal{G}_{\mathsf{ledger}}, \mathcal{G}_{\mathsf{clock}}, \mathcal{F}_{\mathsf{sig}})$-hybrid model with conditions (i) to (iii), the channel-leaf protocol realises $\mathcal{F}_{\mathsf{chan}}$ with closing bound $\Delta(q) + 3 + \mathsf{csv_delay}$ for channels whose HTLC deadlines respect the budget above.

The proof obligations are the three conditions, the HTLC deadline argument under [C], and the rebinding of pre-signed states across tree transitions (§7); the state machine and the budget run end to end on the simulator's chain model, with two blocks of margin, but are not proved.

9 Channel Leaves: Scripts and Costs

Selectors and context. Two `OP_TX` selectors are used, both with the collate flag so that one call leaves one blob: $\mathsf{SEL_CHAN_UPD}$ selects version, number of inputs, the current input's sequence, number of outputs, all output amounts and

REBIND_n (0xC2, 81 B)	SETTLE_n (0xC2, 79 B)
$\langle S_0 + n + 1 \rangle$ OP_CHECKLOCKTIMEVERIFY OP_DROP	$\langle$csv_delay$\rangle$ OP_CHECKSEQUENCEVERIFY OP_DROP
$\langle$SEL_CHAN_UPD$\rangle$ OP_TX // collated blob	$\langle$SEL_CHAN_SET$\rangle$ OP_TX // + amount, spk
$\langle \mathsf{ctx}_{\mathsf{upd}} \rangle$ OP_SWAP OP_CAT OP_SHA256	$\langle \mathsf{ctx}_{\mathsf{set}} \rangle$ OP_SWAP OP_CAT OP_SHA256
$\langle P_{\mathsf{chan}} \rangle$ OP_CHECKSIGFROMSTACK	$\langle P_{\mathsf{chan}} \rangle$ OP_CHECKSIGFROMSTACK

Fig. 3. The two tapleaves (version 0xC2) of a channel leaf's update output for state n, sized at $\mathsf{csv_delay} = 144$; under 0xC0 SETTLE_n is 80 B, since 144 needs two bytes as a CScriptNum.

scripts, and lock time; SEL_CHAN_SET adds the current input's amount and scriptPubKey. The per-channel context is

$$\mathsf{chan_id} = H_{\mathsf{chanid}}(m \,\|\, \mathsf{owner_key} \,\|\, \mathsf{le}_{32}(\mathsf{nonce}) \,\|\, x(P_{\mathsf{chan}}) \,\|\, \mathsf{le}_{64}(C) \,\|\, \mathsf{le}_{32}(\mathsf{epoch}) \,\|\, \mathsf{le}_{32}(\mathsf{csv_delay})),$$

with $\mathsf{ctx}_{\mathsf{upd}}$ and $\mathsf{ctx}_{\mathsf{set}}$ two tagged hashes of it, so that an update signature never verifies where a settlement signature is required. The signed messages are $\mathsf{SHA256}(\mathsf{ctx} \,\|\, \mathtt{OP_TX}(\mathsf{SEL}))$.

Update output scripts. The update transaction for state n pays its whole input value to an output U_n with internal key P_{chan} (a MuSig2 [29,28] key of the two parties) and the two-leaf taptree of Figure 3. A later update binds to U_n through REBIND_n, whose lock-time check admits only states above n; the settlement of state n binds through SETTLE_n after csv_delay blocks, and only to U_n because SEL_CHAN_SET commits the spent script, which embeds n.

Transactions. The update transaction is version 3, lock time $S_0 + n$, one input with sequence 0xFFFFFFFD, one output of the full capacity C to U_n and a zero-value pay-to-anchor output; it pays no fee and is valid whether bound to the funding output or to any earlier update output, both of value C. The settlement is version 3 with input sequence csv_delay, outputs for the two balances and each in-flight HTLC, and an anchor; $b_U + b_V + \sum \mathrm{htlc} = C$. Fees for both come from a child spending the anchor, so both parties need an on-chain fee reserve to close unilaterally. The signing order is settlement before update for each state, as in eltoo.

Force-close cost. At $D = 16$ dense, $\mathsf{csv_delay} = 144$ and no HTLCs in flight:

step	vB earliest
EXIT_1 by owner or evictor, with fee input and change	$\approx$ 1,020 t
update$_n$ bound to the funding output	153 $t+1$
anchor child for update$_n$	152 $t+1$
settlement$_n$ (two payouts, anchor)	204 $t+1+\mathsf{csv}$
anchor child for settlement$_n$	152 $t+1+\mathsf{csv}$
total	$\approx$ 1,681 $\approx$ 146 blocks
contested rebind to an earlier update output	161 per contest
settlement with h HTLC outputs	$204+43h$
HTLC-success sweep	155 after settlement
HTLC-timeout sweep	138 after $\mathsf{cltv_expiry}$

An HTLC output has two leaves, so each sweep carries a 65-byte control block. The lifetime on-chain footprint of a channel leaf that is force-closed is therefore about 1,680 vB. For a simple taproot channel [34] priced the same way, the funding transaction (one P2TR key-path input, the funding output and change) is 154 vB, the commitment (a MuSig2 key-path input, two balance outputs and two anchors) 240 vB, its anchor child 152 vB and the delayed sweep of the broadcaster's output 138 vB: about 680 vB in all, with both constructions carrying their fee-bumping children (BOLT 3's anchor commitment [24] is 1,124 WU, 281 vB, in the non-taproot case). Cooperative closes, splices and lease renewals of a channel leaf cost one tree transition or nothing. The update transaction cannot be packaged with the exit because a version-3 transaction may not spend an unconfirmed version-2 parent, so G_{exit} in §8 is measured to confirmation, and the costs above are for a tree of the occupancy that budget allows.

Limits and execution. Under `0xC0` every stack element is at most 520 bytes, including the collated settlement blob of $156+43h$ bytes, so a MID channel carries at most seven HTLC outputs per state; FULL has no such limit. The scripts and transactions above run, in both tapleaf versions, on a chain model that enforces BIP-65 and BIP-68 (210 checks): a leaf leaves a tree by $\mathsf{EXIT}_1^{=}$, a state signed before the exit binds to whichever funding output an exit or an eviction creates, a stale update is answered in the last block of its window, and both HTLC sweeps confirm at their deadlines. Counted as those rules count, the budget of §8 has two blocks of margin: with $G_{\text{exit}} = 72$ and $G_{\text{sweep}} = 2$ the payee's sweep confirms at $t + 219$ against a bound of 221. A Go mirror generates the same scripts byte for byte, with channel messages in Lightning's type-length-value (TLV) encoding.

10 Evaluation

Implementation. A Python tapscript interpreter for both leaf versions models BIP-441 numerics, `OP_TX`, CSFS, the EC opcodes and varops, computes the real BIP-341 signature message and enforces `MINIMALDATA`; it models consensus limits, not standardness. Every compiled predicate runs against a reference tree with positive and negative cases per rule (forged leaf, stale root, attacker-chosen m,

Table 4. Batch shapes at dense $D = 16$ on the model above: vB per leaf that leaves or changes, or per member that joins. Self-funded joins add each member's P2TR key-path funding input (58 vB) and the signing round of §4.4. The $\mathsf{REWRITE}_h$ rows are FULL only.

removal	leaves	tx	per leaf	addition or change	leaves	tx	per leaf
EXIT_1	1	1,020	1,020	JOIN_0	1	937	937
EXIT_2	2	1,684	842	JOIN_2	4	872	218
$\mathsf{EVICT_SUBTREE}_4$	13	2,638	203	JOIN_8 (operator-funded)	256	679	2.7
$\mathsf{EVICT_SUBTREE}_6$	51	8,057	158	sparse JOIN_0, $D = 32$	1	1,601	1,601
$\mathsf{REWRITE}_2$, all leave	4	1,448	362	$\mathsf{REWRITE}_6$, keys rotated	64	9,384	147
$\mathsf{REWRITE}_6$, all leave	64	10,280	161	UPDATE, one leaf	1	1,030	1,030

unconserved amount, early eviction, voucher replay, and others): 1,054 FULL, 268 MID, 55 MIN and 972 CCV checks, failing if a script uses an opcode outside its tier, plus 210 for channel leaves and 61 for the hypothetical opcode of §17. A Go implementation on `btcd` and `lnd` generates the same scripts, byte-exact against 79 golden scripts, and builds, signs and verifies every FULL transition (42 cases); its BIP-341 signature hash equals the simulator's.

The transaction model. Every vB figure is a whole transaction: script, witness elements and a 129-byte control block (the eight-leaf reference script set) as measured, plus a 946 WU skeleton (the tree input, a P2WPKH fee input, the tree output, one P2TR payout, a P2WPKH change output, version, lock time and counts) and 172 WU per further payout, with vB $= \lceil \mathrm{WU}/4 \rceil$; transitions without a payout (joins, updates, carves) are charged 172 WU less. The witness item count and per-item length prefixes are excluded (on a dense $D = 16$ exit, 10.5 vB in FULL, 10 in CCV, 42.5 in MID, 44.75 in MIN), and the baselines omit the same overhead. A deployment that carries the sweeps, rewrites and variants needs 16 to 32 tapleaves, which adds 8 to 16 vB to every spend.

Exit cost. A dense exit (Figure 4, Table 8) costs 762 vB at $D = 8$ and 1,149 vB at $D = 20$, about 32 vB per level. One tier step costs as much as 10 (MID, +33%) to 14 (MIN, +44%) levels of depth. The sparse profile at $D = 32$ with 2^{16} members costs about 1,780 vB, 1.75 times the dense profile at $D = 16$; its witness grows with $\log_2 N$, not D.

Comparison with transaction trees. The dashed line of Figure 4 is a *model*: an exiter publishes D transactions of about 150 vB (one key-path input, two P2TR outputs) and a 111 vB claim, anchors and their fee children omitted, which favours the tree. LayerTree is about 60% smaller at $D = 16$; a radix-4 tree needs 2,031 vB, and an Ark member of a 256-member round unrolls 1,311 vB. Under mass exit a tree's interior is shared, about 261 vB per member at radix 2 and 191 vB at radix 4 with the claim, against 161 vB for a cohort leaving by $\mathsf{REWRITE}_6$ (Table 10, Appendix C). The model prices neither the latency of D sequential confirmations nor the larger opcode set.

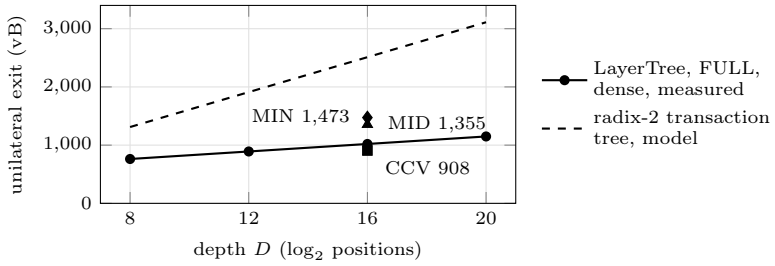


Fig. 4. Unilateral exit cost against tree depth: measured for LayerTree (FULL, and the other tiers at $D = 16$), modelled for an unamortised radix-2 transaction tree.

Batching and joins. A second exit in the same transaction adds 665 vB (its marginal weight rounded up on its own, one more than $1{,}684 - 1{,}020$), against 158 to 161 vB per leaf for a sweep or a rewrite (Table 4). A graft costs the operator under 3 vB per member, so a self-funded member's own input dominates its entry. Off-chain, a member follows every spend of the tree output (about 1 kB of witness each), and the operator stores the whole tree.

Fees and budgets. At 10 sat/vB a $D = 16$ exit costs 10,200 sat, a 1% fee on 1,020,000 sat (Appendix C). Varops usage is 5.6 to 7.7% of the BIP-440 budget in the dense profile and 3.6 to 4.7% in the sparse ($D = 64$ to 32), falling with depth because the budget scales with weight; `0xC0` tiers use one to three signature operations of the BIP-342 sigop budget.

Artefact. Every number here is regenerated from the repository [33] at the revision on the title page (56c2a1b) by the scripts under `tools/`: `test_scripts.py` (1,054 checks; `--rewrite` and `--subtree` print the cohort shapes), `test_mid.py` (268), `test_min.py` (55), `test_ccv.py` (972), `test_chan.py` (210), `test_msu.py` (61), `gen_listings.py` (the size table), `fee_analysis.py` (the fee model), `gen_golden.py` and `experiments/radix4.py` (55); by `go test ./...`; and by `lake build` in `formal/`, which prints the axiom audit. Figure coordinates are copied from these outputs by hand.

11 Costs Against the Alternatives

Joining, updating and leaving a shared output each cost more bytes than the same operation on a bare UTXO. What sharing buys is a different placement of the cost: entry is nearly free in bulk, value moves off-chain for free, a bulk exit costs about a bare spend, and a solo exit costs about ten times one but is a single transaction with no timelock and no counterparty. All figures in this and the following sections use the model of §10. Figures marked *estimate* are built from measured sizes for scripts or populations not measured directly.

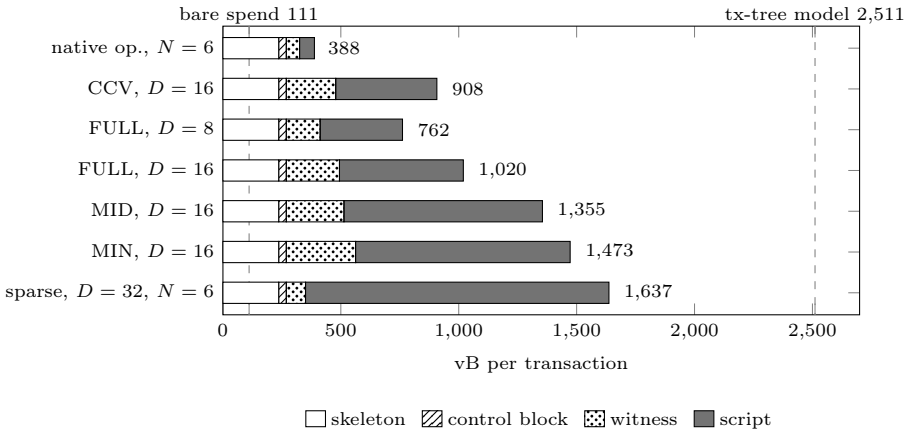


Fig. 5. One unilateral exit by variant, dense unless noted. Skeleton (236.5 vB, white) and control block (32.25 vB, hatched) are common to every row, then witness (dotted) and script (grey); the script dominates because the levels are unrolled. Dashed: a bare P2TR spend and the radix-2 transaction-tree model at $D = 16$. The sparse and native-opcode rows are measured on a six-leaf tree; the native row is a hypothetical opcode in the simulator (§17) with a bitmap-compressed proof.

Exit. Figure 5 decomposes exits; taking the fee from the payout brings the FULL exit at $D = 16$ to 921 vB. The hypothetical native opcode's 388 vB is measured on a six-leaf tree, where the whole witness (leaf, signature and compressed proof) is 56 vB; with 2^{16} members the proof grows by about 14 non-empty siblings to roughly 530 vB, the better estimate of the floor for a design that publishes a proof in script. The transaction-tree model (Table 10) ignores the leaf's timelock and the fee of each of the exiter's transactions. In bulk the cost per leaf depends on who signs: with EXIT_K every leaf carries its own path, so a second exit adds 665 vB, while the operator's sweep of expired leases (only for members who accepted a lease) and the owners' cohort rewrite share the path and reach about 160 vB per leaf, under 1.5 times a bare spend (§13). The chain limit counts transactions, not leaves.

Entry and moving value. JOIN_h walks only the levels above its slot, so at $D = 16$ an operator-funded 1,024-member graft costs 614 vB (estimate, without a payout output), under 1 vB per member, against 43 vB for one output in an on-chain batch and about 154 vB for a channel opened in its own transaction (§14); a self-funded member adds its own 58 vB input and the signing round of §4.4. An on-chain UPDATE is 1,030 vB and a voucher redemption 1,617 vB (neither has a payout output; on the skeleton that carries one, an UPDATE is 1,073 vB), so moving value by rewriting leaves would cost about ten times a bare spend per payment. Value moves off-chain instead, as it does in Lightning and Ark: in channel leaves as HTLC forwards (§8), or by vouchers that the issuer usually

Table 5. Lifetime on-chain bytes per member, by how it leaves, and the unilateral exit's latency (a tower watches for stale states; an Ark member must refresh before expiry). LayerTree rows are dense at $D = 16$ unless marked sparse. Entry is one 43 vB output in a shared transaction; for LayerTree it is a share of a 256-slot operator-funded graft, or that plus the member's 58 vB input when self-funded. LayerTree's batch exit is the 51-leaf sweep, which requires a lease. The sparse row enters by one sparse JOIN_0 (1,601 vB, no payout output) and exits at the 2^{16}-member estimate (1,781 vB), not the six-leaf measurement. Lightning is a simple taproot channel: cooperative close 154 vB; force close a 240 vB commitment, its 152 vB anchor child and a 138 vB sweep. Ark's unilateral exit is the radix-2 unroll of its round.

	vB per member, leaving			unilateral exit		
	never alone	in a batch	alone	txs	blocks	online
LayerTree, operator-funded	3	161	1,023	1	1	nobody
LayerTree, self-funded	61	219	1,081	1	1	nobody
LayerTree sparse, $D = 32$	1,601	3,209	3,382	1	1	nobody
Lightning channel	197	197	573	3	≈ 145	tower
Ark, round of 256	86	86	1,354	9	8+ timelock	member
Ark, round of 65,536	86	86	2,554	17	16+ timelock	member
bare UTXO	154	154	154	1	1	nobody

settles rather than the holder redeems (§15). UPDATE is maintenance, not payment.

Lifetime and latency. Table 5 adds entry and exit. The dense tree wins outright for a member who never exits alone or who is swept, and loses to Lightning and to a bare UTXO for a member who exits alone, though it beats an Ark unroll there; the sparse profile is the most expensive design in every column.

12 Profiles and Radix

Both profiles are the same Merkle-sum tree. They differ in who decides where a leaf sits, and that choice sets the depth, the proof encoding, which predicates exist and the price of every exit.

One node format. A parent hashes each child's sum, not only their total, and this is a soundness property. A tree hashing $h_L \,\|\, h_R \,\|\, (s_L + s_R)$ admits inflation: two leaves of 100 under a root of sum 100 each have a proof whose sibling claims sum zero, and together the proofs claim 200. Here the two proofs hash the distinct preimages $(A, 100, B, 0)$ and $(A, 0, B, 100)$, so at most one reaches the root; in Lean this is serializeNode_inj with hNode_inj, from which fold_sound gives that a verified leaf carries its true amount.

What the profiles decide. A sparse position is a function of the leaf, so a joiner computes it alone, a key and nonce cannot appear twice, the root depends only on the leaf set, and absence is an inclusion proof of E_0. None of LayerTree's predicates needs these properties: an exit proves inclusion and a join proves its slot is E_h, the same way in both profiles. The dense profile has aligned subtrees instead, on which JOIN_h, $\mathsf{EVICT_SUBTREE}_h$ and every shape of §13 depend.

Table 6. Radix in the dense FULL tier, measured: a radix-r level generated and run in the simulator against a radix-r reference tree (55 checks), and EXIT_1 for 65,536 slots (262,144 at radix 8). The radix-2 row reproduces the shipped LEVEL.

radix	level script (B)	level siblings (B)	levels	EXIT_1 script (B)	witness (B)	tx (vB)
2	88	41	16	2,101	902	1,020
4	146	123	8	1,861	1,230	1,042
8	215	287	6	1,983	1,968	1,257

Where the bytes go. A sparse proof is the smaller witness and the larger script, and the script wins (Table 8: about 1,781 vB at $D = 32$ and 2^{16} members against 1,020 vB dense at $D = 16$). Both causes follow from hashing the position. Random positions collide, $N^2/2^{D+1}$ expected collisions among N members (about one half at $D = 32$ for $N = 2^{16}$, about 32,000 at $D = 16$), so the sparse tree needs about twice the depth, and every level is unrolled. And a sparse level is 135 bytes against 88, because it derives the direction bit from the position, substitutes E_k for an elided sibling and computes E_{k+1} for the next level. The full-width tree of Taproot Assets [31] ($D = 256$) costs 35,611 bytes of script per exit, about 17 times the dense script and 9 times the dense transaction; MID caps sparse depth at 31.

Dense is therefore the default (spec 5.4, `spec/layertree.md`): 1.7 to 2.8 times cheaper per exit at 2^{16} members, cohort grafts at one price, sweeps at about 160 vB per leaf, and every batch shape, at the price of an operator who keeps the slot map. The sparse profile is kept, specified and tested, as the measured alternative and for deployments that are key-addressed registries; it still hosts channel leaves and vouchers and still exits in one transaction.

Radix. A radix-r path carries $41(r-1)D/\log_2 r$ sibling bytes, $41D$ at radix 2 and $61.5D$ at radix 4, so radix 2 minimises the witness and a higher radix can only save the fixed part of the unrolled level. Table 6 measures it. Of the radix-4 level's 146 bytes about 62 are hashing and sums shared with radix 2 and the rest sibling and direction handling, so the fixed part a higher radix amortises is about 60 bytes while the sibling part is what it multiplies; radix 4 comes out about 2% larger overall. In the sparse profile most siblings are empty and cost one witness byte, so a radix-4 level of about 180 bytes could replace two levels of 135, saving about 360 vB per exit (estimate; no sparse radix-r level has been written). With a native Merkle-sum opcode a level costs only its sibling and radix 2 is optimal.

13 Batching

The path is half of a transition's cost, about 516 vB of a 1,020 vB exit at $D = 16$, so batching is the question of how many leaves can share one. Read off the measured EXIT_1 and EXIT_2, a transition has a fixed part (skeleton, state binding, epilogue, control block) of about 355 vB paid once per transaction, 32 vB per level paid once per path, and about 148 vB per leaf touched (its 116 bytes, signature,

payout and hashing script), which no shape removes; for one exit at $D = 16$ these sum to 1,020 vB. Batching therefore helps only leaves under one aligned subtree, which cohort grafts create and the sparse profile cannot.

The sweep. EXIT_K chains single-leaf steps, each with its full path, and saves only the skeleton (665 vB for a second exit). $\mathsf{EVICT_SUBTREE}_h$ is the real batch: the witness carries the path above an aligned subtree once and every position of the subtree, the script folds the subtree in post order checking each lease and evictor key, and folds the result and E_h up the shared path (158 vB per leaf for 51 leaves). It is the operator's tool, and it needs a lease on every leaf it sweeps.

The cohort rewrite, $\mathit{REWRITE}_h$. Running the sweep's fold over two subtrees gives the general cohort transition. For each position of an aligned subtree the witness carries an old and a new element (a leaf or the empty string); the old root, folded from the old elements, must fold with the shared path to (r, S); the new root is folded from the new elements; output 0 is $(S - \sum \text{old} + \sum \text{new}, \mathsf{TreeSPK}(r', m))$, an increase funded by other inputs. The owner of each changed leaf signs, except for a leaf whose only change is a larger amount, and the operator signs once. The rule that makes the rewrite a safe mass exit is the payout: a position that goes from a leaf to E_0 must be paid exactly its amount, to its exit key, from the next unused output. An owner who signs a transaction that empties its position is therefore paid by the covenant, and the operator cannot remove a leaf without that owner's signature and that payout; the owner multiproof of spec 13.4 (`spec/layertree.md`) is this special case, not a separate script. In one transaction a cohort can exit, join, resize, renew, rotate keys and pay between members with the sum conserved, with only the members whose leaves change taking part: the settlement step of an Ark round without a round, an expiry, or unchanged members online.

Figure 6 plots the measured costs; $\mathsf{REWRITE}_6$ is 17,148 bytes of script. At $h = 6$ each position costs 244 bytes of script and 234 of witness for its two elements; an owner signature adds 16 vB, and an exit adds its 43-byte output and drops the new element, so an unchanged position costs about 120 vB, a changed one 136 vB and an exiting one 150 vB, with 16 vB more per signed position under the exit-consent form. No shape lowers this per-leaf floor, and the unchanged positions' bytes make the rewrite a tool for cohorts that settle together. The Lean model covers it (rewrite_sound): the old and new subtrees are published, so a cohort that joins this way is grounded by construction and cannot be opaque, and a rewrite that empties the subtree pays out exactly what it held (rewrite_exit_conservation).

Caching the path, DETACH_h. Script cannot read a previous witness, so the upper levels of a cohort's path can only be cached in an output of its own. DETACH_h, signed by the operator, proves an aligned subtree's node (R_h, s_h) against (r, S), leaves E_h in output 0 for $S - s_h$, and creates output 1 as a depth-h tree with root R_h, amount s_h and a taptree m_h fixed at creation; members' paths truncated to the lower h levels are their new proofs (detach_sound, detach_member_payable). At

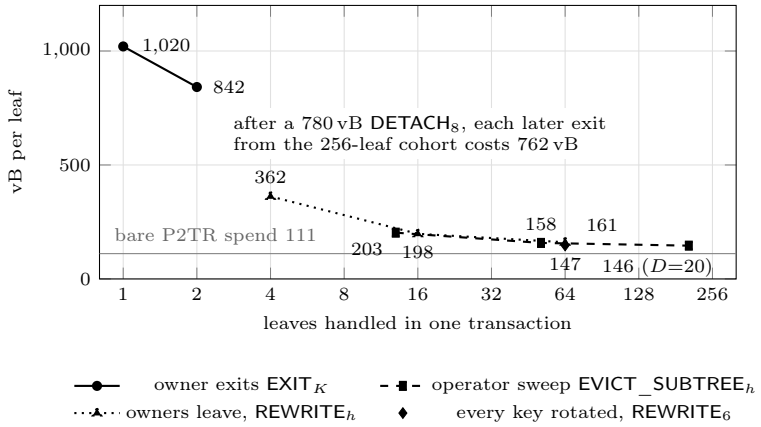


Fig. 6. Batch shapes on the model of §10, all measured at $D = 16$ except the 205-leaf sweep (EVICT_SUBTREE$_8$), measured at $D = 20$: vbytes per leaf that leaves or changes against the leaves one transaction handles. The detach is noted in the plot, not plotted, since its cost is per later exit.

$D = 16$, $h = 8$ it is 1,548 bytes of script and 497 of witness, about 780 vB, and every later exit from the detached tree costs the depth-8 figure, 762 vB instead of 1,020 vB, so it pays for itself at the fourth exit; a cohort of 256 that all leave alone saves about a quarter. The larger effect is throughput: the cohort's exits form their own chain of unconfirmed spends, so assumption [C] applies to it separately, and the operator can shard on demand when, say, channel leaves near their HTLC deadlines. Under spec 0.10.0, m_h must be compiled with the parent's creation options, so a detached channel leaf keeps exact payouts and the other parameters the composition conditions of §8 need; vouchers bind the lineage's tree_id, which the detached tree inherits, so they survive a detach; and channel leaves need nothing, as their chan_id fixes m at creation and a detach leaves every leaf's bytes unchanged. Theorem 5 does not cover trees that carry DETACH$_h$ or REWRITE$_h$.

14 Grafting and Whom It Trusts

JOIN$_h$ replaces an empty node E_h with a node (R_h, s_h) that the operator built off-chain from up to 2^h new leaves, and adds s_h to the tree output. Nothing below height h appears on-chain and the script walks only the $D - h$ levels above the slot, so a larger cohort is a smaller transaction (Table 7). Each height is a tapleaf fixed at creation; the sparse profile batches only by JOINK$_K$.

What the covenant checks. The predicate requires that the slot held E_h, that output 0 carries $S + s_h$, and the operator's signature. It sees a hash and a sum and cannot tell a real cohort from random bytes, which is what makes the graft's size

Table 7. Entry cost per member at $D = 16$ on the model of §10, with no payout output. JOIN_0 and JOIN_2 are measured; JOIN_4 and JOIN_8 are the measured JOIN_0 with the same number of levels ($D = 12$ and $D = 8$), and JOIN_{10} is JOIN_8 less two levels (estimate). Per member is operator-funded; member-funded adds each member's P2TR key-path input (58 vB) and the signing round of §4.4.

graft	members	tx (vB)	per member	member-funded
JOIN_0	1	937	937	≈ 995
JOIN_2	4	872	218	≈ 276
JOIN_4	16	808	51	≈ 109
JOIN_8	256	679	2.7	≈ 61
JOIN_{10}	1,024	614	0.60	≈ 59
sparse JOIN_0, $D = 32$	1	1,601	1,601	$\approx 1{,}659$
sparse JOINK_2, $D = 32$	2	2,821	1,411	$\approx 1{,}469$

independent of the cohort's and is the source of every trust question. join_sound states the consequences: the slot was empty, output 0 is the old sum plus s_h, and for every tree whose root is the grafted node the new root is the old tree with it inserted. A graft therefore cannot move, shrink or rewrite an existing leaf. A node with no preimage is an opaque node of $\mathcal{F}_{\mathsf{LT}}$: its s_h, exactly the value the graft added, is frozen, every other leaf stays payable (Theorem 3, with Lemma 3 for a path past the opaque node), and the terminal exit is lost.

Who carries what risk. Existing members never sign and take no risk. The operator can fill an empty aligned slot with any node it signs and funds, so an opaque or zero-sum graft freezes only the value it brings and wastes only slots no one else could fill. Assurance is needed only by those who put money in. A member funding with an input checks the unsigned transaction (the slot folds as E_h to the current root, its leaf folds to (R_h, s_h) with sums recomputed, output 0 commits to the new root for $S + s_h$) and signs over all outputs, so its input can only be spent into a root containing its leaf. A member buying in off-chain checks the same path against the confirmed root before paying over Lightning, or pays through a hash-locked voucher (§15). Neither checks anyone else's leaf: leaves whose paths verify sum to at most the root, so a co-member's bad leaf cannot make a good one unpayable.

Funding. In an operator-funded graft the operator supplies s_h from the input the model already counts and members buy their balances off-chain; the operator bears the credit risk of a member who never pays, a risk to it and not to other members. In a member-funded graft each member adds an input and signs one transaction after the cohort is fixed, so one absent member stalls the cohort and the rest retry (a liveness cost, not a safety one), and the saving against an on-chain batch, 43 vB per output, is gone. Had the operator grafted a node with no tree behind it, every member's check would fail, nobody would pay, and only the operator's own coins would be frozen.

Designing a cohort. The slot the operator picks is what later batching depends on, and a cohort should share a lease, since one leaf the operator admits with

a NUMS evictor or a later lease blocks the sweep of its subtree (a member cannot create one by carving, which keeps the evictor and the lease, spec 0.10.0). $\mathsf{REWRITE}_h$ is a graft that publishes its leaves: grounded by construction, at about 120 vB per position of the rewritten subtree (measured at $h = 6$, §13) against under 3 per member for JOIN_8. A graft also settles off-chain credit as exitable leaves (§15).

15 Receiving over Lightning

A user with no on-chain funds receives a Lightning payment and wants a balance it can leave with alone. An Ark client gets an out-of-round output co-signed by the operator that expires with its round, or, below the operator's floor, a ledger credit with no on-chain claim. The tree takes the same payment with less trust, through a *cumulative* voucher (Appendix C.2): since a redemption advances the source leaf's seq and voids every other voucher against it, repeated receives are one voucher that the issuer re-signs for the running total against the same claim leaf. The recipient keeps the latest and needs no contest window, since it never prefers a smaller state. The covenant stops the issuer from redeeming more than the source holds; it does not stop over-issuance or voiding (advancing seq, or redeeming an older, smaller voucher, before the recipient redeems). Both are closed by the source leaf's shape.

The dedicated credit leaf. A dedicated credit leaf is a source leaf that the issuer owns and whose exit key is one recipient's, in a tree whose taptree carries CLAIM^x, UPDATE^x and the exact-payout exit, carries $\mathsf{REWRITE}^x_h$ in place of $\mathsf{REWRITE}_h$ (or no rewrite at all), and omits SPLIT. CLAIM^x is CLAIM with one further signature over the transaction, by the source leaf's exit key: 7 bytes of script and 64 of witness in FULL, the only tier that builds it. The recipient's own redemptions carry that signature at no cost, and the issuer cannot carve the leaf at all, not even with a voucher it signs to itself. Over-issuance then has no second victim, since every voucher pays the same recipient, who can watch the leaf's amount. Voiding needs the leaf to change, and every transition that could change it needs the exit key: CLAIM^x, UPDATE^x and $\mathsf{REWRITE}^x_h$ by rule, while the exact exit and an eviction pay the whole balance to that key and so void nothing the recipient is owed. The balance can only grow or pay its recipient, whose remaining exposure is the issuer's liveness: a provider that stops issuing.

Three details keep it that way. The recipient checks from the taptree root m that no plain $\mathsf{REWRITE}_h$ is present, since it needs only the owner's and the operator's signatures, both the issuer's. The issuer funds the leaf only with credit it has issued or is about to, since any excess is the recipient's on every path the issuer can take alone, and tops it up by an amount-only increase under $\mathsf{REWRITE}^x_h$, which needs only the operator's signature. And the leaf has a finite lease with the operator as evictor, not a NUMS evictor, which every carve would inherit, blocking the sweep of whatever cohort the claim leaf lands in. The credit is one-way: if the recipient could also spend from it, an older, larger voucher would

be a stale state needing a window to supersede, as in a bidirectional channel. To spend, the recipient redeems into a channel leaf (§8).

The flow. A user with no on-chain funds receives a Lightning payment into a credit leaf that the operator owns and that names the user's key as exit key. The payer's HTLC reaches the operator, which obtains the preimage and settles that inbound HTLC *first*; only then does it sign the user a voucher for the new running total, a plain cumulative voucher against the credit leaf, redeemable under CLAIM^x. Nothing reaches the chain, there is no minimum per payment beyond DUST_MIN at redemption, nothing expires, and the recipient need not be online when it is paid. The order matters. A hash-locked voucher issued before the HTLC settles, with the preimage held by the recipient, breaks the rule that the party the HTLC pays draws the preimage (spec 9.6.2), and on a credit leaf it cannot be repaired: if the HTLC then fails, retiring the voucher means advancing the credit leaf's seq, which needs the recipient's exit key, so the voucher stays redeemable for a payment that never arrived. Settling first leaves the recipient exposed for one payment, the one the operator has settled and not yet vouched for.

Settlement. Normally the voucher is never redeemed. The operator places a leaf of the recipient's own in its next cohort graft, at under 3 vB per member in a 256-member JOIN_8, and retires the voucher by advancing the credit leaf's seq and lowering its amount by what it has paid out: one UPDATE^x per credit leaf (1,048 vB), which the recipient co-signs, or one $\mathsf{REWRITE}^x_h$ for a cohort of credit leaves, about 147 vB per changed leaf at $h = 6$ plus 16 vB for each exit-key signature. If the operator stops cooperating, the recipient redeems its latest voucher alone with one CLAIM^x: 1,635 vB at $D = 16$, about 16,350 sat at 10 sat/vB, with no timelock, against 1,311 to 2,511 vB and a relative timelock to unroll an Ark round tree of 256 to 65,536 members (§10).

Where one operator leaf backs many recipients instead, over-issuance is equivocation with a compact proof, which a slashable bond can deter; voiding has no compact proof, so no bond reaches it.

16 Related Work

Payment pools. CoinPool [27] is a shared UTXO with per-user balances whose one-transaction withdrawal rests on a new opcode, `OP_MERKLESUB`; `OP_TAPLEAF_UPDATE_VERIFY` [45] recreates a pool output with the leaver's key and leaf removed, and `OP_EVICT` [50] removes unresponsive members. LayerTree reaches one-transaction exit from opcodes proposed as general primitives, holds balances in a Merkle-sum tree rather than keys in a taptree, and adds lease-based eviction, cohort grafts, channel leaves and a composable statement. *Transaction trees.* Channel factories [7] need all N members to sign every change. Ark [22], deployed today [1], batches users into rounds whose outputs are leaves of a pre-signed MuSig2 tree; covenant Ark replaces the signing with CTV templates [38] but keeps

the tree, its unrolled exit and its expiry. Timeout trees [23] and SuperScalar [51] add laddered expiries and Decker-Wattenhofer factories; MATT-style trees [18,19] replace signatures by templates. Table 9 (Appendix C) compares them. *Beyond Bitcoin.* Plasma's forced exits [36] prove a balance against a committed root, enforced by a contract. Proofs of liabilities [20,11] use Merkle-sum trees, where a node that hashes only its children's total admits the inflation that hashing each child's sum avoids. None of these states its guarantees as a functionality; we build on the ledger model of [3] and the channel functionalities of [2,16]. *Components.* The Merkle-sum sparse Merkle tree is Taproot Assets' [31], truncated and verified in script; the data-tweaked key follows BIP-443 [19]; the generator-key signature is Poelstra's [35], and its ECDSA analogue has been used on Bitcoin Cash as `OP_PUSH_TX`. Liquid's tapscript [6] has `OP_CAT`, CSFS, per-field introspection and 64-bit arithmetic, so MID is close to expressible there; Simplicity [30] would express the predicates directly.

17 Discussion and Limitations

The functionality is deliberately modest, and its liveness more so: all activity is serialised through one output, so an honest exit's delay is set by policy and fees, which [C] names rather than derives. Cluster mempool [12] (Bitcoin Core 31) replaced the 25-transaction chain limit with a 64-transaction cluster limit, which raises κ for a cooperative tree but shares the cluster with every descendant anyone attaches. Two open design changes would narrow the dependence: a relative lock on transitions that remove no value, so that only removals chain, and an exit authorised by a CSFS signature over m, the leaf and the payout rather than the outpoint, so a tower could rebuild and rebroadcast it. A Liquid prototype of MID is the obvious next experiment; a hypothetical native Merkle-sum opcode (simulator only) takes EXIT_1 to 252 B of script at every depth.

18 Conclusion

LayerTree specifies a shared output by what it lets members do (join, leave alone, be evicted after a lease, be swept as a cohort) and realises it with a tree kept off-chain as data: a one-transaction exit of about a kilobyte, no pre-signing, no expiry of the shared output, from proposed opcodes; its price is that all activity funnels through one UTXO. Future work: a verified compiler, a proof of Conjecture 1, mechanising Lemma 3, and a P model [15].

References

1. Ark Labs: arkd: Backend services powering Arkade. Repository, `https://github.com/arkade-os/arkd` (2023)
2. Aumayr, L., Ersoy, O., Erwig, A., Faust, S., Hostáková, K., Maffei, M., Moreno-Sanchez, P., Riahi, S.: Generalized channels from limited blockchain scripts and adaptor signatures. In: ASIACRYPT 2021. LNCS, vol. 13091, pp. 635–664. Springer (2021)

3. Badertscher, C., Maurer, U., Tschudi, D., Zikas, V.: Bitcoin as a transaction ledger: A composable treatment. In: CRYPTO 2017. LNCS, vol. 10401, pp. 324–356. Springer (2017)
4. Bitcoin Core: Mempool policy defaults: Ancestor and descendant count limits (25) and cluster limit (64). `src/policy/policy.h` at v31.1, `https://github.com/bitcoin/bitcoin/blob/v31.1/src/policy/policy.h` (2026)
5. Black, B., Rubin, J.: BIP-348: CHECKSIGFROMSTACK. `https://github.com/bitcoin/bips/blob/master/bip-0348.md` (2024)
6. Blockstream: Elements tapscript opcodes: Introspection, streaming SHA256, 64-bit arithmetic, ECMULSCALARVERIFY, TWEAKVERIFY. `https://github.com/ElementsProject/elements/blob/master/doc/tapscript_opcodes.md` (2021)
7. Burchert, C., Decker, C., Wattenhofer, R.: Scalable funding of bitcoin micropayment channel networks. In: Royal Society Open Science. vol. 5 (2018)
8. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: FOCS 2001. pp. 136–145. IEEE (2001)
9. Canetti, R.: Universally composable signature, certification, and authentication. In: CSFW 2004. pp. 219–233. IEEE (2004)
10. Canetti, R., Dodis, Y., Pass, R., Walfish, S.: Universally composable security with global setup. In: TCC 2007. LNCS, vol. 4392, pp. 61–85. Springer (2007)
11. Chalkias, K., Chatzigiannis, P., Ji, Y.: Broken proofs of solvency in blockchain custodial wallets and exchanges. In: Financial Cryptography and Data Security, FC 2022 International Workshops. LNCS, Springer (2022)
12. Daftuar, S.: An overview of the cluster mempool proposal. Delving Bitcoin, `https://delvingbitcoin.org/t/an-overview-of-the-cluster-mempool-proposal/393` (2024)
13. Decker, C., Russell, R., Osuntokun, O.: eltoo: A simple layer2 protocol for bitcoin. `https://blockstream.com/eltoo.pdf` (2018)
14. Decker, C., Towns, A.: BIP-118: SIGHASH_ANYPREVOUT for taproot scripts. `https://github.com/bitcoin/bips/blob/master/bip-0118.mediawiki` (2017)
15. Desai, A., Gupta, V., Jackson, E., Qadeer, S., Rajamani, S., Zufferey, D.: P: Safe asynchronous event-driven programming. In: PLDI 2013. pp. 321–332. ACM (2013)
16. Dziembowski, S., Eckey, L., Faust, S., Malinowski, D.: PERUN: Virtual payment hubs over cryptocurrencies. In: IEEE S&P 2019. pp. 106–123. IEEE (2019)
17. Heilman, E., Sabouri, A.: BIP-347: OP_CAT in tapscript. `https://github.com/bitcoin/bips/blob/master/bip-0347.mediawiki` (2023)
18. Ingala, S.: Merkleize all the things. bitcoin-dev mailing list, `https://merkle.fun` (2022)
19. Ingala, S.: BIP-443: OP_CHECKCONTRACTVERIFY. `https://github.com/bitcoin/bips/blob/master/bip-0443.mediawiki` (2025)
20. Ji, Y., Chalkias, K.: Generalized proof of liabilities. In: ACM CCS 2021. ACM (2021)
21. Katz, J., Maurer, U., Tackmann, B., Zikas, V.: Universally composable synchronous computation. In: TCC 2013. LNCS, vol. 7785, pp. 477–498. Springer (2013)
22. Keceli, B.: Ark: An alternative privacy-preserving second layer solution. bitcoin-dev mailing list, 22 May 2023, `https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2023-May/021694.html` (2023)
23. Law, J.: Scaling lightning with simple covenants. bitcoin-dev mailing list, `https://github.com/JohnLaw2/ln-scaling-covenants` (2023)
24. Lightning Network specification: BOLT #3: Bitcoin transaction and script formats. `https://github.com/lightning/bolts/blob/master/03-transactions.md` (2024)

25. Merkle, R.C.: A digital signature based on a conventional encryption function. In: CRYPTO 1987. LNCS, vol. 293, pp. 369–378. Springer (1988)
26. de Moura, L., Ullrich, S.: The Lean 4 theorem prover and programming language. In: CADE 2021. LNCS, vol. 12699, pp. 625–635. Springer (2021)
27. Naumenko, G., Riard, A.: CoinPool: Efficient off-chain payment pools for Bitcoin. `https://github.com/coinpool-dev/paper/blob/master/coinpool-v0.1.pdf`; `OP_MERKLESUB` draft BIP, `https://github.com/ariard/bips/blob/coinpool-bips/bip-merklesub.mediawiki` (2022)
28. Nick, J., Ruffing, T., Jin, E.: BIP-327: MuSig2 for BIP340-compatible multi-signatures. `https://github.com/bitcoin/bips/blob/master/bip-0327.mediawiki` (2022)
29. Nick, J., Ruffing, T., Seurin, Y.: MuSig2: Simple two-round Schnorr multi-signatures. In: CRYPTO 2021. LNCS, vol. 12825, pp. 189–221. Springer (2021)
30. O'Connor, R.: Simplicity: A new language for blockchains. PLAS 2017, `https://blockstream.com/simplicity.pdf` (2017)
31. Osuntokun, O.: Merkle sum sparse Merkle trees. Draft BIP (Taproot Assets), `https://github.com/Roasbeef/bips/blob/bip-tap/bip-tap-ms-smt.mediawiki` (2021)
32. Osuntokun, O.: Elliptic curve operations for Bitcoin script. Draft BIP, `bitcoin/bips` PR #1945 (closed unmerged), `https://github.com/bitcoin/bips/pull/1945` (2025)
33. Osuntokun, O.: LayerTree: Specification, reference implementation and formal model. `https://github.com/roasbeef/layertree` (2026)
34. Osuntokun, O.: Simple taproot channels (extension BOLT, feature bits 80/81). `lightning/bolts` PR #995, merged May 2026, `https://github.com/lightning/bolts/blob/master/bolt-simple-taproot.md` (2026)
35. Poelstra, A.: CAT and Schnorr tricks I and II. Blog posts, 20 Jan and 10 Feb 2021, `https://www.wpsoftware.net/andrew/blog/cat-and-schnorr-tricks-i.html`, `https://www.wpsoftware.net/andrew/blog/cat-and-schnorr-tricks-ii.html` (2021)
36. Poon, J., Buterin, V.: Plasma: Scalable autonomous smart contracts. White paper, `https://plasma.io/plasma.pdf` (2017)
37. Poon, J., Dryja, T.: The bitcoin lightning network: Scalable off-chain instant payments. `https://lightning.network/lightning-network-paper.pdf` (2016)
38. Roose, S.: The Ark case for CTV. Delving Bitcoin, `https://delvingbitcoin.org/t/the-ark-case-for-ctv/1528` (2025)
39. Rubin, J.: BIP-119: CHECKTEMPLATEVERIFY. `https://github.com/bitcoin/bips/blob/master/bip-0119.mediawiki` (2020)
40. Rubin, J.: BIP-449: OP_TWEAKADD, x-only key tweak addition. `https://github.com/bitcoin/bips/blob/master/bip-0449.md` (2026)
41. Russell, R.: OP_TX. Draft BIP on a personal fork, `https://github.com/rustyrussell/bips/blob/guilt/varops/bip-unknown-optx.mediawiki` (2024)
42. Russell, R., Moik, J.: BIP-440: Varops budget for script runtime constraint. `https://github.com/bitcoin/bips/blob/master/bip-0440.mediawiki` (2025)
43. Russell, R., Moik, J.: BIP-441: Restoration of disabled script (tapleaf 0xC2). `https://github.com/bitcoin/bips/blob/master/bip-0441.mediawiki` (2025)
44. Sanders, G.: LN-Symmetry (eltoo) transaction formats, draft. `https://github.com/instagibbs/bolts/blob/eltoo_draft/XX-eltoo-transactions.md` (2023)
45. Towns, A.: TAPLEAF_UPDATE_VERIFY covenant opcode. bitcoin-dev mailing list, 9 Sep 2021, `https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2021-September/019419.html` (2021)

46. Wuille, P., Nick, J., Ruffing, T.: BIP-340: Schnorr signatures for secp256k1. https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki (2020)
47. Wuille, P., Nick, J., Towns, A.: BIP-341: Taproot: SegWit version 1 spending rules. https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki (2020)
48. Wuille, P., Nick, J., Towns, A.: BIP-342: Validation of taproot scripts. https://github.com/bitcoin/bips/blob/master/bip-0342.mediawiki (2020)
49. Zhao, G.: BIP-431: Topology restrictions for pinning. https://github.com/bitcoin/bips/blob/master/bip-0431.mediawiki (2024)
50. ZmnSCPxj: OP_EVICT: An alternative to OP_TAPLEAFUPDATEVERIFY. bitcoin-dev mailing list, 18 Feb 2022, https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2022-February/019926.html (2022)
51. ZmnSCPxj: SuperScalar: Laddered timeout-tree-structured Decker-Wattenhofer factories. Delving Bitcoin, https://delvingbitcoin.org/t/superscalar-laddered-timeout-tree-structured-decker-wattenhofer-factories/1143 (2024)

A The Lean Formalisation

This appendix describes the mechanised model behind §6 in enough detail that a reader can judge what has been proved, under what assumptions, and what has not. The development is about 3,400 lines of Lean 4 [26] in eight modules and depends on nothing but the core library: no Mathlib, no external tactics. It builds in seconds and prints an axiom audit for 60 headline theorems; as `lake build` prints it, every theorem lists a subset of propext, Quot.sound and Classical.choice, and none lists sorryAx. The code is in the artefact [33].

A.1 Design of the Model

The model states the *rules* of §4.3 as relations over the objects the scripts manipulate: nodes, paths, amounts. It does not model tapscript bytecode, signatures, keys or lock times. Authorisation is carried as an opaque proposition, and the input and output amounts the scripts read with `OP_TX` are parameters. Three choices keep the proofs short.

Depth as a type index. A tree of depth d is Tree d, a perfect binary tree whose tips are Option Leaf. Positions are Pos d h, paths from depth d down to a subtree of height h, so that Pos d 0 addresses a leaf and Pos d h addresses the slot a JOIN_h fills. Indexing by both endpoints avoids all arithmetic on depths: the recursions on getSub, setSub and pathSub pattern-match on the position and the tree simultaneously, and the one impossible case (a cons step below height 0) is dismissed by Pos.height_le.

```
inductive Tree : Nat → Type
  | tip : Option Leaf → Tree 0
  | bin {d : Nat} : Tree d → Tree d → Tree (d + 1)

inductive Pos : Nat → Nat → Type
  | nil {h : Nat} : Pos h h
  | cons {d h : Nat} : Bool → Pos d h → Pos (d + 1) h

def parent (a b : Node H) : Node H :=
```

```
    ⟨M.hNode a.hash a.sum b.hash b.sum, a.sum + b.sum⟩

  def combine (n : Node H) (x : Bool × Node H) : Node H :=
    if x.1 then parent x.2 n else parent n x.2

  def fold (n : Node H) : List (Bool × Node H) → Node H
    | [] => n
    | x :: rest => combine (fold n rest) x
```

Root-first paths. The model orders a sibling path from the root down, so fold is a structural recursion on the list and the soundness proof is a plain induction. The witness of §5 supplies the same pairs leaf-first, because the script consumes the deepest level first; nothing in the theorems depends on the order.

Hashes as parameters. Everything is parametric in a class HashModel whose fields are the two hash functions, the zero digest, and the assumptions the argument uses:

```
class HashModel (Leaf : Type) (H : outParam Type) where
  hLeaf : Leaf → H
  hNode : H → Nat → H → Nat → H
  empty0 : H
  hLeaf_inj : ∀ {l₁ l₂}, hLeaf l₁ = hLeaf l₂ → l₁ = l₂
  hNode_inj : ∀ {a₁ b₁ a₂ b₂ sa₁ sb₁ sa₂ sb₂},
    hNode a₁ sa₁ b₁ sb₁ = hNode a₂ sa₂ b₂ sb₂ →
      a₁ = a₂ ∧ sa₁ = sa₂ ∧ b₁ = b₂ ∧ sb₁ = sb₂
  hNode_ne_hLeaf : ∀ a sa b sb l, hNode a sa b sb ≠ hLeaf l
  hNode_ne_empty : ∀ a sa b sb, hNode a sa b sb ≠ empty0
  hLeaf_ne_empty : ∀ l, hLeaf l ≠ empty0
```

Injectivity is collision resistance idealised; the three inequalities are the tag separation that distinct BIP-340 tags provide. The encodings module discharges the serialisation half of each injectivity field (Lemma 1): given SHA-256 collision resistance, serializeLeaf_inj and serializeNode_inj are what remains. Nothing quantifies the adversary's advantage; a proof under injectivity transfers to any adversary that cannot exhibit a collision.

A.2 Theorem Statements

The statements below are verbatim from the development; those shown with their theorems in §6 are not repeated. t.root is the root node of a tree, t.getSub p the subtree at position p, t.setSub p u the tree with that subtree replaced by u, t.pathSub p the honest sibling path, and t.get, t.set, t.path their leaf-level specialisations.

```
theorem fold_pathSub (t : Tree d) (p : Pos d h) :
    fold (t.getSub p).root (t.pathSub p) = t.root
```

Note that the conclusion of fold_sound equates *nodes*, so the sum the prover presented is the true sum of the subtree.

A Step is a path, a height, an old node and a new node; it is valid for roots r, r' when the old node folds to r and the new node to r', and well formed for depth d when the path has $d - h$ entries. Steps chains steps, each against the root the previous one produced.

```
theorem Step.sound (t : Tree d) (s : Step H) (r' : Node H)
    (hwf : s.WF d) (hv : s.valid t.root r') :
    ∃ p : Pos d s.height, s.old = (t.getSub p).root ∧
      ∀ (u : Tree s.height), u.root = s.new → r' = (t.setSub p u).root
```

Steps.sum is Theorem 2 and needs no hash assumption. Steps.sound is Theorem 4; s.Grounded says the new node is the root of some tree of the step's height, which leaf-level steps satisfy by construction (grounded_of_leaf) and which a JOIN graft must be given.

The transitions are structures bundling the steps with the output rule and an opaque authorisation. Their theorems:

```
theorem ExitK.exact_conservation (h : ExitK true r blocks r' out0) :
    out0 + payoutSum blocks = r.sum

theorem claim_sound (t : Tree d) (hwf₁ : srcPath.length = d)
    (hwf₂ : claimPath.length = d)
    (hc : Claim auth t.root srcPath claimPath src claim r' out0) :
    out0 = t.root.sum ∧
    ∃ (p₁ p₂ : Pos d 0), t.get p₁ = some src ∧
      (t.set p₁ (claimedSource src claim.amount)).get p₂ = none ∧
      r' = ((t.set p₁ (claimedSource src claim.amount))
              .set p₂ (some claim)).root
```

payable is Theorem 3. join_sound uses root_inj (via eq_empty_of_root): since JOIN_h hard-codes E_h as the old node, the slot it fills is provably the empty tree, and the theorem gives the successor root for every tree whose root is the grafted node. claim_sound exposes the drained-source case: the claim position must be empty in the tree *after* the source step, which is why a source reduced to zero may host the claim.

The terminal branch of Final and the sparse profile have their own statements: terminal_no_leaf (a tree whose root is E_D holds no leaf, so the terminal branch is reachable only when the last leaf leaves), ExitKT.terminal_conservation (on that branch the payouts are at most S, exactly S under exact payouts), Tree.Sparse.unique (two leaves with the same owner key and nonce are the same leaf at the same position) and sparseJoin_sound (a sparse join lands at the position derived from the leaf, which was empty, and keeps the tree sparse).

Cohort transitions. The sweep, $\mathsf{REWRITE}_h$ and DETACH_h are one step each from a subtree root at a position p : Pos d h; a position inside it is p.append q. The sweep and the rewrite take the published subtrees as values of Tree h, since the witness carries their 2^h elements and leaves_inj says those name one tree, so neither needs a Grounded hypothesis. In Rewrite, authorisation is a predicate required at every position whose element changes, and the payouts are those of the leaves replaced by E_0, in position order:

```
theorem rewrite_sound (t : Tree d) (hwf : path.length + h = d)
    (hr : Rewrite h auth t.root path old new r' out0 payouts) :
    ∃ p : Pos d h, t.getSub p = old ∧ r' = (t.setSub p new).root ∧
      (∀ q : Pos h 0, t.get (p.append q) = old.get q ∧
        (t.setSub p new).get (p.append q) = new.get q) ∧
      (∀ q : Pos d h, q ≠ p → (t.setSub p new).getSub q = t.getSub q) ∧
      out0 + old.total = t.root.sum + new.total ∧
      out0 + payoutTotal payouts ≤ t.root.sum + new.total

theorem detach_sound (t : Tree d) (hwf : path.length + h = d)
    (hd : Detach auth h t.root path sub r' out0 r₁ out1) :
```

```
∃ p : Pos d h, out0 + out1 = t.root.sum ∧
  r' = (t.setSub p (Tree.empty h)).root ∧
  r₁ = (t.getSub p).root.hash ∧ out1 = (t.getSub p).total ∧
  ∀ q : Pos h 0, (t.getSub p).get q = t.get (p.append q) ∧
    (t.getSub p).path q = (t.path (p.append q)).drop (d - h)
```

rewrite_exit_conservation specialises rewrite_sound to an emptied subtree: output 0 plus the payouts equals the input amount. detach_member_payable applies payable inside the detached tree to the truncated path. evictSubtree_sound is the sweep's analogue of rewrite_sound with the new subtree empty, on either Final branch. Detach states its second tree output as a root hash and an amount rather than generalising Step: the detached output's taptree m_h is a binding fact, outside the step.

A.3 Assumption Ledger

Every hypothesis outside Lean's logic, what it stands for in the deployment, and which theorems consume it.

hypothesis	stands for	used by
hLeaf_inj	SHA-256 collision resistance + serializeLeaf_inj	leafNode_inj, root_inj
hNode_inj	SHA-256 collision resistance + serializeNode_inj	root_inj, fold_sound
hNode_ne_hLeaf, hNode_ne_empty, hLeaf_ne_empty	distinct tags (random-oracle heuristic)	leafNode_inj, fold_sound
treeSpk_inj	DL of NUMS + CR of H_{state}, TapTweak (opposite-sign keys); taproot binding in the ROM (same-sign keys)	witness_root_binds
CCVTweak.dataTweak_inj, tapTweak_inj	as treeSpk_inj, one per stage	CCVTweak.binding, ccv_input_binds
real-tree invariant	creation check + grounded grafts (off-chain)	every theorem stated for a Tree
used by Theorem 5 but outside Lean		
compiler correctness	the bytecode implements Pred_τ (tested, 2,349 checks)	$\mathcal{G}_{\mathsf{ledger}}[\mathsf{Pred}_\tau]$
opcode semantics	[A-1] to [A-6] (Table 2)	every tier
[C]	contention: κ, β (§3)	liveness
[MIN]	CR of TapSighash and the challenge hash	MIN
Lemma 3	paper proof from hNode_inj	honest paths
(none)		Steps.sum, ExitK.no_inflation, encodings

The conservation results use no hypothesis at all, which is the formal content of "the covenant conserves sums whatever it is shown". The hash axioms are stated for all leaves and all Nat sums, while the serialisation injectivities hold for well-formed leaves and sums below 2^{64}; the bridge is that every sum on a verified path is at most the input amount, and the 41-byte size check on each sibling bounds its sum below 2^{64}, so the axioms are only ever applied inside the encodings' range.

A.4 Modules and Effort

module	lines	contents
Hash.lean	59	the HashModel class
Leaf.lean	96	Leaf, Leaf.WF, Node, leafNode, leafNode_inj
Tree.lean	440	trees, positions, paths, fold, subtree positions (Pos.append), all tree theorems
Transitions.lean	625	Step, Steps, the transitions, the terminal branch, sparse positions
Cohort.lean	300	EVICT_SUBTREE$_h$, REWRITE$_h$, DETACH$_h$ on published subtrees
Binding.lean	111	BindingModel, the binding theorems, the CCV tweak (CCVTweak)
Encoding.lean	238	encodings, serialisation injectivity, SEQ_FINAL
Axioms.lean	45	the axiom audit

The only proofs that took real effort were fold_sound, where the two-index positions and the domain-separation case for a tip had to be found, and Steps.sound, where the height of the grounding tree must be reconciled with the height the soundness lemma returns, which is why Step carries its height explicitly. A verified interpreter for the tapscript subset would need a model of the stack machine, of BIP-441 numerics, of `OP_TX`'s serialisations and of the BIP-341 message, and a proof that each generated script's execution relation equals the corresponding Step relation; we estimate it at several thousand lines and regard it as the natural sequel.

A.5 Binding to the Output

Binding.lean records the one cryptographic step outside the hash model. A structure BindingModel carries an abstract treeSpk : H → TapRoot → Spk with the hypothesis that it is injective in both arguments, and two one-line theorems conclude that a witness (r', m') passing the scriptPubKey check equals the committed (r, m), and that the successor output commits to the computed root under the same m. In the deployment the map is $(r, m) \mapsto x\big(P_r + H_{\mathsf{TapTweak}}(x(P_r) \,\|\, m) \cdot G\big)$ with $P_r = \mathsf{lift_x}(x(\mathsf{NUMS} + H_{\mathsf{state}}(r)G))$, and the two x-projections identify a point with its negation. A collision $(r, m) \neq (r', m')$ yields one of: $x(P_r) = x(P_{r'})$ with $r \neq r'$, which is a collision in H_{state} or the relation $\mathsf{NUMS} + tG = -(\mathsf{NUMS} + t'G)$, solvable for the discrete logarithm of NUMS; $x(P_r) = x(P_{r'})$ with $m \neq m'$, a collision in H_{TapTweak}; or distinct internal keys with $Q = \pm Q'$. Write $P_r = \sigma(\mathsf{NUMS} + hG)$ with $\sigma = \pm 1$ and $h = H_{\mathsf{state}}(r)$, and $t = H_{\mathsf{TapTweak}}(x(P_r) \,\|\, m)$. When the two keys have opposite signs, $Q = \pm Q'$ is a linear relation in NUMS and G, solvable for the discrete logarithm of NUMS. When they have the same sign the NUMS terms cancel and what remains is $h + \sigma t = h' + \sigma t' \bmod n$, a collision of the composed map $(r, m) \mapsto H_{\mathsf{state}}(r) + \sigma H_{\mathsf{TapTweak}}(x(P_r) \,\|\, m)$: neither a discrete logarithm nor a collision of either hash. We assume it hard in the random-oracle model, which is how BIP-341's own binding argument goes. Making the hypothesis explicit is the point: it is what a future script change must not break. For the CCV tier CCVTweak.binding decomposes treeSpk_inj into two stage assumptions, the BIP-443 data tweak of NUMS by the root and the taproot tweak by m, each of

which carries the same case split, so the binding theorems hold for a CCV tree unchanged (ccv_input_binds states the first for the input's own taptree).

A.6 Encodings

Encoding.lean proves the byte-level facts the design leans on, over natural-number bytes so that every proof stays within linear arithmetic: fixed-width little-endian encodings are injective on their range; minimalLE (n + 2^(8k)) = leBytes k n ++ [1] for $n < 2^{8k}$, which with $k = 8$ is enc_{64} and justifies producing it with one addition; the 116-byte leaf and the 82-byte node preimage are injective; and the SEQ_FINAL facts of Lemma 1, computed by decide on a bounded-recursion model of CScriptNum.

A.7 What Is Not Proved

The tapscript bytecode is outside the model. That the scripts of §5 implement the relations above is established by execution: every script of every tier runs in the simulator against a reference tree with positive and negative cases, and the Go generators are compared byte for byte with the Python output. A verified interpreter for the tapscript subset (BIP-441 numerics, OP_TX, OP_CHECKSIGFROMSTACK) would close this gap and is a project of a different size. Signatures and lock times are authorisation and deliberately opaque. Payout destinations are not authorisation but are not proved either: that output k pays P2TR of leaf k's exit key, and that an eviction pays the exit key and not the evictor, is established by execution, since ExitBlock carries no exit key. Nor is Lemma 3, which has a paper proof only. Concurrency, mempool policy, reorganisations and the off-chain protocol are not theorems about hash trees and belong to a model checker; a design for a P [15] model is in the artefact.

B Script Listings

Figure 7 states the step every transition is built from; the listings below are taken from the reference generators and are what the size figures of §10 measure. Stack comments show the main stack left to right with the top on the right, and the alt stack after a bar.

One level of the fold (FULL tier, 88 bytes). LEVEL advances both accumulators, the old node (oh, os) and the new node (nh, ns), by one level using the same direction bit and sibling $sc = h \,\|\, \mathsf{enc}_{64}(s)$ from the witness. T is the 64-byte tag prefix of H_{node}; PAD8 is $\langle 2^{64} \rangle$ OP_ADD, which yields enc_{64} of the sum by Lemma 1.

```
OP_6 OP_ROLL OP_6 OP_ROLL           # T oh os nh ns dir sc
OP_SIZE 41 OP_EQUALVERIFY           #   sc is exactly hash || enc64(sum)
OP_DUP 32 OP_8 OP_SUBSTR            # ... dir sc ss8   (sibling sum, raw le64)
OP_DUP OP_TOALTSTACK                #                          | ss8
OP_3 OP_ROLL OP_TUCK OP_ADD         # T oh os nh dir sc ns ns'
OP_TOALTSTACK PAD8                  # T oh os nh dir sc ns9    | ss8 ns'
OP_3 OP_ROLL OP_SWAP OP_CAT         # T oh os dir sc nc        (nc = nh || enc64(ns))
```

$\mathsf{Step}(\pi, n_{\text{old}}, n_{\text{new}}, (r, S))$

1 : $(r_0, S_0) \leftarrow \mathsf{fold}(n_{\text{old}}, \pi)$ // $D - h$ unrolled levels
2 : **assert** $r_0 = r \wedge S_0 = S$ // CHECK_OLD: S is input 0's amount
3 : $(r', S') \leftarrow \mathsf{fold}(n_{\text{new}}, \pi)$ // same siblings, same sums
4 : **return** (r', S')

$\mathsf{Final}((r', S'), m)$

1 : **if** $(r', S') = E_D$ **then return** // terminal: output 0 free
2 : $(a_0, \mathsf{spk}_0) \leftarrow$ OP_TX(output 0)
3 : **assert** $\mathsf{spk}_0 = \mathsf{TreeSPK}(r', m) \wedge a_0 = S'$

Fig. 7. The Merkle-sum step shared by all transitions, and the output rule. A transition is a sequence of steps, each proving its old node against the root the previous step produced (CHECK_PREV), followed by Final. Authorisation and payout checks are specific to each transition.

```
OP_OVER OP_SWAP OP_3 OP_PICK          # T oh os dir sc sc nc dir
OP_NOTIF OP_SWAP OP_ENDIF OP_CAT      # T oh os dir sc pre_n     (dir ? sc||nc : nc||sc)
OP_5 OP_PICK OP_SWAP OP_CAT OP_SHA256 # T oh os dir sc nh'
OP_TOALTSTACK                         #                          | ss8 ns' nh'
OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
                                      # T oh os dir sc nh' ns' ss8
OP_5 OP_ROLL OP_TUCK OP_ADD           # T oh dir sc nh' ns' os os'
OP_TOALTSTACK PAD8                    # T oh dir sc nh' ns' os9  | os'
OP_5 OP_ROLL OP_SWAP OP_CAT           # T dir sc nh' ns' oc      (oc = oh || enc64(os))
OP_3 OP_ROLL OP_SWAP OP_4 OP_ROLL     # T nh' ns' sc oc dir
OP_NOTIF OP_SWAP OP_ENDIF OP_CAT      # T nh' ns' pre_o
OP_3 OP_PICK OP_SWAP OP_CAT OP_SHA256 # T nh' ns' oh'
OP_FROMALTSTACK OP_2SWAP              # T oh' os' nh' ns'
```

Checking the old root and writing the new one. After D levels, CHECK_OLD compares the old accumulator with the committed state: the sum against input 0's amount and the hash against the witness root r parked on the alt stack (itself bound to the input scriptPubKey by the prologue). FINAL constrains output 0 unless the new root is the empty tree.

```
# CHECK_OLD:  [T oh os nh ns] | ... r  ->  [T nh ns]
OP_2SWAP <SEL_IN_AMOUNT> OP_TX OP_NUMEQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

# FINAL:  [T nh ns] | m  ->  [1]
OP_ROT OP_DROP                                  # nh ns
OP_OVER <E_D.hash> OP_EQUAL
OP_IF   OP_2DROP OP_FROMALTSTACK OP_DROP OP_1   # terminal: output 0 free
OP_ELSE OP_SWAP OP_FROMALTSTACK TREE_SPK        # ns spk'
        OP_0 <SEL_OUT_SINGLE> OP_TX             # ns spk' amt0 spk0
        OP_ROT OP_EQUALVERIFY OP_NUMEQUAL       # 1
OP_ENDIF
```

The MID gadget. Under 0xC0 a computed 4-byte number x is widened to 8 bytes by two witness elements z, w: the script checks $\mathsf{size}(x) + \mathsf{size}(z) = 4$ and $z \,\|\, w = 00000000$ and uses $x \,\|\, z \,\|\, w$ as $\mathsf{le}_{64}(x)$. Two such gadgets per level and the per-field size checks account for almost all of MID's premium over FULL.

The MIN message. The MIN exit assembles the BIP-341 signature message for input 0 with hash type `SIGHASH_DEFAULT` and checks it as described in §5. Fields marked *derived* are computed by the script from values it has verified; fields marked *raw* are witness elements the script does not constrain but must be presented consistently, since the whole message is hashed and compared with the real one through `OP_CHECKSIG`.

field	source	note
epoch, hash type	constant `00 00`	`SIGHASH_DEFAULT`
version, lock time	raw	free
sha_prevouts	raw	free
sha_amounts	derived: $\mathsf{SHA256}(\mathsf{le}_{64}(S) \,\|\|\, amt_1)$	S verified by CHECK_OLD; amt_1 raw; two inputs exactly
sha_scriptpubkeys	derived: $\mathsf{SHA256}(22 \,\|\|\, \mathsf{TreeSPK}(r, m) \,\|\|\, spk_1)$	TreeSPK recomputed; spk_1 raw
sha_sequences	raw	the field the spender grinds
sha_outputs	derived: $\mathsf{SHA256}(out_0 \,\|\|\, out_1 \,\|\|\, extra)$	$out_0 = \mathsf{le}_{64}(S') \,\|\| 22 \,\|\| \mathsf{TreeSPK}(r', m)$, out_1 the payout; *extra* raw
spend type, input index	constant `02 00000000`	script path, no annex, input 0
tapleaf hash	raw	the leaf being executed
key version, codesep	constant `00 ffffffff`	

The message is hashed with the TapSighash tag and the challenge e is computed from $G_x \,\|\, G_x \,\|\, \mathsf{msg}$ under the challenge tag. The script verifies $e_{[0,31)} \,\|\, \mathtt{00} = e$ against the witness-supplied $e_{[0,31)}$ and finally executes

$$\langle G_x \,\|\, e_{[0,31)} \,\|\, \mathtt{01} \rangle \;\; \langle G_x \rangle \;\; \texttt{OP_CHECKSIG}.$$

Grinding and fee bumping. The spender grinds the fee input's `nSequence` until e ends in `00` (so that $e + 1$ is formed by replacing the last byte), 256 hashes on average, choosing values that set no BIP-68 relative lock (the disable bit set) and stay below `0xFFFFFFFE` so the transaction keeps signalling replaceability. Any change to the transaction changes the message, so a replacement with a higher fee must be re-ground and re-signed by the owner; a third party bumps it only by a child spending the fee change or an anchor. The raw *extra* outputs travel as one witness element, and Bitcoin Core's policy limits `0xC0` witness items to 80 bytes: one P2TR change output and a P2A anchor (56 bytes) fit, two P2TR change outputs (86 bytes) make the exit non-standard. The simulator checks consensus limits only.

C Measurements, Vouchers and Fee Tables

C.1 Measured Sizes

Table 8 gives the script, witness and transaction sizes behind Figure 4 and Table 3.

Table 8. Measured EXIT_1 cost (FULL unless noted) on the model of §10. Witness weight is script, witness data and a 129-byte control block for the eight-leaf reference script set (each doubling of the set adds 8 vB); the transaction adds the 946 WU skeleton. The sparse witness is $244 + 41k$ bytes for k non-empty siblings: $k = 2$ on the measured six-leaf tree (326 B), and $k \approx \log_2 N$, so the rows give the estimate at $N = 2^{16}$ ($k = 16$). Last column: unamortised radix-2 transaction tree of 2^D leaves, $150D + 111$ vB; sparse D is address width.

profile	D	script (B)	witness data (B)	witness (WU)	tx (vB)	radix-2 tx tree (vB)
dense	8	1,397	574	2,100	762	1,311
dense	12	1,749	738	2,616	891	1,911
dense	16	2,101	902	3,132	1,020	2,511
dense	20	2,453	1,066	3,648	1,149	3,111
sparse	32	5,147	≈ 900	≈ 6,176	≈ 1,781	n/a
sparse	64	9,499	≈ 900	≈ 10,528	≈ 2,869	n/a
MID dense	16	3,367	978	4,474	1,355	
MIN dense	16	3,640	1,175	4,944	1,473	
CCV dense	16	1,716	838	2,683	908	

C.2 Vouchers (CLAIM)

A voucher is a BIP-340 signature by the source leaf's owner over

$$H_{\mathsf{voucher}}(\mathsf{tree_id} \,\|\, \ell_{\mathrm{src}} \,\|\, \ell_{\mathrm{claim}}),$$

verified with `OP_CHECKSIGFROMSTACK`. Under spec 0.10.0 every tier binds tree_id, a constant fixed when a tree is created and inherited by the cohorts detached from it, in place of m, which names a rule set and so is shared by trees created with the same parameters; a voucher therefore verifies only in its own lineage, and survives a detach. The claim leaf must keep the source's evictor key and a lease no later than the source's (spec 0.10.0), and neither the claim leaf nor a non-zero residual may be below DUST_MIN, a creation constant of 330 sat by default (spec 0.11.0), so that no carve writes a leaf whose removal default policy would not relay. CLAIM is two steps: the source leaf's amount is reduced by the claim amount and its seq advanced by one (or the position emptied if drained), then the claim leaf is placed at an empty position. The total is unchanged. Replay is impossible because the first step changes the source leaf, so the signed message never again matches a leaf in the tree; seq prevents a balance that later returns to an earlier value from re-enabling an old voucher. Because a voucher lets an owner move value with a lone signature, leaves whose balance is not the owner's alone set seq to the tier's SEQ_FINAL (§8), whose successor does not fit in four bytes, so the CLAIM script rejects them. Vouchers are the mechanism for operator-to-member push payments without the operator's later participation.

Hash-locked vouchers (CLAIM^h). Adding a payment hash to the signed message, under its own tag, and requiring the preimage at redemption turns a voucher into a hash-locked cheque that can be paired with a channel-leaf HTLC or a Lightning invoice carrying the same hash. It is not an HTLC and the pairing is not atomic: the value stays in the source leaf, and until the redemption confirms the issuer

can void the cheque with a conflicting EXIT or SPLIT of that leaf, so the redeemer bears issuer risk for that window, which contention on the tree output lengthens. The script does not compare a witness-supplied hash against the preimage: it computes SHA256(preimage) and feeds it into the signed message, so a wrong preimage fails as an invalid signature. That costs one hash opcode, eight bytes of script in every tier, and no opcode beyond those CLAIM already needs. The separate tag is load-bearing: reusing H_{voucher} for both forms would let a redeemer strip the hash lock.

The variant adds ordering, not security. It cannot create an on-chain deadline, because script can require that a spend not happen before a height but not that it happen before one, and the value backing a voucher never leaves the source leaf, so there is nothing to refund; deadlines live in the other leg of the swap, and an issuer retires an unredeemed voucher by advancing the source leaf's seq. Redemption publishes the preimage, which is what settles the other leg and also what obliges a redeemer to secure its own leg first.

Exit-key consent (CLAIM^x, CLAIM^{hx}). The FULL tier also builds both forms with one further signature over the transaction by the source leaf's exit key, as UPDATE^x adds to UPDATE: 7 bytes of script over the plain form, as UPDATE^x over UPDATE, and 64 of witness. Its use is a credit leaf owned by an issuer and exiting to one recipient: every voucher against it pays that recipient, whose own redemptions carry the exit-key signature at no cost, and the issuer cannot carve the leaf even with a voucher it signs to itself.

C.3 Comparisons

C.4 Fee Tables

Fees for the measured sizes at fee rates from 1 to 250 sat/vB, on the model of §10; every fee is the tabulated vB times the rate. The sparse row is the six-leaf measurement.

case	vB	1	10	25	50	250 sat/vB
dense EXIT$_1$, $D = 8$	762	762	7,620	19,050	38,100	190,500
dense EXIT$_1$, $D = 16$	1,020	1,020	10,200	25,500	51,000	255,000
dense EXIT$_1$, $D = 20$	1,149	1,149	11,490	28,725	57,450	287,250
sparse EXIT$_1$, $D = 32$, $N = 6$	1,637	1,637	16,370	40,925	81,850	409,250
MID dense EXIT$_1$, $D = 16$	1,355	1,355	13,550	33,875	67,750	338,750
MIN dense EXIT$_1$, $D = 16$	1,473	1,473	14,730	36,825	73,650	368,250
second exit in EXIT$_2$, marginal	665	665	6,650	16,625	33,250	166,250
EVICT_SUBTREE$_4$, per leaf (13)	203	203	2,030	5,075	10,150	50,750
EVICT_SUBTREE$_6$, per leaf (51)	158	158	1,580	3,950	7,900	39,500
plain P2TR to P2TR	111	111	1,110	2,775	5,550	27,750

Break-even balances for the dense $D = 16$ exit, i.e. the leaf amount at which the fee is 1%, 5% or 10% of the balance:

sat/vB	fee (sat)	balance at 1%	at 5%	at 10%
2	2,040	204,000	40,800	20,400
10	10,200	1,020,000	204,000	102,000
50	51,000	5,100,000	1,020,000	510,000
250	255,000	25,500,000	5,100,000	2,550,000

Table 9. Comparison with shared-output constructions. Pools: `OP_TAPLEAF_UPDATE_VERIFY` [45], CoinPool's `OP_MERKLESUB` [27], `OP_EVICT` [50]. GSR: the string opcodes BIP-441 restores; APO: `SIGHASH_ANYPREVOUT`. LayerTree's interactive signing: the covenant needs the operator only; a self-funded cohort's funders sign one transaction together.

	LayerTree	Pools	Ark (deployed)	Ark (covenant)	Timeout tree	Factory
Unilateral exit	1 tx, $O(D)$ bytes	1 tx	$O(\log N)$ txs	$O(\log N)$ txs	$O(\log N)$ txs	$O(\log N)$ txs
Evict one member	1 tx after its lease	1 tx (`EVICT`)	round expires	round expires	tree expires	no
Member position expiry	lease, renewed by the operator; eviction pays the member	none	per round; operator reclaims	per round	per tree	none
Interactive signing	operator only; a self-funded cohort once	all, to update	round participants	funders, per round	none or all	all
Shared output expiry	none	none	per round	per round	per tree	none
Batch add	1 tx, any subtree	re-sign	per round	per round	per tree	re-sign all
Opcodes	FULL: GSR, TX, CSFS, EC; MID: CAT, TX, CSFS, TWEAKADD; MIN: CAT, TWEAKADD; CCV: CCV, TX	a pool opcode (and APO for CoinPool)	none	CTV or CCV	CTV	none

Table 10. The transaction-tree model across round sizes N. A level is one key-path P2TR input and r P2TR outputs, $68 + 43r$ vB (154 at radix 2, used as 150; 240 at radix 4); the claim is a plain P2TR spend, 111 vB; anchors and their fee children are omitted, which favours the tree. Mass exit: the tree's interior shared by all members, against LayerTree's $\mathsf{REWRITE}_6$ or sweep of a 64-slot cohort. LayerTree is measured at $D = \log_2 N$.

	LayerTree, dense	radix-2 tree	radix-4 tree
isolated exit, $N = 2^8$	762	1,311	1,071
isolated exit, $N = 2^{12}$	891	1,911	1,551
isolated exit, $N = 2^{16}$	1,020	2,511	2,031
mass exit, per member	158 to 161	≈ 261	≈ 191

Operators should size cohorts and minimum leaf amounts so that a sweep stays below about 10% of the smallest leaf at the fee-rate tail they intend to survive; the FULL-tier fee-from-payout option saves a constant 99 vB and is unavailable to channel leaves.

D Complete Script Listings

This appendix reproduces every script the reference generators emit, for the smallest depths that exercise every construct: dense $D = 4$ and sparse $D = 8$, and dense $D = 8$ for the cohort shapes of FULL. Tapscript has no subroutines, so a compiled script is one flat opcode stream; the generators, however, build every script from a small set of helpers, and the listings below are those streams *collapsed back* into named macro calls, one per line, with the remaining raw opcodes wrapped between them. Each tier's macros are defined once (§D.1, §D.6, §D.8, §D.10), themselves collapsed against the other macros; the CCV tier under `0xC0` numerics uses MID's. The collapse is mechanical (`tools/gen_structured.py`) and verified: expanding a structured listing reproduces the compiled bytes exactly, so these listings and the flat ones in `spec/scripts/` describe the same scripts, and the byte count in each heading is the compiled size. A run of n identical calls is written `NAME xn`. Angle brackets denote pushes of named constants: `<TAG_NODE>` is the 64-byte tagged-hash prefix, `<EMPTY_k>` the hash of E_k, `<ENC0>` the constant 2^{64} whose addition implements PAD8, `<NUMS>` the BIP-341 nothing-up-my-sleeve point, and `<SEL_*>` the `OP_TX` selectors of §4.2. Bare integers are minimal number pushes. Sizes at $D = 16$ and $D = 32$ are in Table 8; each further level adds one LEVEL (88 bytes dense, 135 sparse, 160 MID).

D.1 FULL Tier Macros

```
# Macro definitions, FULL tier. Each expansion is itself collapsed
# against the other macros; a macro never contains itself.

## TAGGED(LEAF)
<TAG_LEAF> OP_SWAP OP_CAT OP_SHA256

## TAGGED(NODE)
<TAG_NODE> OP_SWAP OP_CAT OP_SHA256

## TAGGED(STATE)
<TAG_STATE> OP_SWAP OP_CAT OP_SHA256

## TAGGED(VOUCHER)
<TAG_VOUCHER> OP_SWAP OP_CAT OP_SHA256

## TAGGED(POS)
<TAG_POS> OP_SWAP OP_CAT OP_SHA256

## TAGGED(TAPTWEAK)
<TAG_TAPTWEAK> OP_SWAP OP_CAT OP_SHA256

## PAD8   ([n] -> [enc64(n)]: minimal encoding of n + 2^64 is le64(n)||01)
<ENC0> OP_ADD

## LE64   ([n] -> [le64(n)])
```

```
PAD8
OP_8 OP_LEFT

## PAD4   ([n] -> [le32(n)])
<ZERO4> OP_CAT OP_4 OP_LEFT

## TREE_SPK   ([r m] -> [TREE_SPK(r, m)] with EC ops)
OP_SWAP
TAGGED(STATE)
OP_0 OP_SWAP OP_EC_POINT_MUL <NUMS> OP_EC_POINT_ADD OP_EC_POINT_X_COORD
  OP_TUCK OP_SWAP OP_CAT
TAGGED(TAPTWEAK)
OP_0 OP_SWAP OP_EC_POINT_MUL OP_SWAP OP_2 OP_SWAP OP_CAT OP_EC_POINT_ADD
  OP_EC_POINT_X_COORD <0x5120> OP_SWAP OP_CAT

## TREE_SPK_MIN   ([r m] -> [TREE_SPK(r, m)] with OP_TWEAKADD)
OP_SWAP
TAGGED(STATE)
<NUMS_X> OP_TWEAKADD OP_TUCK OP_SWAP OP_CAT
TAGGED(TAPTWEAK)
OP_SWAP OP_TWEAKADD <0x5120> OP_SWAP OP_CAT

## LEAF_ACC   ([leaf] -> [H_leaf(leaf) amount])
OP_DUP OP_8 OP_LEFT OP_SWAP
TAGGED(LEAF)
OP_SWAP

## LEAF_POS   ([leaf] -> [leaf H_pos(owner_key||nonce)])
OP_DUP 40 32 OP_SUBSTR OP_OVER 108 OP_4 OP_SUBSTR OP_CAT
TAGGED(POS)

## PROLOGUE   (input is #0; (r, m) match the input scriptPubKey; alt <- m r)
<SEL_CURR_INPUT> OP_TX OP_0 OP_NUMEQUALVERIFY OP_2DUP
TREE_SPK
<SEL_IN_SPK> OP_TX OP_EQUALVERIFY OP_TOALTSTACK OP_TOALTSTACK

## CARVE_RULES   ([src claim] -> [src claim]: both 116 bytes, claim.evictor_key ==
     src.evictor_key, claim.evict_after <= src.evict_after (7.5 rule 5), claim.amount >=
     DUST_MIN (7.5 rule 6))
OP_SIZE 116 OP_EQUALVERIFY OP_SWAP OP_SIZE 116 OP_EQUALVERIFY OP_SWAP OP_2DUP
  72 32 OP_SUBSTR OP_SWAP 72 32 OP_SUBSTR OP_EQUALVERIFY OP_2DUP 104 OP_4
  OP_SUBSTR OP_SWAP 104 OP_4 OP_SUBSTR OP_LESSTHANOREQUAL OP_VERIFY 330
  OP_OVER OP_8 OP_LEFT OP_LESSTHANOREQUAL OP_VERIFY

## DRAIN_RULE   ([.. nh ns] -> [.. nh' ns]: a zero residual makes the new node E_0, any
     other is at least DUST_MIN (7.5 rules 2 and 6))
OP_DUP OP_NOT OP_IF OP_SWAP OP_DROP <EMPTY_0> OP_SWAP OP_ELSE 330 OP_OVER
  OP_LESSTHANOREQUAL OP_VERIFY OP_ENDIF

## LEVEL   (dense level: dir sc T oh os nh ns -> T oh' os' nh' ns')
OP_6 OP_ROLL OP_6 OP_ROLL OP_SIZE 41 OP_EQUALVERIFY OP_DUP 32 OP_8 OP_SUBSTR
  OP_DUP OP_TOALTSTACK OP_3 OP_ROLL OP_TUCK OP_ADD OP_TOALTSTACK
PAD8
OP_3 OP_ROLL OP_SWAP OP_CAT OP_OVER OP_SWAP OP_3 OP_PICK OP_NOTIF OP_SWAP
  OP_ENDIF OP_CAT OP_5 OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_TOALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_5 OP_ROLL OP_TUCK OP_ADD
  OP_TOALTSTACK
PAD8
OP_5 OP_ROLL OP_SWAP OP_CAT OP_3 OP_ROLL OP_SWAP OP_4 OP_ROLL OP_NOTIF
  OP_SWAP OP_ENDIF OP_CAT OP_3 OP_PICK OP_SWAP OP_CAT OP_SHA256
  OP_FROMALTSTACK OP_2SWAP

## LEVEL_MIN   (dense level without splice opcodes: sibling as two elements)
OP_7 OP_ROLL OP_7 OP_ROLL OP_7 OP_ROLL OP_SIZE OP_8 OP_EQUALVERIFY OP_OVER
  OP_SIZE 32 OP_EQUALVERIFY OP_DROP OP_DUP OP_TOALTSTACK OP_SWAP OP_OVER
PAD8
OP_CAT OP_SWAP OP_3 OP_ROLL OP_TUCK OP_ADD OP_TOALTSTACK
PAD8
```

```
OP_3 OP_ROLL OP_SWAP OP_CAT OP_OVER OP_SWAP OP_3 OP_PICK OP_NOTIF OP_SWAP
  OP_ENDIF OP_CAT OP_5 OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_TOALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_5 OP_ROLL OP_TUCK OP_ADD
  OP_TOALTSTACK
PAD8
OP_5 OP_ROLL OP_SWAP OP_CAT OP_3 OP_ROLL OP_SWAP OP_4 OP_ROLL OP_NOTIF
  OP_SWAP OP_ENDIF OP_CAT OP_3 OP_PICK OP_SWAP OP_CAT OP_SHA256
  OP_FROMALTSTACK OP_2SWAP

## SPARSE_SETUP   ([pos_hash] -> [] ; alt <- pos E_0)
OP_TOALTSTACK <EMPTY_0> OP_TOALTSTACK

## SPARSE_TEARDOWN   (drop pos and E_k from the alt stack)
OP_FROMALTSTACK OP_DROP OP_FROMALTSTACK OP_DROP

## CHECK_OLD   (os == input 0 amount, oh == r (from alt))
OP_2SWAP <SEL_IN_AMOUNT> OP_TX OP_NUMEQUALVERIFY OP_FROMALTSTACK
  OP_EQUALVERIFY

## CHECK_OLD_S   (CHECK_OLD with the sparse teardown)
OP_2SWAP <SEL_IN_AMOUNT> OP_TX OP_NUMEQUALVERIFY
SPARSE_TEARDOWN
OP_FROMALTSTACK OP_EQUALVERIFY

## CHECK_PREV   (os == ns_prev, oh == nh_prev (from alt); chained steps)
OP_2SWAP OP_FROMALTSTACK OP_NUMEQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

## CHECK_PREV_S   (CHECK_PREV with the sparse teardown)
OP_2SWAP
SPARSE_TEARDOWN
OP_FROMALTSTACK OP_NUMEQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

## LEAF_STEP   (cohort sweep: one leaf or empty -> its node, paying the next output)
OP_SIZE OP_0NOTEQUAL OP_IF OP_SIZE 116 OP_EQUALVERIFY OP_DUP 72 32 OP_SUBSTR
  <OPERATOR_KEY> OP_EQUALVERIFY OP_DUP 104 OP_4 OP_SUBSTR
  OP_CHECKLOCKTIMEVERIFY OP_DROP OP_FROMALTSTACK OP_DUP OP_1ADD OP_TOALTSTACK
  <SEL_OUT_SINGLE> OP_TX <0x5120> OP_3 OP_PICK OP_8 32 OP_SUBSTR OP_CAT
  OP_EQUALVERIFY OP_OVER OP_8 OP_LEFT OP_NUMEQUALVERIFY
LEAF_ACC
OP_ELSE OP_DROP <EMPTY_0> OP_0 OP_ENDIF

## LEVEL_S(k)   [shown for k = 0; <EMPTY_k> and the bit index vary with k]   (sparse level
     k: sc T oh os nh ns | pos E_k -> T oh' os' nh' ns' | pos E_k+1)
OP_5 OP_ROLL OP_FROMALTSTACK OP_SWAP OP_SIZE OP_0NOTEQUAL OP_NOTIF OP_DROP
  OP_DUP <ENC0> OP_CAT OP_ENDIF OP_SWAP <ENC0> OP_CAT OP_DUP OP_CAT OP_6
  OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_FROMALTSTACK OP_DUP OP_TOALTSTACK
  OP_SWAP OP_TOALTSTACK OP_0 OP_DOWNSHIFT OP_1 OP_AND OP_0NOTEQUAL OP_SWAP
  OP_SIZE 41 OP_EQUALVERIFY OP_DUP 32 OP_8 OP_SUBSTR OP_DUP OP_TOALTSTACK
  OP_3 OP_ROLL OP_TUCK OP_ADD OP_TOALTSTACK
PAD8
OP_3 OP_ROLL OP_SWAP OP_CAT OP_OVER OP_SWAP OP_3 OP_PICK OP_NOTIF OP_SWAP
  OP_ENDIF OP_CAT OP_5 OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_TOALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_5 OP_ROLL OP_TUCK OP_ADD
  OP_TOALTSTACK
PAD8
OP_5 OP_ROLL OP_SWAP OP_CAT OP_3 OP_ROLL OP_SWAP OP_4 OP_ROLL OP_NOTIF
  OP_SWAP OP_ENDIF OP_CAT OP_3 OP_PICK OP_SWAP OP_CAT OP_SHA256
  OP_FROMALTSTACK OP_2SWAP

## FINAL   [shown for D = 4; <EMPTY_D> is the empty root of the tree's depth]   (output 0 is
     (ns, TREE_SPK(nh, m)) unless nh == E_D)
OP_ROT OP_DROP OP_OVER <EMPTY_4> OP_EQUAL OP_IF OP_2DROP OP_FROMALTSTACK
  OP_DROP OP_1 OP_ELSE OP_SWAP OP_FROMALTSTACK
TREE_SPK
OP_0 <SEL_OUT_SINGLE> OP_TX OP_ROT OP_EQUALVERIFY OP_NUMEQUAL OP_ENDIF

## HASH_STEP(t)   [shown for t = 3; the OP_PICK depth is t-3]   (cohort sweep: two child
     nodes -> parent node)
```

```
OP_2 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK
PAD8
OP_CAT OP_SWAP
PAD8
OP_ROT OP_SWAP OP_CAT OP_SWAP OP_CAT OP_0 OP_PICK OP_SWAP OP_CAT OP_SHA256
  OP_FROMALTSTACK

## REWRITE_STEP(d)   [shown for d = 0; the OP_ROLL depths are d+3, d+4 and d+5, the picks
     d+3 to d+5]   (cohort rewrite: old and new elements of one position -> oh os nh ns,
     with 7.9 rules 3 and 5)
OP_3 OP_ROLL OP_4 OP_ROLL OP_2DUP OP_EQUAL OP_NOTIF OP_OVER OP_SIZE OP_NIP
  OP_ONOTEQUAL OP_IF OP_DUP OP_SIZE OP_NIP OP_ONOTEQUAL OP_IF OP_2DUP OP_8
  108 OP_SUBSTR OP_SWAP OP_8 108 OP_SUBSTR OP_EQUAL OP_IF OP_2DUP OP_SWAP
  OP_8 OP_LEFT OP_SWAP OP_8 OP_LEFT OP_LESSTHANOREQUAL OP_OVER OP_4 OP_RIGHT
  <SEQ_FINAL> OP_EQUAL OP_NOT OP_BOOLAND OP_ELSE OP_0 OP_ENDIF OP_NOTIF
OWNER_AUTH(0)
OP_ENDIF OP_ELSE
OWNER_AUTH(0)
OP_FROMALTSTACK OP_DUP OP_1ADD OP_TOALTSTACK <SEL_OUT_SINGLE> OP_TX <0x5120>
  OP_4 OP_PICK OP_8 32 OP_SUBSTR OP_CAT OP_EQUALVERIFY OP_2 OP_PICK OP_8
  OP_LEFT OP_NUMEQUALVERIFY OP_ENDIF OP_ENDIF OP_ENDIF OP_SWAP
ELEM_NODE(0)
OP_ROT
ELEM_NODE(1)

## REWRITE_STEP_X(d)   [shown for d = 0; the OP_ROLL depths are d+3, d+4 and d+5, the picks
     d+3 to d+5]   (REWRITE_STEP with OWNER_AUTH_X)
OP_3 OP_ROLL OP_4 OP_ROLL OP_2DUP OP_EQUAL OP_NOTIF OP_OVER OP_SIZE OP_NIP
  OP_ONOTEQUAL OP_IF OP_DUP OP_SIZE OP_NIP OP_ONOTEQUAL OP_IF OP_2DUP OP_8
  108 OP_SUBSTR OP_SWAP OP_8 108 OP_SUBSTR OP_EQUAL OP_IF OP_2DUP OP_SWAP
  OP_8 OP_LEFT OP_SWAP OP_8 OP_LEFT OP_LESSTHANOREQUAL OP_OVER OP_4 OP_RIGHT
  <SEQ_FINAL> OP_EQUAL OP_NOT OP_BOOLAND OP_ELSE OP_0 OP_ENDIF OP_NOTIF
OWNER_AUTH_X(0)
OP_ENDIF OP_ELSE
OWNER_AUTH_X(0)
OP_FROMALTSTACK OP_DUP OP_1ADD OP_TOALTSTACK <SEL_OUT_SINGLE> OP_TX <0x5120>
  OP_4 OP_PICK OP_8 32 OP_SUBSTR OP_CAT OP_EQUALVERIFY OP_2 OP_PICK OP_8
  OP_LEFT OP_NUMEQUALVERIFY OP_ENDIF OP_ENDIF OP_ENDIF OP_SWAP
ELEM_NODE(0)
OP_ROT
ELEM_NODE(1)

## HASH_STEP2(d)   [shown for d = 8; the HASH_STEP pick depths are d-3 and d-5]   (cohort
     rewrite: two old and two new child nodes -> old and new parent nodes)
OP_2SWAP OP_7 OP_ROLL OP_7 OP_ROLL OP_2SWAP
HASH_STEP(8)
OP_2SWAP OP_5 OP_ROLL OP_5 OP_ROLL OP_2SWAP
HASH_STEP(6)

## ELEM_NODE(d)   [shown for d = 0; picks L at d+4 and Z at d+3 for the old element, one
     deeper for the new]   ([elem] -> [h s]: a leaf's node, or E_0 for the empty element;
     picks the hoisted TAG_LEAF prefix (L) and E_0 (Z))
OP_SIZE OP_ONOTEQUAL OP_IF OP_SIZE 116 OP_EQUALVERIFY OP_DUP OP_8 OP_LEFT
  OP_SWAP OP_4 OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_SWAP OP_ELSE OP_DROP OP_3
  OP_PICK OP_0 OP_ENDIF

## OWNER_AUTH(d)   [shown for d = 0; the OP_ROLL depth is d+5]   (cohort rewrite: the next
     witness element is a signature by old.owner_key)
OP_5 OP_ROLL OP_2 OP_PICK 40 32 OP_SUBSTR OP_CHECKSIGVERIFY

## OWNER_AUTH_X(d)   [shown for d = 0; the OP_ROLL depth is d+5]   (OWNER_AUTH plus a
     signature by old.exit_key (REWRITE_h^x))
OWNER_AUTH(0)
OP_5 OP_ROLL OP_2 OP_PICK OP_8 32 OP_SUBSTR OP_CHECKSIGVERIFY
```

D.2 FULL Tier, Dense Profile ($D = 4$)

```
# EXIT_1 (dense, D=4): 1045 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_DUP OP_TOALTSTACK OP_IF OP_OVER 104 OP_4 OP_SUBSTR OP_CHECKLOCKTIMEVERIFY
  OP_DROP OP_OVER 72 32 OP_SUBSTR OP_ELSE OP_OVER 40 32 OP_SUBSTR OP_ENDIF
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY OP_1 <SEL_OUT_SINGLE> OP_TX
  <0x5120> OP_3 OP_PICK OP_8 32 OP_SUBSTR OP_CAT OP_EQUALVERIFY OP_OVER OP_8
  OP_LEFT OP_FROMALTSTACK OP_IF OP_NUMEQUALVERIFY OP_ELSE OP_LESSTHANOREQUAL
  OP_VERIFY OP_ENDIF
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL  x4
CHECK_OLD
FINAL

# EXIT_2 (dense, D=4): 1635 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_DUP OP_TOALTSTACK OP_IF OP_OVER 104 OP_4 OP_SUBSTR OP_CHECKLOCKTIMEVERIFY
  OP_DROP OP_OVER 72 32 OP_SUBSTR OP_ELSE OP_OVER 40 32 OP_SUBSTR OP_ENDIF
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY OP_1 <SEL_OUT_SINGLE> OP_TX
  <0x5120> OP_3 OP_PICK OP_8 32 OP_SUBSTR OP_CAT OP_EQUALVERIFY OP_OVER OP_8
  OP_LEFT OP_FROMALTSTACK OP_IF OP_NUMEQUALVERIFY OP_ELSE OP_LESSTHANOREQUAL
  OP_VERIFY OP_ENDIF
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL  x4
CHECK_OLD
OP_SWAP OP_TOALTSTACK OP_TOALTSTACK OP_DROP OP_DUP OP_TOALTSTACK OP_IF
  OP_OVER 104 OP_4 OP_SUBSTR OP_CHECKLOCKTIMEVERIFY OP_DROP OP_OVER 72 32
  OP_SUBSTR OP_ELSE OP_OVER 40 32 OP_SUBSTR OP_ENDIF OP_CHECKSIGVERIFY
  OP_SIZE 116 OP_EQUALVERIFY OP_2 <SEL_OUT_SINGLE> OP_TX <0x5120> OP_3
  OP_PICK OP_8 32 OP_SUBSTR OP_CAT OP_EQUALVERIFY OP_OVER OP_8 OP_LEFT
  OP_FROMALTSTACK OP_IF OP_NUMEQUALVERIFY OP_ELSE OP_LESSTHANOREQUAL
  OP_VERIFY OP_ENDIF
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL  x4
CHECK_PREV
FINAL

# JOIN_0 (dense, D=4): 961 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE OP_8 OP_EQUALVERIFY OP_OVER OP_SIZE
  32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_OLD
FINAL

# JOIN_2 (dense, D=4): 785 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE OP_8 OP_EQUALVERIFY OP_OVER OP_SIZE
  32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT <EMPTY_2> OP_0 OP_2SWAP
LEVEL  x2
CHECK_OLD
FINAL

# JOINK_2 (dense, D=4): 1433 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
```

```
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE OP_8 OP_EQUALVERIFY OP_OVER OP_SIZE
  32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_OLD
OP_SWAP OP_TOALTSTACK OP_TOALTSTACK OP_DROP OP_SIZE OP_8 OP_EQUALVERIFY
  OP_OVER OP_SIZE 32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT
  <EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_PREV
FINAL

# UPDATE (dense, D=4): 1081 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_2 OP_PICK 40 32 OP_SUBSTR
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY OP_SWAP OP_SIZE 116
  OP_EQUALVERIFY <TAG_NODE> OP_ROT OP_ROT
LEAF_ACC
OP_ROT
LEAF_ACC
LEVEL  x4
CHECK_OLD
FINAL

# UPDATE_X (exit-key consent, dense, D=4): 1088 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_4 OP_PICK OP_8 32 OP_SUBSTR OP_CHECKSIGVERIFY <OPERATOR_KEY>
  OP_CHECKSIGVERIFY OP_2 OP_PICK 40 32 OP_SUBSTR OP_CHECKSIGVERIFY OP_SIZE
  116 OP_EQUALVERIFY OP_SWAP OP_SIZE 116 OP_EQUALVERIFY <TAG_NODE> OP_ROT
  OP_ROT
LEAF_ACC
OP_ROT
LEAF_ACC
LEVEL  x4
CHECK_OLD
FINAL

# CLAIM (dense, D=4): 1762 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_2 OP_PICK OP_2 OP_PICK OP_CAT <TREE_ID> OP_SWAP OP_CAT
TAGGED(VOUCHER)
OP_3 OP_PICK 40 32 OP_SUBSTR OP_CHECKSIGFROMSTACK OP_VERIFY
CARVE_RULES
OP_OVER OP_8 OP_LEFT OP_OVER OP_8 OP_LEFT OP_SUB
LE64
OP_2 OP_PICK OP_8 104 OP_SUBSTR OP_CAT OP_2 OP_PICK 112 OP_4 OP_SUBSTR
  OP_1ADD OP_SIZE OP_4 OP_LESSTHANOREQUAL OP_VERIFY
PAD4
OP_CAT OP_FROMALTSTACK OP_ROT OP_TOALTSTACK OP_TOALTSTACK <TAG_NODE> OP_ROT
  OP_ROT OP_SWAP
LEAF_ACC
OP_ROT
LEAF_ACC
DRAIN_RULE
LEVEL  x4
CHECK_OLD
OP_FROMALTSTACK OP_2 OP_ROLL OP_2 OP_ROLL OP_SWAP OP_TOALTSTACK OP_TOALTSTACK
LEAF_ACC
<EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_PREV
FINAL
```

```
# CLAIM_H (hash-locked voucher, dense, D=4): 1770 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_SIZE 32 OP_EQUALVERIFY OP_SHA256 OP_TOALTSTACK OP_2 OP_PICK OP_2 OP_PICK
  OP_CAT <TREE_ID> OP_SWAP OP_CAT OP_FROMALTSTACK OP_CAT <TAG_VOUCHER_H>
  OP_SWAP OP_CAT OP_SHA256 OP_3 OP_PICK 40 32 OP_SUBSTR OP_CHECKSIGFROMSTACK
  OP_VERIFY
CARVE_RULES
OP_OVER OP_8 OP_LEFT OP_OVER OP_8 OP_LEFT OP_SUB
LE64
OP_2 OP_PICK OP_8 104 OP_SUBSTR OP_CAT OP_2 OP_PICK 112 OP_4 OP_SUBSTR
  OP_1ADD OP_SIZE OP_4 OP_LESSTHANOREQUAL OP_VERIFY
PAD4
OP_CAT OP_FROMALTSTACK OP_ROT OP_TOALTSTACK OP_TOALTSTACK <TAG_NODE> OP_ROT
  OP_ROT OP_SWAP
LEAF_ACC
OP_ROT
LEAF_ACC
DRAIN_RULE
LEVEL  x4
CHECK_OLD
OP_FROMALTSTACK OP_2 OP_ROLL OP_2 OP_ROLL OP_SWAP OP_TOALTSTACK OP_TOALTSTACK
LEAF_ACC
<EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_PREV
FINAL

# CLAIM_X (exit-key consent, dense, D=4): 1769 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_3 OP_PICK OP_8 32 OP_SUBSTR OP_CHECKSIGVERIFY OP_2 OP_PICK OP_2 OP_PICK
  OP_CAT <TREE_ID> OP_SWAP OP_CAT
TAGGED(VOUCHER)
OP_3 OP_PICK 40 32 OP_SUBSTR OP_CHECKSIGFROMSTACK OP_VERIFY
CARVE_RULES
OP_OVER OP_8 OP_LEFT OP_OVER OP_8 OP_LEFT OP_SUB
LE64
OP_2 OP_PICK OP_8 104 OP_SUBSTR OP_CAT OP_2 OP_PICK 112 OP_4 OP_SUBSTR
  OP_1ADD OP_SIZE OP_4 OP_LESSTHANOREQUAL OP_VERIFY
PAD4
OP_CAT OP_FROMALTSTACK OP_ROT OP_TOALTSTACK OP_TOALTSTACK <TAG_NODE> OP_ROT
  OP_ROT OP_SWAP
LEAF_ACC
OP_ROT
LEAF_ACC
DRAIN_RULE
LEVEL  x4
CHECK_OLD
OP_FROMALTSTACK OP_2 OP_ROLL OP_2 OP_ROLL OP_SWAP OP_TOALTSTACK OP_TOALTSTACK
LEAF_ACC
<EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_PREV
FINAL

# SPLIT (owner-authorised carve, dense, D=4): 1653 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_2 OP_PICK 40 32 OP_SUBSTR OP_CHECKSIGVERIFY
CARVE_RULES
OP_OVER OP_8 OP_LEFT OP_OVER OP_8 OP_LEFT OP_SUB
LE64
OP_2 OP_PICK OP_8 104 OP_SUBSTR OP_CAT OP_2 OP_PICK 112 OP_4 OP_SUBSTR
```

```
  OP_1ADD OP_SIZE OP_4 OP_LESSTHANOREQUAL OP_VERIFY
PAD4
OP_CAT OP_FROMALTSTACK OP_ROT OP_TOALTSTACK OP_TOALTSTACK <TAG_NODE> OP_ROT
  OP_ROT OP_SWAP
LEAF_ACC
OP_ROT
LEAF_ACC
DRAIN_RULE
LEVEL  x4
CHECK_OLD
OP_FROMALTSTACK OP_2 OP_ROLL OP_2 OP_ROLL OP_SWAP OP_TOALTSTACK OP_TOALTSTACK
LEAF_ACC
<EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_PREV
FINAL
```

D.3 FULL Tier, Cohort Shapes ($D = 8$, $h = 2$)

```
# EVICT_SUBTREE_2 (dense, D=8): cohort sweep of a 4-position subtree: 2005 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_1 OP_TOALTSTACK <TAG_NODE> OP_1 OP_ROLL
LEAF_STEP
OP_3 OP_ROLL
LEAF_STEP
HASH_STEP(4)
OP_3 OP_ROLL
LEAF_STEP
OP_5 OP_ROLL
LEAF_STEP
HASH_STEP(6)
HASH_STEP(4)
OP_FROMALTSTACK OP_DROP <EMPTY_2> OP_0
LEVEL  x6
CHECK_OLD
FINAL

# REWRITE_2 (dense, D=8): cohort rewrite of a 4-position subtree: 2071 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_1 OP_TOALTSTACK <EMPTY_0> <TAG_LEAF>
  <TAG_NODE>
REWRITE_STEP(0)
REWRITE_STEP(4)
HASH_STEP2(8)
REWRITE_STEP(4)
REWRITE_STEP(8)
HASH_STEP2(12)
HASH_STEP2(8)
OP_FROMALTSTACK OP_DROP OP_6 OP_ROLL OP_DROP OP_5 OP_ROLL OP_DROP
LEVEL  x6
CHECK_OLD
FINAL

# DETACH_2 (dense, D=8): detach a 4-position subtree into its own tree output: 1372 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE OP_8 OP_EQUALVERIFY OP_OVER OP_SIZE
  32 OP_EQUALVERIFY OP_DROP OP_OVER
  <c82f2b2516684bbef6dc1a319e3d1fd011cf4557613c9b6e8bd7c0b710101aa1>
TREE_SPK
OP_1 <SEL_OUT_SINGLE> OP_TX OP_ROT OP_EQUALVERIFY OP_OVER OP_NUMEQUALVERIFY
  <TAG_NODE> OP_ROT OP_ROT <EMPTY_2> OP_0
```

```
LEVEL  x6
CHECK_OLD
FINAL
```

D.4 FULL Tier, Sparse Profile ($D = 8$)

```
# EXIT_1 (sparse, D=8): 1892 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_DUP OP_TOALTSTACK OP_IF OP_OVER 104 OP_4 OP_SUBSTR OP_CHECKLOCKTIMEVERIFY
  OP_DROP OP_OVER 72 32 OP_SUBSTR OP_ELSE OP_OVER 40 32 OP_SUBSTR OP_ENDIF
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY OP_1 <SEL_OUT_SINGLE> OP_TX
  <0x5120> OP_3 OP_PICK OP_8 32 OP_SUBSTR OP_CAT OP_EQUALVERIFY OP_OVER OP_8
  OP_LEFT OP_FROMALTSTACK OP_IF OP_NUMEQUALVERIFY OP_ELSE OP_LESSTHANOREQUAL
  OP_VERIFY OP_ENDIF
LEAF_POS
SPARSE_SETUP
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL_S(k)  for k = 0 to 7
CHECK_OLD_S
FINAL

# JOIN_0 (sparse, D=8): 1876 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY
LEAF_POS
SPARSE_SETUP
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL_S(k)  for k = 0 to 7
CHECK_OLD_S
FINAL

# JOINK_2 (sparse, D=8): 3263 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY
LEAF_POS
SPARSE_SETUP
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL_S(k)  for k = 0 to 7
CHECK_OLD_S
OP_SWAP OP_TOALTSTACK OP_TOALTSTACK OP_DROP OP_SIZE 116 OP_EQUALVERIFY
LEAF_POS
SPARSE_SETUP
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL_S(k)  for k = 0 to 7
CHECK_PREV_S
FINAL

# UPDATE (sparse, D=8): 2092 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_2 OP_PICK 40 32 OP_SUBSTR
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY OP_SWAP OP_SIZE 116
  OP_EQUALVERIFY
LEAF_POS
OP_ROT
LEAF_POS
```

```
OP_ROT OP_EQUALVERIFY OP_SWAP
LEAF_POS
SPARSE_SETUP
<TAG_NODE> OP_ROT OP_ROT
LEAF_ACC
OP_ROT
LEAF_ACC
LEVEL_S(k)  for k = 0 to 7
CHECK_OLD_S
FINAL

# CLAIM (sparse, D=8): 3458 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_2 OP_PICK OP_2 OP_PICK OP_CAT <TREE_ID> OP_SWAP OP_CAT
TAGGED(VOUCHER)
OP_3 OP_PICK 40 32 OP_SUBSTR OP_CHECKSIGFROMSTACK OP_VERIFY
CARVE_RULES
OP_OVER OP_8 OP_LEFT OP_OVER OP_8 OP_LEFT OP_SUB
LE64
OP_2 OP_PICK OP_8 104 OP_SUBSTR OP_CAT OP_2 OP_PICK 112 OP_4 OP_SUBSTR
  OP_1ADD OP_SIZE OP_4 OP_LESSTHANOREQUAL OP_VERIFY
PAD4
OP_CAT OP_FROMALTSTACK OP_ROT OP_TOALTSTACK OP_TOALTSTACK OP_OVER
LEAF_POS
OP_NIP
SPARSE_SETUP
<TAG_NODE> OP_ROT OP_ROT OP_SWAP
LEAF_ACC
OP_ROT
LEAF_ACC
DRAIN_RULE
LEVEL_S(k)  for k = 0 to 7
CHECK_OLD_S
OP_FROMALTSTACK OP_2 OP_ROLL OP_2 OP_ROLL OP_SWAP OP_TOALTSTACK OP_TOALTSTACK
LEAF_POS
SPARSE_SETUP
LEAF_ACC
<EMPTY_0> OP_0 OP_2SWAP
LEVEL_S(k)  for k = 0 to 7
CHECK_PREV_S
FINAL
```

D.5 FULL Numerics Without Splice or EC Opcodes ($D = 4$)

The intermediate data point of §5: BIP-441 numerics kept, while `OP_SUBSTR`, `OP_LEFT`, `OP_RIGHT` and the EC point opcodes are replaced by `OP_CAT`, `OP_SIZE` and `OP_TWEAKADD`.

```
# EXIT_1 (minimal inventory, dense, D=4): 1118 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

<SEL_CURR_INPUT> OP_TX OP_0 OP_NUMEQUALVERIFY OP_2DUP
TREE_SPK_MIN
<SEL_IN_SPK> OP_TX OP_EQUALVERIFY OP_TOALTSTACK OP_TOALTSTACK OP_DUP
  OP_TOALTSTACK OP_IF OP_3 OP_PICK OP_CHECKLOCKTIMEVERIFY OP_DROP OP_4
  OP_PICK OP_ELSE OP_5 OP_PICK OP_ENDIF OP_CHECKSIGVERIFY OP_SIZE OP_4
  OP_EQUALVERIFY OP_OVER OP_SIZE OP_4 OP_EQUALVERIFY OP_DROP OP_2 OP_PICK
  OP_SIZE OP_4 OP_EQUALVERIFY OP_DROP OP_3 OP_PICK OP_SIZE 32 OP_EQUALVERIFY
  OP_DROP OP_4 OP_PICK OP_SIZE 32 OP_EQUALVERIFY OP_DROP OP_5 OP_PICK OP_SIZE
  32 OP_EQUALVERIFY OP_DROP OP_6 OP_PICK OP_SIZE OP_8 OP_EQUALVERIFY OP_DROP
  OP_6 OP_PICK OP_TOALTSTACK OP_5 OP_PICK OP_TOALTSTACK OP_CAT OP_CAT OP_CAT
  OP_CAT OP_CAT OP_CAT
TAGGED(LEAF)
OP_1 <SEL_OUT_SINGLE> OP_TX <0x5120> OP_FROMALTSTACK OP_CAT OP_EQUALVERIFY
  OP_FROMALTSTACK OP_FROMALTSTACK OP_IF OP_2DUP OP_NUMEQUALVERIFY OP_ELSE
```

```
  OP_2DUP OP_LESSTHANOREQUAL OP_VERIFY OP_ENDIF OP_NIP <TAG_NODE> OP_ROT
  OP_ROT <EMPTY_0> OP_0
LEVEL_MIN  x4
CHECK_OLD
OP_ROT OP_DROP OP_OVER <EMPTY_4> OP_EQUAL OP_IF OP_2DROP OP_FROMALTSTACK
  OP_DROP OP_1 OP_ELSE OP_SWAP OP_FROMALTSTACK
TREE_SPK_MIN
OP_0 <SEL_OUT_SINGLE> OP_TX OP_ROT OP_EQUALVERIFY OP_NUMEQUAL OP_ENDIF
```

D.6 MID Tier Macros

```
# Macro definitions, MID tier. Each expansion is itself collapsed
# against the other macros; a macro never contains itself.

## TAGGED(LEAF)
<TAG_LEAF> OP_SWAP OP_CAT OP_SHA256

## TAGGED(NODE)
<TAG_NODE> OP_SWAP OP_CAT OP_SHA256

## TAGGED(STATE)
<TAG_STATE> OP_SWAP OP_CAT OP_SHA256

## TAGGED(VOUCHER)
<TAG_VOUCHER> OP_SWAP OP_CAT OP_SHA256

## TAGGED(POS)
<TAG_POS> OP_SWAP OP_CAT OP_SHA256

## TAGGED(TAPTWEAK)
<TAG_TAPTWEAK> OP_SWAP OP_CAT OP_SHA256

## NONNEG   ([x] -> [x]: x == |x|, a minimal non-negative CScriptNum (rejects a negative, a
     negative zero and a non-minimal encoding))
OP_DUP OP_DUP OP_ABS OP_EQUALVERIFY

## PADZ   ([x z w] -> [le32(x)]: z||w == 00000000 and size(x)+size(z) == 4)
OP_OVER OP_SWAP OP_CAT <ZERO4> OP_EQUALVERIFY OP_SIZE OP_ROT OP_SIZE OP_ROT
  OP_ADD OP_4 OP_NUMEQUALVERIFY OP_SWAP OP_CAT

## ENC64   ([x z w] -> [enc64(x)])
PADZ
<00000000_01> OP_CAT

## LE64   ([x z w] -> [le64(x)])
PADZ
<ZERO4> OP_CAT

## TREE_SPK_MIN   ([r m] -> [TREE_SPK(r, m)] with OP_TWEAKADD)
OP_SWAP
TAGGED(STATE)
<NUMS_X> OP_TWEAKADD OP_TUCK OP_SWAP OP_CAT
TAGGED(TAPTWEAK)
OP_SWAP OP_TWEAKADD <0x5120> OP_SWAP OP_CAT

## PROLOGUE   (input is #0; (r, m) match the input scriptPubKey; alt <- m r)
32768 OP_TX OP_0 OP_NUMEQUALVERIFY OP_2DUP
TREE_SPK_MIN
<SEL_IN_SPK> OP_TX OP_EQUALVERIFY OP_TOALTSTACK OP_TOALTSTACK

## LEVEL_MID   (dense level with PADZ gadgets)
OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13
  OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_2 OP_PICK
NONNEG
OP_TOALTSTACK
```

```
ENC64
OP_CAT OP_FROMALTSTACK OP_7 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK OP_7 OP_ROLL
  OP_5 OP_ROLL OP_5 OP_ROLL
ENC64
OP_6 OP_ROLL OP_SWAP OP_CAT OP_2 OP_PICK OP_SWAP OP_4 OP_PICK OP_NOTIF
  OP_SWAP OP_ENDIF OP_CAT OP_8 OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_TOALTSTACK
  OP_5 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK OP_DROP OP_4 OP_ROLL OP_4 OP_ROLL
  OP_4 OP_ROLL
ENC64
OP_3 OP_ROLL OP_SWAP OP_CAT OP_ROT OP_NOTIF OP_SWAP OP_ENDIF OP_CAT OP_OVER
  OP_SWAP OP_CAT OP_SHA256 OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK

## LEVEL_MID_S   (sparse level: dir folded into the position accumulator)
OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13
  OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_3 OP_ROLL
  OP_FROMALTSTACK OP_SWAP OP_SIZE OP_0NOTEQUAL OP_NOTIF OP_DROP OP_DUP
  OP_ENDIF OP_SWAP <ENC0> OP_CAT OP_DUP OP_CAT
TAGGED(NODE)
OP_FROMALTSTACK OP_FROMALTSTACK OP_7 OP_PICK OP_IF OP_OVER OP_ADD OP_ENDIF
  OP_SWAP OP_DUP OP_ADD OP_SWAP OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK
  OP_3 OP_ROLL OP_3 OP_ROLL OP_3 OP_ROLL OP_2 OP_PICK
NONNEG
OP_TOALTSTACK
ENC64
OP_CAT OP_FROMALTSTACK OP_7 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK OP_7 OP_ROLL
  OP_5 OP_ROLL OP_5 OP_ROLL
ENC64
OP_6 OP_ROLL OP_SWAP OP_CAT OP_2 OP_PICK OP_SWAP OP_4 OP_PICK OP_NOTIF
  OP_SWAP OP_ENDIF OP_CAT OP_8 OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_TOALTSTACK
  OP_5 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK OP_DROP OP_4 OP_ROLL OP_4 OP_ROLL
  OP_4 OP_ROLL
ENC64
OP_3 OP_ROLL OP_SWAP OP_CAT OP_ROT OP_NOTIF OP_SWAP OP_ENDIF OP_CAT OP_OVER
  OP_SWAP OP_CAT OP_SHA256 OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK

## SPARSE_SETUP   ([ok nonce plo phi] -> [] ; alt <- plo 0 1 E_0)
OP_2SWAP OP_CAT
TAGGED(POS)
OP_ROT OP_DUP OP_TOALTSTACK OP_SIZE OP_4 OP_EQUALVERIFY OP_ROT OP_CAT
  OP_EQUALVERIFY OP_0 OP_TOALTSTACK OP_1 OP_TOALTSTACK <EMPTY_0>
  OP_TOALTSTACK

## SPARSE_TEARDOWN   (check the accumulated position against plo; drop bookkeeping)
OP_FROMALTSTACK OP_DROP OP_FROMALTSTACK OP_DROP OP_FROMALTSTACK
  OP_FROMALTSTACK OP_NUMEQUALVERIFY

## CHECK_OLD   (os (padded) == input 0 amount, oh == r)
OP_2SWAP OP_6 OP_ROLL OP_6 OP_ROLL
LE64
<SEL_IN_AMOUNT> OP_TX OP_EQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

## CHECK_OLD_S   (CHECK_OLD with the sparse teardown)
OP_2SWAP OP_6 OP_ROLL OP_6 OP_ROLL
LE64
<SEL_IN_AMOUNT> OP_TX OP_EQUALVERIFY
SPARSE_TEARDOWN
OP_FROMALTSTACK OP_EQUALVERIFY

## CHECK_PREV   (os == ns_prev, oh == nh_prev; chained steps)
OP_2SWAP OP_FROMALTSTACK OP_NUMEQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

## CHECK_PREV_S   (CHECK_PREV with the sparse teardown)
OP_2SWAP
SPARSE_TEARDOWN
OP_FROMALTSTACK OP_NUMEQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

## LEAF_JOIN   ([amt8 ek ok evk ea nonce seq amt_num za wa] -> [leaf] | amt_num)
OP_2 OP_PICK
```

```
NONNEG
OP_TOALTSTACK
LE64
OP_7 OP_PICK OP_EQUALVERIFY OP_DUP OP_SIZE OP_4 OP_EQUALVERIFY OP_DROP OP_1
  OP_PICK OP_SIZE OP_4 OP_EQUALVERIFY OP_DROP OP_2 OP_PICK OP_SIZE OP_4
  OP_EQUALVERIFY OP_DROP OP_3 OP_PICK OP_SIZE 32 OP_EQUALVERIFY OP_DROP OP_4
  OP_PICK OP_SIZE 32 OP_EQUALVERIFY OP_DROP OP_5 OP_PICK OP_SIZE 32
  OP_EQUALVERIFY OP_DROP OP_6 OP_PICK OP_SIZE OP_8 OP_EQUALVERIFY OP_DROP
  OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT

## CARVE_RULES   ([src(10) claim(10) extras(7) sea zse wse cea zce wce] -> [src(10)
     claim(10) extras(7)]: claim.evk == src.evk and claim.evict_after <= src.evict_after,
     each evict_after a minimal number checked against its field by PADZ (7.5 rule 5), and
     claim.amt_num >= DUST_MIN (rule 6))
19 OP_PICK 30 OP_PICK OP_EQUALVERIFY OP_2 OP_PICK OP_TOALTSTACK
PADZ
OP_16 OP_PICK OP_EQUALVERIFY OP_2 OP_PICK OP_TOALTSTACK
PADZ
23 OP_PICK OP_EQUALVERIFY OP_FROMALTSTACK OP_FROMALTSTACK
  OP_GREATERTHANOREQUAL OP_VERIFY 330 OP_10 OP_PICK OP_LESSTHANOREQUAL
  OP_VERIFY

## DRAIN_RULE   ([.. nh ns] -> [.. nh' ns]: a zero residual makes the new node E_0, any
     other is at least DUST_MIN (7.5 rules 2 and 6))
OP_DUP OP_NOT OP_IF OP_SWAP OP_DROP <EMPTY_0> OP_SWAP OP_ELSE 330 OP_OVER
  OP_LESSTHANOREQUAL OP_VERIFY OP_ENDIF

## FINAL   [shown for D = 4; <EMPTY_D> is the empty root of the tree's depth]   (output 0 is
     (le64(ns), TREE_SPK(nh, m)) unless nh == E_D)
OP_ROT OP_DROP OP_OVER <EMPTY_4> OP_EQUAL OP_IF OP_2DROP OP_2DROP
  OP_FROMALTSTACK OP_DROP OP_1 OP_ELSE OP_3 OP_ROLL OP_3 OP_ROLL
LE64
OP_SWAP OP_FROMALTSTACK
TREE_SPK_MIN
OP_0 <SEL_OUT_SINGLE> OP_TX OP_ROT OP_EQUALVERIFY OP_EQUAL OP_ENDIF

## EVICT_LEAF_STEP   (cohort sweep pre-pass: [slot idx] -> [idx'] | s sc, paying the next
     output)
OP_SWAP OP_IF OP_TOALTSTACK OP_2 OP_PICK OP_CHECKLOCKTIMEVERIFY OP_DROP
PADZ
OP_6 OP_PICK OP_EQUALVERIFY OP_6 OP_PICK <OPERATOR_KEY> OP_EQUALVERIFY
  OP_FROMALTSTACK OP_DUP OP_1ADD OP_TOALTSTACK <SEL_OUT_SINGLE> OP_TX
  <0x5120> OP_11 OP_PICK OP_CAT OP_EQUALVERIFY OP_10 OP_PICK OP_EQUALVERIFY
  OP_9 OP_PICK OP_1 OP_CAT OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK OP_FROMALTSTACK OP_ROT OP_SWAP OP_CAT OP_FROMALTSTACK OP_ROT
  OP_TOALTSTACK OP_SWAP OP_TOALTSTACK OP_ELSE OP_0 OP_TOALTSTACK <E_0.sc>
  OP_TOALTSTACK OP_ENDIF

## EVICT_HASH_STEP(4)
EVICT_HASH_ROOT(4)
OP_DUP OP_5 OP_ROLL OP_5 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(5)
EVICT_HASH_ROOT(5)
OP_DUP OP_6 OP_ROLL OP_6 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(6)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT OP_3 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP OP_7 OP_ROLL OP_7 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP
```

```
## EVICT_HASH_STEP(7)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT OP_4 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP OP_8 OP_ROLL OP_8 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(8)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT OP_5 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP OP_9 OP_ROLL OP_9 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(9)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT OP_6 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP OP_10 OP_ROLL OP_10 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(10)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT OP_7 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP OP_11 OP_ROLL OP_11 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(11)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT OP_8 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP OP_12 OP_ROLL OP_12 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(12)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT OP_9 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP OP_13 OP_ROLL OP_13 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(13)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT OP_10 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP OP_14 OP_ROLL OP_14 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(14)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT OP_11 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP OP_15 OP_ROLL OP_15 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(15)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT OP_12 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP OP_16 OP_ROLL OP_16 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(16)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT OP_13 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 17 OP_ROLL 17 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(17)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT OP_14 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 18 OP_ROLL 18 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(18)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT OP_15 OP_PICK OP_SWAP OP_CAT
```

```
  OP_SHA256 OP_FROMALTSTACK OP_DUP 19 OP_ROLL 19 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(19)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT OP_16 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 20 OP_ROLL 20 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(20)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 17 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 21 OP_ROLL 21 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(21)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 18 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 22 OP_ROLL 22 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(22)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 19 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 23 OP_ROLL 23 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(23)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 20 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 24 OP_ROLL 24 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(24)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 21 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 25 OP_ROLL 25 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(25)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 22 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 26 OP_ROLL 26 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(26)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 23 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 27 OP_ROLL 27 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(27)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 24 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 28 OP_ROLL 28 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(28)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 25 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 29 OP_ROLL 29 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(29)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 26 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 30 OP_ROLL 30 OP_ROLL
ENC64
```

```
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(30)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 27 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 31 OP_ROLL 31 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(31)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 28 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 32 OP_ROLL 32 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(32)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 29 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 33 OP_ROLL 33 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(33)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 30 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 34 OP_ROLL 34 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(34)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 31 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 35 OP_ROLL 35 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(35)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 32 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 36 OP_ROLL 36 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(36)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 33 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 37 OP_ROLL 37 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(37)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 34 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 38 OP_ROLL 38 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(38)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 35 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 39 OP_ROLL 39 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_STEP(39)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT 36 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_DUP 40 OP_ROLL 40 OP_ROLL
ENC64
OP_ROT OP_SWAP OP_CAT OP_SWAP

## EVICT_HASH_ROOT(4)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT OP_1 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK

## EVICT_HASH_ROOT(5)
OP_2 OP_ROLL OP_ADD OP_TOALTSTACK OP_SWAP OP_CAT OP_2 OP_PICK OP_SWAP OP_CAT
```

```
  OP_SHA256 OP_FROMALTSTACK
```

D.7 MID Tier, Dense Profile ($D = 4$)

```
# EXIT_1 (MID tier, dense, D=4): 1447 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_DUP OP_TOALTSTACK OP_IF OP_3 OP_PICK OP_CHECKLOCKTIMEVERIFY OP_DROP OP_10
  OP_PICK OP_CHECKSIGVERIFY
PADZ
OP_6 OP_PICK OP_EQUALVERIFY OP_ELSE OP_11 OP_PICK OP_CHECKSIGVERIFY OP_2DROP
  OP_DROP OP_ENDIF OP_1 <SEL_OUT_SINGLE> OP_TX <0x5120> OP_11 OP_PICK OP_CAT
  OP_EQUALVERIFY OP_10 OP_PICK OP_EQUALVERIFY OP_FROMALTSTACK OP_DROP
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL_MID  x4
CHECK_OLD
FINAL

# EXIT_2 (MID tier, dense, D=4): 2405 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_DUP OP_TOALTSTACK OP_IF OP_3 OP_PICK OP_CHECKLOCKTIMEVERIFY OP_DROP OP_10
  OP_PICK OP_CHECKSIGVERIFY
PADZ
OP_6 OP_PICK OP_EQUALVERIFY OP_ELSE OP_11 OP_PICK OP_CHECKSIGVERIFY OP_2DROP
  OP_DROP OP_ENDIF OP_1 <SEL_OUT_SINGLE> OP_TX <0x5120> OP_11 OP_PICK OP_CAT
  OP_EQUALVERIFY OP_10 OP_PICK OP_EQUALVERIFY OP_FROMALTSTACK OP_DROP
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL_MID  x4
CHECK_OLD
OP_SWAP OP_TOALTSTACK OP_TOALTSTACK OP_DROP OP_DUP OP_TOALTSTACK OP_IF OP_3
  OP_PICK OP_CHECKLOCKTIMEVERIFY OP_DROP OP_10 OP_PICK OP_CHECKSIGVERIFY
PADZ
OP_6 OP_PICK OP_EQUALVERIFY OP_ELSE OP_11 OP_PICK OP_CHECKSIGVERIFY OP_2DROP
  OP_DROP OP_ENDIF OP_2 <SEL_OUT_SINGLE> OP_TX <0x5120> OP_11 OP_PICK OP_CAT
  OP_EQUALVERIFY OP_10 OP_PICK OP_EQUALVERIFY OP_FROMALTSTACK OP_DROP
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL_MID  x4
CHECK_PREV
FINAL

# JOIN_0 (MID tier, dense, D=4): 1284 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY
NONNEG
OP_OVER OP_SIZE 32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT <EMPTY_0>
  OP_0 OP_2SWAP
LEVEL_MID  x4
CHECK_OLD
FINAL

# JOIN_2 (MID tier, dense, D=4): 964 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY
NONNEG
```

```
OP_OVER OP_SIZE 32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT <EMPTY_2>
  OP_0 OP_2SWAP
LEVEL_MID  x2
CHECK_OLD
FINAL

# JOINK_2 (MID tier, dense, D=4): 2045 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY
NONNEG
OP_OVER OP_SIZE 32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT <EMPTY_0>
  OP_0 OP_2SWAP
LEVEL_MID  x4
CHECK_OLD
OP_SWAP OP_TOALTSTACK OP_TOALTSTACK OP_DROP
NONNEG
OP_OVER OP_SIZE 32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT <EMPTY_0>
  OP_0 OP_2SWAP
LEVEL_MID  x4
CHECK_PREV
FINAL

# UPDATE (MID tier, dense, D=4): 1559 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY 18 OP_PICK OP_CHECKSIGVERIFY
LEAF_JOIN
OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_SWAP
TAGGED(LEAF)
OP_SWAP <TAG_NODE> OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL
LEVEL_MID  x4
CHECK_OLD
FINAL

# UPDATE_X (MID tier, exit-key consent, dense, D=4): 1563 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
21 OP_PICK OP_CHECKSIGVERIFY <OPERATOR_KEY> OP_CHECKSIGVERIFY 18 OP_PICK
  OP_CHECKSIGVERIFY
LEAF_JOIN
OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_SWAP
TAGGED(LEAF)
OP_SWAP <TAG_NODE> OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL
LEVEL_MID  x4
CHECK_OLD
FINAL

# CLAIM (MID tier, dense, D=4): 2686 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
33 OP_PICK 33 OP_PICK 33 OP_PICK 33 OP_PICK 33 OP_PICK 33 OP_PICK 33 OP_PICK
  OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT 24 OP_PICK 24 OP_PICK 24 OP_PICK
  24 OP_PICK 24 OP_PICK 24 OP_PICK 24 OP_PICK OP_CAT OP_CAT OP_CAT OP_CAT
  OP_CAT OP_CAT OP_CAT <TREE_ID> OP_SWAP OP_CAT
TAGGED(VOUCHER)
32 OP_PICK OP_CHECKSIGFROMSTACK OP_VERIFY
```

```
CARVE_RULES
20 OP_PICK OP_7 OP_PICK OP_7 OP_ROLL OP_7 OP_ROLL
PADZ
OP_EQUALVERIFY OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK
  OP_TOALTSTACK
LEAF_JOIN
OP_TOALTSTACK OP_8 OP_PICK OP_8 OP_PICK OP_8 OP_PICK OP_8 OP_PICK OP_8
  OP_PICK OP_CAT OP_CAT OP_CAT OP_CAT OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK OP_DUP OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_7 OP_PICK OP_TOALTSTACK OP_6 OP_PICK
  OP_TOALTSTACK OP_TOALTSTACK OP_6 OP_ROLL OP_DROP OP_7 OP_ROLL OP_6 OP_ROLL
  OP_SUB OP_DUP OP_0 OP_GREATERTHANOREQUAL OP_VERIFY OP_DUP OP_3 OP_ROLL OP_3
  OP_ROLL
LE64
OP_4 OP_ROLL OP_1ADD OP_4 OP_ROLL OP_4 OP_ROLL
PADZ
OP_3 OP_ROLL OP_SWAP OP_CAT OP_CAT
TAGGED(LEAF)
OP_SWAP
DRAIN_RULE
<TAG_NODE> OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL
LEVEL_MID  x4
CHECK_OLD
OP_FROMALTSTACK OP_FROMALTSTACK
TAGGED(LEAF)
OP_SWAP OP_2SWAP OP_SWAP OP_TOALTSTACK OP_TOALTSTACK <EMPTY_0> OP_0 OP_2SWAP
LEVEL_MID  x4
CHECK_PREV
FINAL

# CLAIM_H (MID tier, hash-locked voucher, dense, D=4): 2694 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_SIZE 32 OP_EQUALVERIFY OP_SHA256 OP_TOALTSTACK 33 OP_PICK 33 OP_PICK 33
  OP_PICK 33 OP_PICK 33 OP_PICK 33 OP_PICK 33 OP_PICK OP_CAT OP_CAT OP_CAT
  OP_CAT OP_CAT OP_CAT 24 OP_PICK 24 OP_PICK 24 OP_PICK 24 OP_PICK 24 OP_PICK
  24 OP_PICK 24 OP_PICK OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT
  <TREE_ID> OP_SWAP OP_CAT OP_FROMALTSTACK OP_CAT <TAG_VOUCHER_H> OP_SWAP
  OP_CAT OP_SHA256 32 OP_PICK OP_CHECKSIGFROMSTACK OP_VERIFY
CARVE_RULES
20 OP_PICK OP_7 OP_PICK OP_7 OP_ROLL OP_7 OP_ROLL
PADZ
OP_EQUALVERIFY OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK
  OP_TOALTSTACK
LEAF_JOIN
OP_TOALTSTACK OP_8 OP_PICK OP_8 OP_PICK OP_8 OP_PICK OP_8 OP_PICK OP_8
  OP_PICK OP_CAT OP_CAT OP_CAT OP_CAT OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK OP_DUP OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_7 OP_PICK OP_TOALTSTACK OP_6 OP_PICK
  OP_TOALTSTACK OP_TOALTSTACK OP_6 OP_ROLL OP_DROP OP_7 OP_ROLL OP_6 OP_ROLL
  OP_SUB OP_DUP OP_0 OP_GREATERTHANOREQUAL OP_VERIFY OP_DUP OP_3 OP_ROLL OP_3
  OP_ROLL
LE64
OP_4 OP_ROLL OP_1ADD OP_4 OP_ROLL OP_4 OP_ROLL
PADZ
OP_3 OP_ROLL OP_SWAP OP_CAT OP_CAT
TAGGED(LEAF)
OP_SWAP
DRAIN_RULE
<TAG_NODE> OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL
LEVEL_MID  x4
```

```
CHECK_OLD
OP_FROMALTSTACK OP_FROMALTSTACK
TAGGED(LEAF)
OP_SWAP OP_2SWAP OP_SWAP OP_TOALTSTACK OP_TOALTSTACK <EMPTY_0> OP_0 OP_2SWAP
LEVEL_MID  x4
CHECK_PREV
FINAL

# SPLIT (MID tier, owner-authorised carve, dense, D=4): 2527 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
31 OP_PICK OP_CHECKSIGVERIFY
CARVE_RULES
20 OP_PICK OP_7 OP_PICK OP_7 OP_ROLL OP_7 OP_ROLL
PADZ
OP_EQUALVERIFY OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK
  OP_TOALTSTACK
LEAF_JOIN
OP_TOALTSTACK OP_8 OP_PICK OP_8 OP_PICK OP_8 OP_PICK OP_8 OP_PICK OP_8
  OP_PICK OP_CAT OP_CAT OP_CAT OP_CAT OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK OP_DUP OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_7 OP_PICK OP_TOALTSTACK OP_6 OP_PICK
  OP_TOALTSTACK OP_TOALTSTACK OP_6 OP_ROLL OP_DROP OP_7 OP_ROLL OP_6 OP_ROLL
  OP_SUB OP_DUP OP_0 OP_GREATERTHANOREQUAL OP_VERIFY OP_DUP OP_3 OP_ROLL OP_3
  OP_ROLL
LE64
OP_4 OP_ROLL OP_1ADD OP_4 OP_ROLL OP_4 OP_ROLL
PADZ
OP_3 OP_ROLL OP_SWAP OP_CAT OP_CAT
TAGGED(LEAF)
OP_SWAP
DRAIN_RULE
<TAG_NODE> OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL
LEVEL_MID  x4
CHECK_OLD
OP_FROMALTSTACK OP_FROMALTSTACK
TAGGED(LEAF)
OP_SWAP OP_2SWAP OP_SWAP OP_TOALTSTACK OP_TOALTSTACK <EMPTY_0> OP_0 OP_2SWAP
LEVEL_MID  x4
CHECK_PREV
FINAL

# EVICT_SUBTREE_2 (MID tier, cohort sweep of a 4-position subtree, dense, D=4): 2249 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_1
EVICT_LEAF_STEP  x4
OP_DROP <TAG_NODE> OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK
EVICT_HASH_STEP(4)
OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
EVICT_HASH_STEP(6)
EVICT_HASH_ROOT(4)
<EMPTY_2> OP_0
LEVEL_MID  x2
CHECK_OLD
FINAL
```

D.8 MIN Tier Macros

```
# Macro definitions, MIN tier. Each expansion is itself collapsed
# against the other macros; a macro never contains itself.
```

```
## TAGGED(LEAF)
<TAG_LEAF> OP_SWAP OP_CAT OP_SHA256

## TAGGED(NODE)
<TAG_NODE> OP_SWAP OP_CAT OP_SHA256

## TAGGED(STATE)
<TAG_STATE> OP_SWAP OP_CAT OP_SHA256

## TAGGED(TAPTWEAK)
<TAG_TAPTWEAK> OP_SWAP OP_CAT OP_SHA256

## TAGGED(TAPSIGHASH)
<TAG_TAPSIGHASH> OP_SWAP OP_CAT OP_SHA256

## TAGGED(CHALLENGE)
<TAG_CHALLENGE> OP_SWAP OP_CAT OP_SHA256

## NONNEG   ([x] -> [x]: x == |x|, a minimal non-negative CScriptNum (rejects a negative, a
     negative zero and a non-minimal encoding))
OP_DUP OP_DUP OP_ABS OP_EQUALVERIFY

## PADZ   ([x z w] -> [le32(x)]: z||w == 00000000 and size(x)+size(z) == 4)
OP_OVER OP_SWAP OP_CAT <ZERO4> OP_EQUALVERIFY OP_SIZE OP_ROT OP_SIZE OP_ROT
  OP_ADD OP_4 OP_NUMEQUALVERIFY OP_SWAP OP_CAT

## LE64   ([x z w] -> [le64(x)])
PADZ
<ZERO4> OP_CAT

## TREE_SPK_MIN   ([r m] -> [TREE_SPK(r, m)] with OP_TWEAKADD)
OP_SWAP
TAGGED(STATE)
<NUMS_X> OP_TWEAKADD OP_TUCK OP_SWAP OP_CAT
TAGGED(TAPTWEAK)
OP_SWAP OP_TWEAKADD <0x5120> OP_SWAP OP_CAT

## LEVEL_MID   (dense level with PADZ gadgets)
OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13
  OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_2 OP_PICK
NONNEG
OP_TOALTSTACK
PADZ
<00000000_01> OP_CAT OP_CAT OP_FROMALTSTACK OP_7 OP_PICK OP_OVER OP_ADD
  OP_TOALTSTACK OP_7 OP_ROLL OP_5 OP_ROLL OP_5 OP_ROLL
PADZ
<00000000_01> OP_CAT OP_6 OP_ROLL OP_SWAP OP_CAT OP_2 OP_PICK OP_SWAP OP_4
  OP_PICK OP_NOTIF OP_SWAP OP_ENDIF OP_CAT OP_8 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_TOALTSTACK OP_5 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK OP_DROP
  OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL
PADZ
<00000000_01> OP_CAT OP_3 OP_ROLL OP_SWAP OP_CAT OP_ROT OP_NOTIF OP_SWAP
  OP_ENDIF OP_CAT OP_OVER OP_SWAP OP_CAT OP_SHA256 OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK

## LEAF_JOIN   ([amt8 ek ok evk ea nonce seq amt_num za wa] -> [leaf] | amt_num)
OP_2 OP_PICK
NONNEG
OP_TOALTSTACK
LE64
OP_7 OP_PICK OP_EQUALVERIFY OP_DUP OP_SIZE OP_4 OP_EQUALVERIFY OP_DROP OP_1
  OP_PICK OP_SIZE OP_4 OP_EQUALVERIFY OP_DROP OP_2 OP_PICK OP_SIZE OP_4
  OP_EQUALVERIFY OP_DROP OP_3 OP_PICK OP_SIZE 32 OP_EQUALVERIFY OP_DROP OP_4
  OP_PICK OP_SIZE 32 OP_EQUALVERIFY OP_DROP OP_5 OP_PICK OP_SIZE 32
  OP_EQUALVERIFY OP_DROP OP_6 OP_PICK OP_SIZE OP_8 OP_EQUALVERIFY OP_DROP
  OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT
```

```
## CARVE_RULES   ([src(10) claim(10) extras(7) sea zse wse cea zce wce] -> [src(10)
     claim(10) extras(7)]: claim.evk == src.evk and claim.evict_after <= src.evict_after,
     each evict_after a minimal number checked against its field by PADZ (7.5 rule 5), and
     claim.amt_num >= DUST_MIN (rule 6))
19 OP_PICK 30 OP_PICK OP_EQUALVERIFY OP_2 OP_PICK OP_TOALTSTACK
PADZ
OP_16 OP_PICK OP_EQUALVERIFY OP_2 OP_PICK OP_TOALTSTACK
PADZ
23 OP_PICK OP_EQUALVERIFY OP_FROMALTSTACK OP_FROMALTSTACK
  OP_GREATERTHANOREQUAL OP_VERIFY 330 OP_10 OP_PICK OP_LESSTHANOREQUAL
  OP_VERIFY

## DRAIN_RULE   ([.. nh ns] -> [.. nh' ns]: a zero residual makes the new node E_0, any
     other is at least DUST_MIN (7.5 rules 2 and 6))
OP_DUP OP_NOT OP_IF OP_SWAP OP_DROP <EMPTY_0> OP_SWAP OP_ELSE 330 OP_OVER
  OP_LESSTHANOREQUAL OP_VERIFY OP_ENDIF
```

D.9 MIN Tier, Dense Profile ($D = 4$)

```
# EXIT_1 (MIN tier, dense, D=4): 1720 bytes
# macros in CAPITALS are defined in MACROS_MIN.txt; raw opcodes are wrapped and indented

OP_2DUP
TREE_SPK_MIN
OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK OP_IF OP_3 OP_PICK
  OP_CHECKLOCKTIMEVERIFY OP_DROP OP_10 OP_PICK OP_CHECKSIGVERIFY
PADZ
OP_6 OP_PICK OP_EQUALVERIFY OP_ELSE OP_11 OP_PICK OP_CHECKSIGVERIFY OP_2DROP
  OP_DROP OP_ENDIF OP_9 OP_PICK 2117922 OP_CAT OP_9 OP_PICK OP_CAT
  OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL_MID  x4
OP_2SWAP OP_6 OP_ROLL OP_6 OP_ROLL
LE64
OP_FROMALTSTACK OP_FROMALTSTACK OP_3 OP_ROLL OP_EQUALVERIFY OP_4 OP_ROLL
  OP_DROP OP_ROT OP_5 OP_ROLL OP_5 OP_ROLL
LE64
OP_3 OP_ROLL OP_FROMALTSTACK
TREE_SPK_MIN
34 OP_SWAP OP_CAT OP_CAT OP_SWAP OP_CAT OP_2 OP_ROLL OP_CAT OP_SHA256 OP_2
  OP_ROLL OP_ROT OP_SWAP OP_CAT OP_SHA256 OP_FROMALTSTACK 34 OP_SWAP OP_CAT
  OP_3 OP_ROLL OP_CAT OP_SHA256 OP_3 OP_ROLL OP_4 OP_ROLL 0 OP_SWAP OP_CAT
  OP_SWAP OP_CAT OP_4 OP_ROLL OP_CAT OP_ROT OP_CAT OP_SWAP OP_CAT OP_3
  OP_ROLL OP_CAT OP_SWAP OP_CAT <0200000000> OP_CAT OP_SWAP OP_CAT
  <00ffffffff> OP_CAT
TAGGED(TAPSIGHASH)
<G_X||G_X> OP_SWAP OP_CAT
TAGGED(CHALLENGE)
OP_OVER 0 OP_CAT OP_EQUALVERIFY <G_X> OP_SWAP OP_CAT OP_1 OP_CAT <G_X>
  OP_CHECKSIG

# SPLIT (MIN tier, owner-authorised carve, dense, D=4): 2811 bytes
# macros in CAPITALS are defined in MACROS_MIN.txt; raw opcodes are wrapped and indented

OP_2DUP
TREE_SPK_MIN
OP_TOALTSTACK OP_DUP OP_TOALTSTACK OP_SWAP OP_TOALTSTACK OP_DROP 31 OP_PICK
  OP_CHECKSIGVERIFY
CARVE_RULES
20 OP_PICK OP_7 OP_PICK OP_7 OP_ROLL OP_7 OP_ROLL
PADZ
OP_EQUALVERIFY OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK
  OP_TOALTSTACK
LEAF_JOIN
OP_TOALTSTACK OP_8 OP_PICK OP_8 OP_PICK OP_8 OP_PICK OP_8 OP_PICK OP_8
```

```
  OP_PICK OP_CAT OP_CAT OP_CAT OP_CAT OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK OP_DUP OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_7 OP_PICK OP_TOALTSTACK OP_6 OP_PICK
  OP_TOALTSTACK OP_TOALTSTACK OP_6 OP_ROLL OP_DROP OP_7 OP_ROLL OP_6 OP_ROLL
  OP_SUB OP_DUP OP_0 OP_GREATERTHANOREQUAL OP_VERIFY OP_DUP OP_3 OP_ROLL OP_3
  OP_ROLL
LE64
OP_4 OP_ROLL OP_1ADD OP_4 OP_ROLL OP_4 OP_ROLL
PADZ
OP_3 OP_ROLL OP_SWAP OP_CAT OP_CAT
TAGGED(LEAF)
OP_SWAP
DRAIN_RULE
<TAG_NODE> OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL
LEVEL_MID  x4
OP_2SWAP OP_6 OP_ROLL OP_6 OP_ROLL
LE64
OP_SWAP OP_FROMALTSTACK OP_EQUALVERIFY OP_FROMALTSTACK OP_FROMALTSTACK OP_ROT
  OP_TOALTSTACK
TAGGED(LEAF)
OP_SWAP OP_2SWAP OP_SWAP OP_TOALTSTACK OP_TOALTSTACK <EMPTY_0> OP_0 OP_2SWAP
LEVEL_MID  x4
OP_2SWAP OP_FROMALTSTACK OP_NUMEQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY
  OP_ROT OP_DROP OP_3 OP_ROLL OP_3 OP_ROLL
LE64
OP_SWAP OP_FROMALTSTACK OP_FROMALTSTACK OP_ROT OP_SWAP
TREE_SPK_MIN
34 OP_SWAP OP_CAT OP_ROT OP_SWAP OP_CAT OP_2 OP_ROLL OP_CAT OP_SHA256 OP_2
  OP_ROLL OP_ROT OP_SWAP OP_CAT OP_SHA256 OP_FROMALTSTACK 34 OP_SWAP OP_CAT
  OP_3 OP_ROLL OP_CAT OP_SHA256 OP_3 OP_ROLL OP_4 OP_ROLL 0 OP_SWAP OP_CAT
  OP_SWAP OP_CAT OP_4 OP_ROLL OP_CAT OP_ROT OP_CAT OP_SWAP OP_CAT OP_3
  OP_ROLL OP_CAT OP_SWAP OP_CAT <0200000000> OP_CAT OP_SWAP OP_CAT
  <00ffffffff> OP_CAT
TAGGED(TAPSIGHASH)
<G_X||G_X> OP_SWAP OP_CAT
TAGGED(CHALLENGE)
OP_OVER 0 OP_CAT OP_EQUALVERIFY <G_X> OP_SWAP OP_CAT OP_1 OP_CAT <G_X>
  OP_CHECKSIG
```

D.10 CCV Tier Macros

```
# Macro definitions, CCV tier. Each expansion is itself collapsed
# against the other macros; a macro never contains itself.

## TAGGED(LEAF)
<TAG_LEAF> OP_SWAP OP_CAT OP_SHA256

## TAGGED(NODE)
<TAG_NODE> OP_SWAP OP_CAT OP_SHA256

## TAGGED(POS)
<TAG_POS> OP_SWAP OP_CAT OP_SHA256

## TAGGED(TAPTWEAK)
<TAG_TAPTWEAK> OP_SWAP OP_CAT OP_SHA256

## TAGGED(VOUCHER)
<TAG_VOUCHER> OP_SWAP OP_CAT OP_SHA256

## TAGGED(VOUCHER_H)
<TAG_VOUCHER_H> OP_SWAP OP_CAT OP_SHA256

## CCV_CHECK_INPUT   (BIP-443 CHECK_INPUT: [data] -> [], binds the old root to the current
     input under its own taptree)
129 OP_0 129 129 OP_CHECKCONTRACTVERIFY
```

```
## CCV_PAYOUT(1)
OP_DUP OP_TOALTSTACK OP_TOALTSTACK OP_0 OP_1 OP_FROMALTSTACK OP_8 32
  OP_SUBSTR OP_0 OP_2 OP_CHECKCONTRACTVERIFY OP_FROMALTSTACK

## CCV_PAYOUT(2)
OP_DUP OP_TOALTSTACK OP_TOALTSTACK OP_0 OP_2 OP_FROMALTSTACK OP_8 32
  OP_SUBSTR OP_0 OP_2 OP_CHECKCONTRACTVERIFY OP_FROMALTSTACK

## LEAF_ACC
OP_DUP OP_8 OP_LEFT OP_SWAP
TAGGED(LEAF)
OP_SWAP

## LEAF_POS
OP_DUP 40 32 OP_SUBSTR OP_OVER 108 OP_4 OP_SUBSTR OP_CAT
TAGGED(POS)

## LEVEL
OP_6 OP_ROLL OP_6 OP_ROLL OP_SIZE 41 OP_EQUALVERIFY OP_DUP 32 OP_8 OP_SUBSTR
  OP_DUP OP_TOALTSTACK OP_3 OP_ROLL OP_TUCK OP_ADD OP_TOALTSTACK <ENC0>
  OP_ADD OP_3 OP_ROLL OP_SWAP OP_CAT OP_OVER OP_SWAP OP_3 OP_PICK OP_NOTIF
  OP_SWAP OP_ENDIF OP_CAT OP_5 OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_TOALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_5 OP_ROLL OP_TUCK OP_ADD
  OP_TOALTSTACK <ENC0> OP_ADD OP_5 OP_ROLL OP_SWAP OP_CAT OP_3 OP_ROLL
  OP_SWAP OP_4 OP_ROLL OP_NOTIF OP_SWAP OP_ENDIF OP_CAT OP_3 OP_PICK OP_SWAP
  OP_CAT OP_SHA256 OP_FROMALTSTACK OP_2SWAP

## SPARSE_SETUP
OP_TOALTSTACK <EMPTY_0> OP_TOALTSTACK

## SPARSE_TEARDOWN
OP_FROMALTSTACK OP_DROP OP_FROMALTSTACK OP_DROP

## CHECK_PREV
OP_2SWAP OP_FROMALTSTACK OP_NUMEQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

## CHECK_PREV_S
OP_2SWAP
SPARSE_TEARDOWN
OP_FROMALTSTACK OP_NUMEQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

## CHECK_OLD   (the input-amount equality plus one CCV_CHECK_INPUT on the folded old root)
OP_2SWAP <SEL_IN_AMOUNT> OP_TX OP_NUMEQUALVERIFY OP_ROT OP_ROT OP_TOALTSTACK
  OP_TOALTSTACK
CCV_CHECK_INPUT
OP_FROMALTSTACK OP_FROMALTSTACK

## CHECK_OLD_S
OP_2SWAP <SEL_IN_AMOUNT> OP_TX OP_NUMEQUALVERIFY
SPARSE_TEARDOWN
OP_ROT OP_ROT OP_TOALTSTACK OP_TOALTSTACK
CCV_CHECK_INPUT
OP_FROMALTSTACK OP_FROMALTSTACK

## PROLOGUE_CCV   (empty: CCV's -1 index binds the current input without OP_TX)

## CARVE_RULES   ([src claim] -> [src claim]: both 116 bytes, claim.evictor_key ==
     src.evictor_key, claim.evict_after <= src.evict_after (7.5 rule 5), claim.amount >=
     DUST_MIN (7.5 rule 6))
OP_SIZE 116 OP_EQUALVERIFY OP_SWAP OP_SIZE 116 OP_EQUALVERIFY OP_SWAP OP_2DUP
  72 32 OP_SUBSTR OP_SWAP 72 32 OP_SUBSTR OP_EQUALVERIFY OP_2DUP 104 OP_4
  OP_SUBSTR OP_SWAP 104 OP_4 OP_SUBSTR OP_LESSTHANOREQUAL OP_VERIFY 330
  OP_OVER OP_8 OP_LEFT OP_LESSTHANOREQUAL OP_VERIFY

## DRAIN_RULE   ([.. nh ns] -> [.. nh' ns]: a zero residual makes the new node E_0, any
     other is at least DUST_MIN (7.5 rules 2 and 6))
OP_DUP OP_NOT OP_IF OP_SWAP OP_DROP <EMPTY_0> OP_SWAP OP_ELSE 330 OP_OVER
```

```
  OP_LESSTHANOREQUAL OP_VERIFY OP_ENDIF

## LEVEL_S(k)   [shown for k = 0; <EMPTY_k> and the bit index vary with k]
OP_5 OP_ROLL OP_FROMALTSTACK OP_SWAP OP_SIZE OP_ONOTEQUAL OP_NOTIF OP_DROP
  OP_DUP <ENC0> OP_CAT OP_ENDIF OP_SWAP <ENC0> OP_CAT OP_DUP OP_CAT OP_6
  OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_FROMALTSTACK OP_DUP OP_TOALTSTACK
  OP_SWAP OP_TOALTSTACK OP_0 OP_DOWNSHIFT OP_1 OP_AND OP_ONOTEQUAL OP_SWAP
  OP_SIZE 41 OP_EQUALVERIFY OP_DUP 32 OP_8 OP_SUBSTR OP_DUP OP_TOALTSTACK
  OP_3 OP_ROLL OP_TUCK OP_ADD OP_TOALTSTACK <ENC0> OP_ADD OP_3 OP_ROLL
  OP_SWAP OP_CAT OP_OVER OP_SWAP OP_3 OP_PICK OP_NOTIF OP_SWAP OP_ENDIF
  OP_CAT OP_5 OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_TOALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_5 OP_ROLL OP_TUCK OP_ADD OP_TOALTSTACK
  <ENC0> OP_ADD OP_5 OP_ROLL OP_SWAP OP_CAT OP_3 OP_ROLL OP_SWAP OP_4 OP_ROLL
  OP_NOTIF OP_SWAP OP_ENDIF OP_CAT OP_3 OP_PICK OP_SWAP OP_CAT OP_SHA256
  OP_FROMALTSTACK OP_2SWAP

## FINAL   [shown for D = 4; <EMPTY_D> is the empty root of the tree's depth]   (output 0 is
     the CCV assign-mode tree output with amount == ns, unless nh == E_D)
OP_ROT OP_DROP OP_OVER <EMPTY_4> OP_EQUAL OP_IF OP_2DROP OP_1 OP_ELSE
  OP_TOALTSTACK OP_0 OP_0 129 OP_0 OP_CHECKCONTRACTVERIFY OP_FROMALTSTACK
  OP_0 <SEL_OUT_SINGLE> OP_TX OP_DROP OP_NUMEQUALVERIFY OP_1 OP_ENDIF

## LEVEL_MIN
OP_7 OP_ROLL OP_7 OP_ROLL OP_7 OP_ROLL OP_SIZE OP_8 OP_EQUALVERIFY OP_OVER
  OP_SIZE 32 OP_EQUALVERIFY OP_DROP OP_DUP OP_TOALTSTACK OP_SWAP OP_OVER
  <ENC0> OP_ADD OP_CAT OP_SWAP OP_3 OP_ROLL OP_TUCK OP_ADD OP_TOALTSTACK
  <ENC0> OP_ADD OP_3 OP_ROLL OP_SWAP OP_CAT OP_OVER OP_SWAP OP_3 OP_PICK
  OP_NOTIF OP_SWAP OP_ENDIF OP_CAT OP_5 OP_PICK OP_SWAP OP_CAT OP_SHA256
  OP_TOALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_5 OP_ROLL
  OP_TUCK OP_ADD OP_TOALTSTACK <ENC0> OP_ADD OP_5 OP_ROLL OP_SWAP OP_CAT OP_3
  OP_ROLL OP_SWAP OP_4 OP_ROLL OP_NOTIF OP_SWAP OP_ENDIF OP_CAT OP_3 OP_PICK
  OP_SWAP OP_CAT OP_SHA256 OP_FROMALTSTACK OP_2SWAP

## LEAF_STEP
OP_SIZE OP_ONOTEQUAL OP_IF OP_SIZE 116 OP_EQUALVERIFY OP_DUP 72 32 OP_SUBSTR
  <OPERATOR_KEY> OP_EQUALVERIFY OP_DUP 104 OP_4 OP_SUBSTR
  OP_CHECKLOCKTIMEVERIFY OP_DROP OP_FROMALTSTACK OP_DUP OP_1ADD OP_TOALTSTACK
  OP_DUP OP_TOALTSTACK OP_0 OP_SWAP OP_2 OP_PICK OP_8 32 OP_SUBSTR OP_0 OP_2
  OP_CHECKCONTRACTVERIFY OP_FROMALTSTACK <SEL_OUT_SINGLE> OP_TX OP_DROP
  OP_OVER OP_8 OP_LEFT OP_NUMEQUALVERIFY
LEAF_ACC
OP_ELSE OP_DROP <EMPTY_0> OP_0 OP_ENDIF
```

D.11 CCV Tier, 0xC2 Numerics, Dense Profile ($D = 4$)

```
# EXIT_1 (CCV tier, dense, D=4): 660 bytes
# macros in CAPITALS are defined in MACROS_CCV.txt; raw opcodes are wrapped and indented

OP_DUP OP_TOALTSTACK OP_IF OP_OVER 104 OP_4 OP_SUBSTR OP_CHECKLOCKTIMEVERIFY
  OP_DROP OP_OVER 72 32 OP_SUBSTR OP_ELSE OP_OVER 40 32 OP_SUBSTR OP_ENDIF
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY
CCV_PAYOUT(1)
OP_1 <SEL_OUT_SINGLE> OP_TX OP_DROP OP_OVER OP_8 OP_LEFT OP_FROMALTSTACK
  OP_DROP OP_NUMEQUALVERIFY
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL  x4
CHECK_OLD
FINAL

# EXIT_2 (CCV tier, dense, D=4): 1250 bytes
# macros in CAPITALS are defined in MACROS_CCV.txt; raw opcodes are wrapped and indented

OP_DUP OP_TOALTSTACK OP_IF OP_OVER 104 OP_4 OP_SUBSTR OP_CHECKLOCKTIMEVERIFY
  OP_DROP OP_OVER 72 32 OP_SUBSTR OP_ELSE OP_OVER 40 32 OP_SUBSTR OP_ENDIF
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY
CCV_PAYOUT(1)
```

```
OP_1 <SEL_OUT_SINGLE> OP_TX OP_DROP OP_OVER OP_8 OP_LEFT OP_FROMALTSTACK
  OP_DROP OP_NUMEQUALVERIFY
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL  x4
CHECK_OLD
OP_SWAP OP_TOALTSTACK OP_TOALTSTACK OP_DROP OP_DUP OP_TOALTSTACK OP_IF
  OP_OVER 104 OP_4 OP_SUBSTR OP_CHECKLOCKTIMEVERIFY OP_DROP OP_OVER 72 32
  OP_SUBSTR OP_ELSE OP_OVER 40 32 OP_SUBSTR OP_ENDIF OP_CHECKSIGVERIFY
  OP_SIZE 116 OP_EQUALVERIFY
CCV_PAYOUT(2)
OP_2 <SEL_OUT_SINGLE> OP_TX OP_DROP OP_OVER OP_8 OP_LEFT OP_FROMALTSTACK
  OP_DROP OP_NUMEQUALVERIFY
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL  x4
CHECK_PREV
FINAL

# JOIN_0 (CCV tier, dense, D=4): 576 bytes
# macros in CAPITALS are defined in MACROS_CCV.txt; raw opcodes are wrapped and indented

<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE OP_8 OP_EQUALVERIFY OP_OVER OP_SIZE
  32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_OLD
FINAL

# JOIN_2 (CCV tier, dense, D=4): 400 bytes
# macros in CAPITALS are defined in MACROS_CCV.txt; raw opcodes are wrapped and indented

<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE OP_8 OP_EQUALVERIFY OP_OVER OP_SIZE
  32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT <EMPTY_2> OP_0 OP_2SWAP
LEVEL  x2
CHECK_OLD
FINAL

# JOINK_2 (CCV tier, dense, D=4): 1048 bytes
# macros in CAPITALS are defined in MACROS_CCV.txt; raw opcodes are wrapped and indented

<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE OP_8 OP_EQUALVERIFY OP_OVER OP_SIZE
  32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_OLD
OP_SWAP OP_TOALTSTACK OP_TOALTSTACK OP_DROP OP_SIZE OP_8 OP_EQUALVERIFY
  OP_OVER OP_SIZE 32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT
  <EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_PREV
FINAL

# UPDATE (CCV tier, dense, D=4): 696 bytes
# macros in CAPITALS are defined in MACROS_CCV.txt; raw opcodes are wrapped and indented

<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_2 OP_PICK 40 32 OP_SUBSTR
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY OP_SWAP OP_SIZE 116
  OP_EQUALVERIFY <TAG_NODE> OP_ROT OP_ROT
LEAF_ACC
OP_ROT
LEAF_ACC
LEVEL  x4
CHECK_OLD
FINAL

# UPDATE_X (CCV tier, exit-key consent, dense, D=4): 703 bytes
# macros in CAPITALS are defined in MACROS_CCV.txt; raw opcodes are wrapped and indented
```

```
OP_4 OP_PICK OP_8 32 OP_SUBSTR OP_CHECKSIGVERIFY <OPERATOR_KEY>
  OP_CHECKSIGVERIFY OP_2 OP_PICK 40 32 OP_SUBSTR OP_CHECKSIGVERIFY OP_SIZE
  116 OP_EQUALVERIFY OP_SWAP OP_SIZE 116 OP_EQUALVERIFY <TAG_NODE> OP_ROT
  OP_ROT
LEAF_ACC
OP_ROT
LEAF_ACC
LEVEL  x4
CHECK_OLD
FINAL

# CLAIM (CCV tier, dense, D=4): 1375 bytes
# macros in CAPITALS are defined in MACROS_CCV.txt; raw opcodes are wrapped and indented

OP_2 OP_PICK OP_2 OP_PICK OP_CAT <TREE_ID> OP_SWAP OP_CAT
TAGGED(VOUCHER)
OP_3 OP_PICK 40 32 OP_SUBSTR OP_CHECKSIGFROMSTACK OP_VERIFY
CARVE_RULES
OP_OVER OP_8 OP_LEFT OP_OVER OP_8 OP_LEFT OP_SUB <ENC0> OP_ADD OP_8 OP_LEFT
  OP_2 OP_PICK OP_8 104 OP_SUBSTR OP_CAT OP_2 OP_PICK 112 OP_4 OP_SUBSTR
  OP_1ADD OP_SIZE OP_4 OP_LESSTHANOREQUAL OP_VERIFY <ZERO4> OP_CAT OP_4
  OP_LEFT OP_CAT OP_SWAP OP_TOALTSTACK <TAG_NODE> OP_ROT OP_ROT OP_SWAP
LEAF_ACC
OP_ROT
LEAF_ACC
DRAIN_RULE
LEVEL  x4
CHECK_OLD
OP_FROMALTSTACK OP_2 OP_ROLL OP_2 OP_ROLL OP_SWAP OP_TOALTSTACK OP_TOALTSTACK
LEAF_ACC
<EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_PREV
FINAL

# CLAIM_H (CCV tier, dense, D=4): 1383 bytes
# macros in CAPITALS are defined in MACROS_CCV.txt; raw opcodes are wrapped and indented

OP_SIZE 32 OP_EQUALVERIFY OP_SHA256 OP_TOALTSTACK OP_2 OP_PICK OP_2 OP_PICK
  OP_CAT <TREE_ID> OP_SWAP OP_CAT OP_FROMALTSTACK OP_CAT
TAGGED(VOUCHER_H)
OP_3 OP_PICK 40 32 OP_SUBSTR OP_CHECKSIGFROMSTACK OP_VERIFY
CARVE_RULES
OP_OVER OP_8 OP_LEFT OP_OVER OP_8 OP_LEFT OP_SUB <ENC0> OP_ADD OP_8 OP_LEFT
  OP_2 OP_PICK OP_8 104 OP_SUBSTR OP_CAT OP_2 OP_PICK 112 OP_4 OP_SUBSTR
  OP_1ADD OP_SIZE OP_4 OP_LESSTHANOREQUAL OP_VERIFY <ZERO4> OP_CAT OP_4
  OP_LEFT OP_CAT OP_SWAP OP_TOALTSTACK <TAG_NODE> OP_ROT OP_ROT OP_SWAP
LEAF_ACC
OP_ROT
LEAF_ACC
DRAIN_RULE
LEVEL  x4
CHECK_OLD
OP_FROMALTSTACK OP_2 OP_ROLL OP_2 OP_ROLL OP_SWAP OP_TOALTSTACK OP_TOALTSTACK
LEAF_ACC
<EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_PREV
FINAL

# SPLIT (CCV tier, owner-authorised carve, dense, D=4): 1266 bytes
# macros in CAPITALS are defined in MACROS_CCV.txt; raw opcodes are wrapped and indented

OP_2 OP_PICK 40 32 OP_SUBSTR OP_CHECKSIGVERIFY
CARVE_RULES
OP_OVER OP_8 OP_LEFT OP_OVER OP_8 OP_LEFT OP_SUB <ENC0> OP_ADD OP_8 OP_LEFT
  OP_2 OP_PICK OP_8 104 OP_SUBSTR OP_CAT OP_2 OP_PICK 112 OP_4 OP_SUBSTR
```

```
  OP_1ADD OP_SIZE OP_4 OP_LESSTHANOREQUAL OP_VERIFY <ZERO4> OP_CAT OP_4
  OP_LEFT OP_CAT OP_SWAP OP_TOALTSTACK <TAG_NODE> OP_ROT OP_ROT OP_SWAP
LEAF_ACC
OP_ROT
LEAF_ACC
DRAIN_RULE
LEVEL  x4
CHECK_OLD
OP_FROMALTSTACK OP_2 OP_ROLL OP_2 OP_ROLL OP_SWAP OP_TOALTSTACK OP_TOALTSTACK
LEAF_ACC
<EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_PREV
FINAL

# EVICT_SUBTREE_1 (CCV tier, dense, D=4): 906 bytes
# macros in CAPITALS are defined in MACROS_CCV.txt; raw opcodes are wrapped and indented

<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_1 OP_TOALTSTACK <TAG_NODE> OP_1 OP_ROLL
LEAF_STEP
OP_3 OP_ROLL
LEAF_STEP
OP_2 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK <ENC0> OP_ADD OP_CAT OP_SWAP <ENC0>
  OP_ADD OP_ROT OP_SWAP OP_CAT OP_SWAP OP_CAT OP_1 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_FROMALTSTACK OP_DROP <EMPTY_1> OP_0
LEVEL  x3
CHECK_OLD
FINAL

# EVICT_SUBTREE_2 (CCV tier, dense, D=4): 1284 bytes
# macros in CAPITALS are defined in MACROS_CCV.txt; raw opcodes are wrapped and indented

<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_1 OP_TOALTSTACK <TAG_NODE> OP_1 OP_ROLL
LEAF_STEP
OP_3 OP_ROLL
LEAF_STEP
OP_2 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK <ENC0> OP_ADD OP_CAT OP_SWAP <ENC0>
  OP_ADD OP_ROT OP_SWAP OP_CAT OP_SWAP OP_CAT OP_1 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_3 OP_ROLL
LEAF_STEP
OP_5 OP_ROLL
LEAF_STEP
OP_2 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK <ENC0> OP_ADD OP_CAT OP_SWAP <ENC0>
  OP_ADD OP_ROT OP_SWAP OP_CAT OP_SWAP OP_CAT OP_3 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_FROMALTSTACK OP_2 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK <ENC0>
  OP_ADD OP_CAT OP_SWAP <ENC0> OP_ADD OP_ROT OP_SWAP OP_CAT OP_SWAP OP_CAT
  OP_1 OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_FROMALTSTACK OP_FROMALTSTACK
  OP_DROP <EMPTY_2> OP_0
LEVEL  x2
CHECK_OLD
FINAL
```

D.12 CCV Tier, 0xC2 Numerics, Sparse Profile ($D = 8$)

```
# EXIT_1 (CCV tier, sparse, D=8): 1507 bytes
# macros in CAPITALS are defined in MACROS_CCV.txt; raw opcodes are wrapped and indented

OP_DUP OP_TOALTSTACK OP_IF OP_OVER 104 OP_4 OP_SUBSTR OP_CHECKLOCKTIMEVERIFY
  OP_DROP OP_OVER 72 32 OP_SUBSTR OP_ELSE OP_OVER 40 32 OP_SUBSTR OP_ENDIF
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY
CCV_PAYOUT(1)
OP_1 <SEL_OUT_SINGLE> OP_TX OP_DROP OP_OVER OP_8 OP_LEFT OP_FROMALTSTACK
  OP_DROP OP_NUMEQUALVERIFY
LEAF_POS
SPARSE_SETUP
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
```

```
LEVEL_S(k)  for k = 0 to 7
CHECK_OLD_S
FINAL

# JOIN_0 (CCV tier, sparse, D=8): 1491 bytes
# macros in CAPITALS are defined in MACROS_CCV.txt; raw opcodes are wrapped and indented

<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY
LEAF_POS
SPARSE_SETUP
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL_S(k)  for k = 0 to 7
CHECK_OLD_S
FINAL

# JOINK_2 (CCV tier, sparse, D=8): 2878 bytes
# macros in CAPITALS are defined in MACROS_CCV.txt; raw opcodes are wrapped and indented

<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY
LEAF_POS
SPARSE_SETUP
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL_S(k)  for k = 0 to 7
CHECK_OLD_S
OP_SWAP OP_TOALTSTACK OP_TOALTSTACK OP_DROP OP_SIZE 116 OP_EQUALVERIFY
LEAF_POS
SPARSE_SETUP
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL_S(k)  for k = 0 to 7
CHECK_PREV_S
FINAL

# UPDATE (CCV tier, sparse, D=8): 1707 bytes
# macros in CAPITALS are defined in MACROS_CCV.txt; raw opcodes are wrapped and indented

<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_2 OP_PICK 40 32 OP_SUBSTR
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY OP_SWAP OP_SIZE 116
  OP_EQUALVERIFY
LEAF_POS
OP_ROT
LEAF_POS
OP_ROT OP_EQUALVERIFY OP_SWAP
LEAF_POS
SPARSE_SETUP
<TAG_NODE> OP_ROT OP_ROT
LEAF_ACC
OP_ROT
LEAF_ACC
LEVEL_S(k)  for k = 0 to 7
CHECK_OLD_S
FINAL
```

D.13 CCV Tier, 0xC0 Numerics, Dense Profile ($D = 4$)

```
# CLAIM (CCV with 0xC0 numerics, in MID macros, dense, D=4): 2316 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

33 OP_PICK 33 OP_PICK 33 OP_PICK 33 OP_PICK 33 OP_PICK 33 OP_PICK 33 OP_PICK
  OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT 24 OP_PICK 24 OP_PICK 24 OP_PICK
  24 OP_PICK 24 OP_PICK 24 OP_PICK 24 OP_PICK OP_CAT OP_CAT OP_CAT OP_CAT
  OP_CAT OP_CAT OP_CAT <TREE_ID> OP_SWAP OP_CAT
TAGGED(VOUCHER)
32 OP_PICK OP_CHECKSIGFROMSTACK OP_VERIFY
CARVE_RULES
```

```
20 OP_PICK OP_7 OP_PICK OP_7 OP_ROLL OP_7 OP_ROLL
PADZ
OP_EQUALVERIFY OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK
  OP_TOALTSTACK
LEAF_JOIN
OP_TOALTSTACK OP_8 OP_PICK OP_8 OP_PICK OP_8 OP_PICK OP_8 OP_PICK OP_8
  OP_PICK OP_CAT OP_CAT OP_CAT OP_CAT OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK OP_DUP OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_6 OP_PICK OP_TOALTSTACK OP_5 OP_PICK OP_TOALTSTACK OP_6
  OP_ROLL OP_DROP OP_7 OP_ROLL OP_6 OP_ROLL OP_SUB OP_DUP OP_0
  OP_GREATERTHANOREQUAL OP_VERIFY OP_DUP OP_3 OP_ROLL OP_3 OP_ROLL
LE64
OP_4 OP_ROLL OP_1ADD OP_4 OP_ROLL OP_4 OP_ROLL
PADZ
OP_3 OP_ROLL OP_SWAP OP_CAT OP_CAT
TAGGED(LEAF)
OP_SWAP
DRAIN_RULE
<TAG_NODE> OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL
LEVEL_MID  x4
OP_2SWAP OP_6 OP_ROLL OP_6 OP_ROLL
LE64
<SEL_IN_AMOUNT> OP_TX OP_EQUALVERIFY 129 OP_0 129 129 OP_CHECKCONTRACTVERIFY
  OP_FROMALTSTACK OP_FROMALTSTACK
TAGGED(LEAF)
OP_SWAP OP_2SWAP OP_SWAP OP_TOALTSTACK OP_TOALTSTACK <EMPTY_0> OP_0 OP_2SWAP
LEVEL_MID  x4
CHECK_PREV
OP_ROT OP_DROP OP_OVER <EMPTY_4> OP_EQUAL OP_IF OP_2DROP OP_2DROP OP_1
  OP_ELSE OP_3 OP_ROLL OP_3 OP_ROLL
LE64
OP_TOALTSTACK OP_0 OP_0 129 OP_0 OP_CHECKCONTRACTVERIFY OP_FROMALTSTACK OP_0
  <SEL_OUT_SINGLE> OP_TX OP_DROP OP_EQUALVERIFY OP_1 OP_ENDIF

# CLAIM_H (CCV with 0xC0 numerics, in MID macros, dense, D=4): 2324 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

OP_SIZE 32 OP_EQUALVERIFY OP_SHA256 OP_TOALTSTACK 33 OP_PICK 33 OP_PICK 33
  OP_PICK 33 OP_PICK 33 OP_PICK 33 OP_PICK 33 OP_PICK OP_CAT OP_CAT OP_CAT
  OP_CAT OP_CAT OP_CAT 24 OP_PICK 24 OP_PICK 24 OP_PICK 24 OP_PICK 24 OP_PICK
  24 OP_PICK 24 OP_PICK OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT
  <TREE_ID> OP_SWAP OP_CAT OP_FROMALTSTACK OP_CAT <TAG_VOUCHER_H> OP_SWAP
  OP_CAT OP_SHA256 32 OP_PICK OP_CHECKSIGFROMSTACK OP_VERIFY
CARVE_RULES
20 OP_PICK OP_7 OP_PICK OP_7 OP_ROLL OP_7 OP_ROLL
PADZ
OP_EQUALVERIFY OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK
  OP_TOALTSTACK
LEAF_JOIN
OP_TOALTSTACK OP_8 OP_PICK OP_8 OP_PICK OP_8 OP_PICK OP_8 OP_PICK OP_8
  OP_PICK OP_CAT OP_CAT OP_CAT OP_CAT OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK OP_DUP OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_6 OP_PICK OP_TOALTSTACK OP_5 OP_PICK OP_TOALTSTACK OP_6
  OP_ROLL OP_DROP OP_7 OP_ROLL OP_6 OP_ROLL OP_SUB OP_DUP OP_0
  OP_GREATERTHANOREQUAL OP_VERIFY OP_DUP OP_3 OP_ROLL OP_3 OP_ROLL
LE64
OP_4 OP_ROLL OP_1ADD OP_4 OP_ROLL OP_4 OP_ROLL
PADZ
OP_3 OP_ROLL OP_SWAP OP_CAT OP_CAT
TAGGED(LEAF)
OP_SWAP
DRAIN_RULE
```

```
<TAG_NODE> OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL
LEVEL_MID  x4
OP_2SWAP OP_6 OP_ROLL OP_6 OP_ROLL
LE64
<SEL_IN_AMOUNT> OP_TX OP_EQUALVERIFY 129 OP_0 129 129 OP_CHECKCONTRACTVERIFY
  OP_FROMALTSTACK OP_FROMALTSTACK
TAGGED(LEAF)
OP_SWAP OP_2SWAP OP_SWAP OP_TOALTSTACK OP_TOALTSTACK <EMPTY_0> OP_0 OP_2SWAP
LEVEL_MID  x4
CHECK_PREV
OP_ROT OP_DROP OP_OVER <EMPTY_4> OP_EQUAL OP_IF OP_2DROP OP_2DROP OP_1
  OP_ELSE OP_3 OP_ROLL OP_3 OP_ROLL
LE64
OP_TOALTSTACK OP_0 OP_0 129 OP_0 OP_CHECKCONTRACTVERIFY OP_FROMALTSTACK OP_0
  <SEL_OUT_SINGLE> OP_TX OP_DROP OP_EQUALVERIFY OP_1 OP_ENDIF

# SPLIT (CCV with 0xC0 numerics, in MID macros, dense, D=4): 2157 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

31 OP_PICK OP_CHECKSIGVERIFY
CARVE_RULES
20 OP_PICK OP_7 OP_PICK OP_7 OP_ROLL OP_7 OP_ROLL
PADZ
OP_EQUALVERIFY OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK
  OP_TOALTSTACK
LEAF_JOIN
OP_TOALTSTACK OP_8 OP_PICK OP_8 OP_PICK OP_8 OP_PICK OP_8 OP_PICK OP_8
  OP_PICK OP_CAT OP_CAT OP_CAT OP_CAT OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK OP_DUP OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_6 OP_PICK OP_TOALTSTACK OP_5 OP_PICK OP_TOALTSTACK OP_6
  OP_ROLL OP_DROP OP_7 OP_ROLL OP_6 OP_ROLL OP_SUB OP_DUP OP_0
  OP_GREATERTHANOREQUAL OP_VERIFY OP_DUP OP_3 OP_ROLL OP_3 OP_ROLL
LE64
OP_4 OP_ROLL OP_1ADD OP_4 OP_ROLL OP_4 OP_ROLL
PADZ
OP_3 OP_ROLL OP_SWAP OP_CAT OP_CAT
TAGGED(LEAF)
OP_SWAP
DRAIN_RULE
<TAG_NODE> OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL
LEVEL_MID  x4
OP_2SWAP OP_6 OP_ROLL OP_6 OP_ROLL
LE64
<SEL_IN_AMOUNT> OP_TX OP_EQUALVERIFY 129 OP_0 129 129 OP_CHECKCONTRACTVERIFY
  OP_FROMALTSTACK OP_FROMALTSTACK
TAGGED(LEAF)
OP_SWAP OP_2SWAP OP_SWAP OP_TOALTSTACK OP_TOALTSTACK <EMPTY_0> OP_0 OP_2SWAP
LEVEL_MID  x4
CHECK_PREV
OP_ROT OP_DROP OP_OVER <EMPTY_4> OP_EQUAL OP_IF OP_2DROP OP_2DROP OP_1
  OP_ELSE OP_3 OP_ROLL OP_3 OP_ROLL
LE64
OP_TOALTSTACK OP_0 OP_0 129 OP_0 OP_CHECKCONTRACTVERIFY OP_FROMALTSTACK OP_0
  <SEL_OUT_SINGLE> OP_TX OP_DROP OP_EQUALVERIFY OP_1 OP_ENDIF

# EVICT_SUBTREE_2 (CCV with 0xC0 numerics, in MID macros, dense, D=4): 1897 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_1 OP_SWAP OP_IF OP_TOALTSTACK OP_2
  OP_PICK OP_CHECKLOCKTIMEVERIFY OP_DROP
PADZ
OP_6 OP_PICK OP_EQUALVERIFY OP_6 OP_PICK <OPERATOR_KEY> OP_EQUALVERIFY
  OP_FROMALTSTACK OP_DUP OP_1ADD OP_TOALTSTACK OP_DUP OP_TOALTSTACK OP_0
  OP_SWAP OP_10 OP_PICK OP_0 OP_2 OP_CHECKCONTRACTVERIFY OP_FROMALTSTACK
```

```
  <SEL_OUT_SINGLE> OP_TX OP_DROP OP_10 OP_PICK OP_EQUALVERIFY OP_9 OP_PICK
  OP_1 OP_CAT OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK OP_FROMALTSTACK OP_ROT OP_SWAP OP_CAT OP_FROMALTSTACK OP_ROT
  OP_TOALTSTACK OP_SWAP OP_TOALTSTACK OP_ELSE OP_0 OP_TOALTSTACK <E_0.sc>
  OP_TOALTSTACK OP_ENDIF OP_SWAP OP_IF OP_TOALTSTACK OP_2 OP_PICK
  OP_CHECKLOCKTIMEVERIFY OP_DROP
PADZ
OP_6 OP_PICK OP_EQUALVERIFY OP_6 OP_PICK <OPERATOR_KEY> OP_EQUALVERIFY
  OP_FROMALTSTACK OP_DUP OP_1ADD OP_TOALTSTACK OP_DUP OP_TOALTSTACK OP_0
  OP_SWAP OP_10 OP_PICK OP_0 OP_2 OP_CHECKCONTRACTVERIFY OP_FROMALTSTACK
  <SEL_OUT_SINGLE> OP_TX OP_DROP OP_10 OP_PICK OP_EQUALVERIFY OP_9 OP_PICK
  OP_1 OP_CAT OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK OP_FROMALTSTACK OP_ROT OP_SWAP OP_CAT OP_FROMALTSTACK OP_ROT
  OP_TOALTSTACK OP_SWAP OP_TOALTSTACK OP_ELSE OP_0 OP_TOALTSTACK <E_0.sc>
  OP_TOALTSTACK OP_ENDIF OP_SWAP OP_IF OP_TOALTSTACK OP_2 OP_PICK
  OP_CHECKLOCKTIMEVERIFY OP_DROP
PADZ
OP_6 OP_PICK OP_EQUALVERIFY OP_6 OP_PICK <OPERATOR_KEY> OP_EQUALVERIFY
  OP_FROMALTSTACK OP_DUP OP_1ADD OP_TOALTSTACK OP_DUP OP_TOALTSTACK OP_0
  OP_SWAP OP_10 OP_PICK OP_0 OP_2 OP_CHECKCONTRACTVERIFY OP_FROMALTSTACK
  <SEL_OUT_SINGLE> OP_TX OP_DROP OP_10 OP_PICK OP_EQUALVERIFY OP_9 OP_PICK
  OP_1 OP_CAT OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK OP_FROMALTSTACK OP_ROT OP_SWAP OP_CAT OP_FROMALTSTACK OP_ROT
  OP_TOALTSTACK OP_SWAP OP_TOALTSTACK OP_ELSE OP_0 OP_TOALTSTACK <E_0.sc>
  OP_TOALTSTACK OP_ENDIF OP_SWAP OP_IF OP_TOALTSTACK OP_2 OP_PICK
  OP_CHECKLOCKTIMEVERIFY OP_DROP
PADZ
OP_6 OP_PICK OP_EQUALVERIFY OP_6 OP_PICK <OPERATOR_KEY> OP_EQUALVERIFY
  OP_FROMALTSTACK OP_DUP OP_1ADD OP_TOALTSTACK OP_DUP OP_TOALTSTACK OP_0
  OP_SWAP OP_10 OP_PICK OP_0 OP_2 OP_CHECKCONTRACTVERIFY OP_FROMALTSTACK
  <SEL_OUT_SINGLE> OP_TX OP_DROP OP_10 OP_PICK OP_EQUALVERIFY OP_9 OP_PICK
  OP_1 OP_CAT OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK OP_FROMALTSTACK OP_ROT OP_SWAP OP_CAT OP_FROMALTSTACK OP_ROT
  OP_TOALTSTACK OP_SWAP OP_TOALTSTACK OP_ELSE OP_0 OP_TOALTSTACK <E_0.sc>
  OP_TOALTSTACK OP_ENDIF OP_DROP <TAG_NODE> OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK
EVICT_HASH_STEP(4)
OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
EVICT_HASH_STEP(6)
EVICT_HASH_ROOT(4)
<EMPTY_2> OP_0
LEVEL_MID  x2
OP_2SWAP OP_6 OP_ROLL OP_6 OP_ROLL
LE64
<SEL_IN_AMOUNT> OP_TX OP_EQUALVERIFY 129 OP_0 129 129 OP_CHECKCONTRACTVERIFY
  OP_ROT OP_DROP OP_OVER <EMPTY_4> OP_EQUAL OP_IF OP_2DROP OP_2DROP OP_1
  OP_ELSE OP_3 OP_ROLL OP_3 OP_ROLL
LE64
OP_TOALTSTACK OP_0 OP_0 129 OP_0 OP_CHECKCONTRACTVERIFY OP_FROMALTSTACK OP_0
  <SEL_OUT_SINGLE> OP_TX OP_DROP OP_EQUALVERIFY OP_1 OP_ENDIF
```